VECTOR

vBaseEnv

User Manual

How to use Vector Base Environment

Version 2.30.0

Authors	visbwa, virmzi, visjhr, vismaa
Status	Released

Document Information

History

Author	Date	Version	Remarks
visbwa	2018-10-08	0.5.0	Initial creation
virmzi	2018-10-11	0.5.1	Added program flow
virmzi	2018-10-16	0.5.2	Added chapter for preconfig concept
visbwa	2018-10-19	1.0.0	Finalized and released document (part of vBaseEnv 1.01.00)
virmzi	2018-10-23	1.0.1	Added chapter ALM storage and dependencies concept
virmzi	2018-11-06	1.0.2	Added chapter MCAL sample configuration
visbwa	2018-11-19	1.1.0	Added chapters vBRS Testsuite and module stubs (part of vBaseEnv 1.02.00)
visbwa	2019-01-29	1.2.0	Reworked program flow charts and preconfig description, introduced OS stub (part of vBaseEnv 1.03.00)
visbwa	2019-03-18	1.3.0	Updated vBRS parameters and EcuM+SchM Stub description (part of vBaseEnv 1.04.00)
visbwa	2019-07-12	1.4.0	Updated program flow of BrsMainStartup and BrsMain
visbwa	2019-08-14	1.5.0	Added description of vBRS DriverHandling settings for CAN / LIN / Flexray / Ethernet, description for new FBL UseCase parameter and vBRSMemoryInit container. Added chapters for vDrvCanStub and vDrvLinStub (part of vBaseEnv 1.07.00)
visbwa	2019-10-24	1.6.0	Added description of vBaseEnv derivative and sub-family specific parameters (chapter 2.3.x). Removed Dem_MemMap.h from vBRS stubs list (part of vBaseEnv 1.08.00)
visbwa	2019-12-11	1.7.0	Updated Program Flow chapter to fit with changes of vBaseEnv 1.9.0. Added chapters 2.5.1 (Memory Region mapping) and 3.1 (vBRS Setup Support).
visbwa	2020-02-24	1.8.0	Updates for vBaseEnv 1.10.0: • HSM support (added chapter 3.2.4, updated Program Flow) • Added static CanSM stub
visbwa	2020-03-20	1.9.0	Updates for vBaseEnv 1.11.0: • Updated SchM stub description • Updated FBL UseCases (SchM stub)
visbwa	2020-05-11	1.10.0	Updates for vBaseEnv 1.12.0: • Added a hint for FBL UseCase WithoutOSScheduling
visbwa	2020-06-09	2.0.0	Updates for vBaseEnv 2.0 • Updated program flow • Removed chapter “ALM storage and dependencies concept” • Removed chapter “Preconfig Concept” • Added chapter for BrsMain
visbwa	2020-07-06	2.1.0	Updates for vBaseEnv 2.01 • Added description for new BRS feature “prefer Wdg & PLL initialization” • Updated program flow
visbwa	2020-07-14	2.1.1	Small fix in program flow (BrsMainInit())
visbwa	2020-09-23	2.1.2	Added example code disclaimer
visbwa	2020-10-22	2.2.0	Updates for vBaseEnv 2.05 • Introduction of new FBL UseCases

visbwa	2020-11-23	2.3.0	Updates for vBaseEnv 2.06 • Postbuild selectable support of vBrs EcuM Stub
visbwa	2021-02-15	2.4.0	Updates for vBaseEnv 2.08 • added new BrsHwConfig nodes for I2c and vHyperbus • vBrs SchM stub updates (schedulable entities)
visbwa	2021-04-13	2.5.0	Updates for vBaseEnv 2.09 • support for OS UseCase “Interrupt only” • added vBrs DriverHandlingSpi
visbwa	2021-07-05	2.6.0	Updates for vBaseEnv 2.12 • Removed vBaseEnv interrupt handling • Reworked vBrs generated files description • Added description for new vBrs functionality “enable core specific vLinkGen config validation” • Updated program flow of main()
visbwa	2021-10-26	2.7.0	Updates for vBaseEnv 2.16 • Added 3.3.4 Startup Stack Settings • Updated vBRSGeneral description • Exchanged the memory init stages “HardReset” and “One” inside BrsMainStartup • Updated program flow, added samples
visbwa	2021-11-17	2.8.0	Updates for vBaseEnv 2.17 • New chapter 5 for BrsHw • New vBRS parameter “FBL/BM Exception Table in RAM” • Rework of FBL program flows
visbwa	2021-11-26	2.8.1	Small fixes for FBL/BM Exception Table in RAM
visbwa	2021-12-16	2.9.0	Updates for vBaseEnv 2.18 • Added description for new included module vSet, reworked description for module EcuC • Reworked description of vBaseEnv derivative information • Reworked description of vLinkGen memory region mapping to better fit with actual vLinkGen
visbwa	2022-02-16	2.10.0	Updates for vBaseEnv 2.20 • Updated EcuM Stub description with new override feature • Updated vLinkGen chapter with new default memory region layout
visbwa	2022-03-15	2.11.0	Updates for vBaseEnv 2.21 • Added description for new default MemoryRegionBlock Variables_Shared
visbwa	2022-04-11	2.12.0	Updates for vBaseEnv 2.22 • Renamed Microsar to MICROSAR Classic • Updated chapter 4.1 vBRS Setup Support
visbwa	2022-10-13	2.13.0	Updates for vBaseEnv 2.29 • Updated SchM Stub description with new empty exclusive area handling feature
visbwa	2022-11-28	2.14.0	Updates for vBaseEnv 2.31 • Introduced new static files BrsMain_Appl.c/.h • Introduced FBL PresencePattern

visbwa	2023-01-16	2.15.0	Updates for vBaseEnv 2.32 • Introduced new parameters vBRSVariantUsageOfvLinkGen and vBRSProjectName • Added description for vLinkGen Variants
visjhr, visbwa	2023-03-09	2.16.0	Updates for vBaseEnv 2.34 • Added chapter 5 “Configuration” from the vLinkGen TechRef as chapter 8 “vLinkGen” • Introduced BrsHwApplicationInit(), updated all program flows
visbwa	2023-04-06	2.17.0	Updates for vBaseEnv 2.35 • Introduced new execution instance “FlashDriver” • Introduced fix mapping of Brs_ApplicationEntry() via new memory region block “ApplicationEntry”
visbwa	2023-06-02	2.18.0	Updates for vBaseEnv 2.37 • Introduced new execution instance “HSM_Updater”
visbwa	2023-07-25	2.19.0	Updates for vBaseEnv 2.39 • Introduced Small Devices optimization • BrsMainTask changed to basic task, alarm set by InitTask
visbwa	2023-10-23	2.20.0	Updates for vBaseEnv 2.42 • Updated SchM stub description for support of new Can_30_Core/vCan drivers
visbwa	2023-12-14	2.21.0	Updates for vBaseEnv 2.44 • Introduction of vBRSDriverHandlingCAN_Instance and the possibility to configure 2 DrvCan instances in parallel
visbwa	2024-01-11	2.22.0	Updates for vBaseEnv 2.45 • Introduction of vBRSDriverHandlingEthernet_Instance and the possibility to configure 2 DrvEth instances in parallel
visbwa	2024-07-02	2.23.0	Updates for vBaseEnv 2.51 • Reworked chapter 4.12 vBRSMemoryInit • Introduced chapter 6.1.1 Memory Initialization • Removed description of outdated BrsHw_InterruptHandling.txt
visjhr	2024-07-16	2.24.0	Updated for vBaseEnv 2.52 • Added chapter 8.5 for optional core local addressing parameter vBaseEnvHwRegionCoreLocalAddressing
visbwa	2024-08-28	2.25.0	Updated for vBaseEnv 2.53 • Added chapter 6.1.3 for new init stage POWER_ON, updated program flows
visbwa	2024-09-16	2.26.0	Updated for vBaseEnv 2.54 • Added module MemMap to general vBaseEnv description • Adapted vSet and EcuC descriptions with information about vSetBswInitialization • Adapted EcuM Stub description, as vSetBswInitialization is used now • Added chapter 4.5 Hypervisor support, updated program flow • Reworked chapter 3.4.1 “Derivative Information”, added description for memory layout parameters
visbwa	2024-11-11	2.27.0	Updated for vBaseEnv 2.56 • Added hints for new DrvEthSwitch support • Added hint for simple “cloning” of derivative configurations

visbwa	2025-02-06	2.28.0	Updated for vBaseEnv 2.59 • Introduced BrsMain_Cfg.h • Updated Hypervisor description for Hypervisor MultiCore support • Added program flow sample for Hypervisor host instance • Updated vBRSTestsuiteSupport • Added new API “BrsHw_StartCore” • Added 64-Bit Init Pattern for memory initialization in vBRS
visbwa	2025-03-11	2.29.0	Updated for vBaseEnv 2.60 • Description of the new vLinkGen default variants (chapter 3.6.3)
vismaa	2025-03-26	2.30.0	Updated for vBaseEnv 2.61 • Added possibility to disable memory init globally

Contents

1 Disclaimer 9
1.1 Example code only 9

2 Introduction 10

3 Environment in Configurator 5 11
3.1 EcuC 11
3.2 vSet 12
3.3 OS 13
3.4 vBaseEnv 14
3.4.1 Derivative information 14
3.4.2 Memory Layout Hw Regions 16
3.4.3 Peripheral Clocks 16
3.4.4 Startup Stack Settings 17
3.5 vBRS 18
3.6 vLinkGen 19
3.6.1 Memory Region mapping 19
3.6.2 Default MemoryRegionBlocks 20
3.6.3 Default Variants 23
3.7 MemMap 25

4 vBRS 26
4.1 vBRS Setup Support 27
4.1.1 OS tasks creation 28
4.1.2 OS SystemTimer calculation 28
4.1.3 Core specific vLinkGen settings 28
4.1.4 Additional consistency checks 28
4.2 vBRSGeneral 29
4.2.1 Small Devices optimization 32
4.3 FBL support 34
4.3.1 FBL UseCases 35
4.3.2 vLinkGen Variants 36
4.3.3 FlashDriver 37
4.4 HSM support 37
4.4.1 HSM Updater 37
4.5 Hypervisor support 38
4.5.1 Hypervisor Guest 38
4.6 EcuM Stub 39
4.6.1 EcuM Stub override feature 40

4.7 SchM Stub 42
4.8 vBRSHwConfig 43
4.8.1 DriverHandlingCAN 45
4.8.2 DriverHandlingLIN 47
4.8.3 DriverHandlingFlexray 48
4.8.4 DriverHandlingEthernet 49
4.8.5 DriverHandlingI2C 51
4.8.6 DriverHandlingHyperbus 52
4.8.7 DriverHandlingSpi 52
4.9 vBRSTestsuiteSupport 53
4.9.1 Testsuite initialization and cyclic calls 54
4.10 vBRS module stubs 55
4.10.1 vBRS module stubs callout concept 57
4.11 vDrvCanStub 58
4.12 vDrvLinStub 59
4.13 vBRSMemoryInit 60

5 BrsHw 61
5.1 Startup code 61
5.2 Hardware initialization 62
5.2.1 BrsHwEarlyInitPowerOn 62
5.2.2 BrsHwApplicationInit 62
5.2.3 BrsHwPreInitPowerOn 62
5.2.4 BrsHwWatchdogInitPowerOn 62
5.2.5 BrsHwPllInitPowerOn 62
5.2.6 BrsHwPortInitPowerOn 63
5.2.7 BrsHwDisableInterruptAtPowerOn 63
5.2.8 BrsHw_ExceptionTable_Init 63
5.2.9 BrsHwSoftwareResetECU/BrsHwGetResetReasonStartup 63
5.2.10 BrsHwTime100NOP 63
5.2.11 BrsHw_EnableHwAccess 63
5.2.12 BrsHw_GetCore 63
5.2.13 BrsHw_EnableInterrupt / BrsHw_DisableInterrupt / BrsHw_TriggerSoftwareInterrupt 63
5.2.14 BrsHwDisableEccErrorReporting 64
5.2.15 BrsHw_UnlockCores 64
5.2.16 BrsHw_StartCore 64
5.3 Core exception table 64
5.4 MakeSupport 64

6 BrsMain 65

6.1 BrsMainStartup 65
6.1.1 Memory Initialization 66
6.1.2 HardReset 66
6.1.3 Power_On 67
6.2 BrsMain_Callout_Stubs 67
6.3 BrsMain 67
6.3.1 BrsMainInit 67
6.3.2 BrsMainTogglePin 67
6.3.3 Default Tasks 68
6.3.4 BrsMainExceptionHandler 68

7 Program Flow 69
7.1 Sample for MSR4 73
7.2 Sample for FBL 74
7.2.1 FBL without BM 74
7.2.2 FBL with BM 75
7.3 Sample for HSM 78
7.3.1 HSM Updater 79
7.4 Sample for CANbedded 81
7.5 Sample for Hypervisor host instance 82

8 vLinkGen 83
8.1 Linking of Linker Sections 83
8.1.1 Code or Constant Section 83
8.1.2 Data or Variable Section 84
8.1.3 Sections with Special Values 85
8.2 Memory initialization Initialization of ECC Protected Memory 86
8.2.1 Initialization of complete memory region blocks 86
8.2.2 Initialization of VAR Section Group 86
8.2.3 Copy Code to RAM 87
8.3 User Defined Linker Symbols 88
8.4 Variant Handling 89
8.4.1 Variant Specific Region Block Mappings 89
8.5 Using Hardware Regions with Core Local Addressing 91

9 MCAL sample configuration 92

1 Disclaimer



Caution
All application code in any of the Vector User Manuals is for training purposes only. They are slightly tested and designed only to understand the basic idea of using a certain component or set of components. Vector gives consent that you use it as a basis for developing your own code and modify it (Vector waives its copyright to forbid you the use), however, with regard to the fact that the code is designed for training purposes only and not for the use by you, usability of the code is not warranted and Vector's liability shall be expressly excluded in cases of normal negligence, to the extent admissible by law or statute. Of course, you must test the software developed on the basis of the code with diligent care before using the software.

1.1 Example code only

All included source code files contain the following disclaimer:

This Example Code is only intended for illustrating an example of a possible BSW integration and BSW configuration. The Example Code has not passed any quality control measures and may be incomplete. The Example Code is neither intended nor qualified for use in series production. The Example Code as well as any of its modifications and/or implementations must be tested with diligent care and must comply with all quality requirements which are necessary according to the state of the art before their use.

2 Introduction

The Vector Base Environment contains a package of modules, necessary to get a microcontroller platform running.

All modules together provide everything that is necessary to get the microcontroller project

- Compiled
- Linked
- Started
- Initialized

In the end, a cyclic runtime behavior is granted. Especially for Bootloader projects, it is also possible to configure several execution instances, which will be flashed in parallel on the controller.

The vBaseEnv package can be used as single package for HW provision tests or driver developments, or as part of a complete embedded software project.

The vBaseEnv is based on DaVinci Configurator 5 as configuration tool. The necessary configuration files are generated out of the Configurator 5. The modules itself will help the user via so called Solving Actions to configure the environment for a “straight forward” system configuration.

The following modules are always part of a vBaseEnv package:

- EcuC (Autosar EcuC module)
- MemMap
- OS (Vector MICROSAR Classic Operating System) – except FBL and HSM Updater projects, where no OS is used
- vBaseEnv (Vector Base Environment)
- vBRS (Vector Basic Runtime System)
- vLinkGen (Vector Linkerscript Generator)
- vSet (Vector Common Settings module)

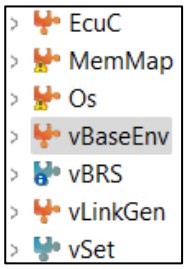
The modules EcuC, MemMap and vSet are normally completely platform and compiler specific preconfigured. Nothing needs to be configured inside there manually.

On some platforms, also a 3rd-party MCAL, provided by the microcontroller vendor, could be part of the vBaseEnv.

The whole environment is microcontroller-platform and compiler specific. The correct packages and files need to be provided project specific.

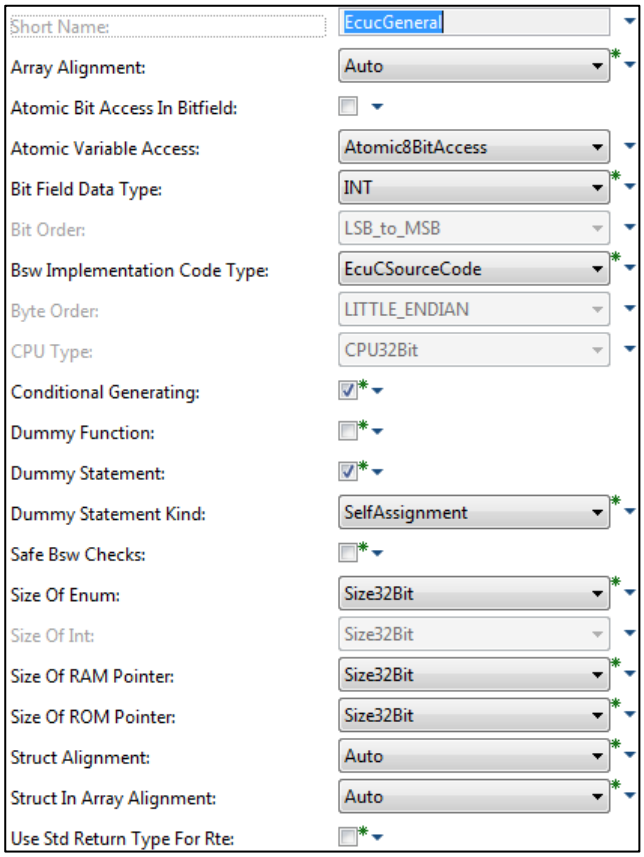
3 Environment in Configurator 5

For a successful usage of a vBaseEnv environment, the following modules must be enabled at least:

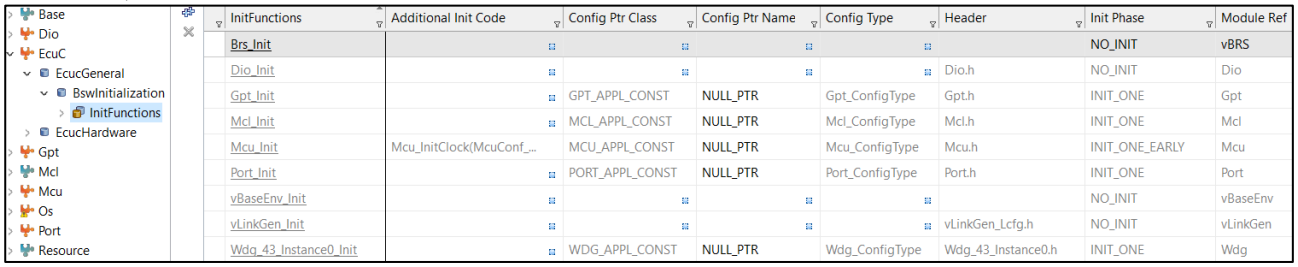


3.1 EcuC

The Autosar EcuC module consists of generic platform information, like e.g., the CPU type (e.g., 32-bit). If the vBaseEnv package of your platform was modelled correct, nothing should be needed to be modified in here, every value should have a meaningful default value:

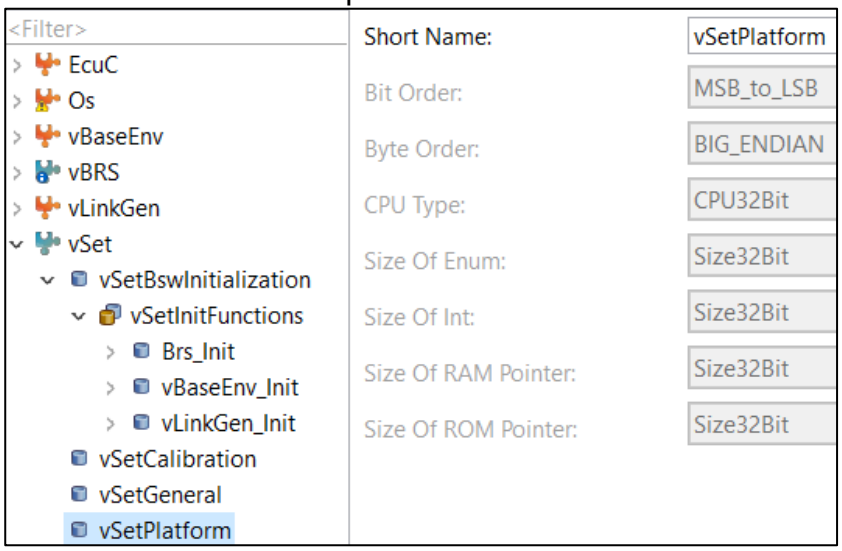


Additionally, the EcuC module includes the initialization settings of the configuration included Autosar modules. Every module configures its mandatory initialization API and the suitable init phase inside the EcuC BswInitialization container.
This information is not used by the vBrs EcuM stub anymore (was changed with vBaseEnv 2.54), as this information is meanwhile configured within the vSet module. In future MSR releases, this information will be removed from EcuC module.



3.2 vSet

Up to now, the vSet module is containing parameters, which are mainly inherited from the EcuC module. In the future, Vector-own parameters, which are not specified by the Autosar standard to be included in the EcuC module, will move completely from the EcuC module into the Vector specific vSet module.

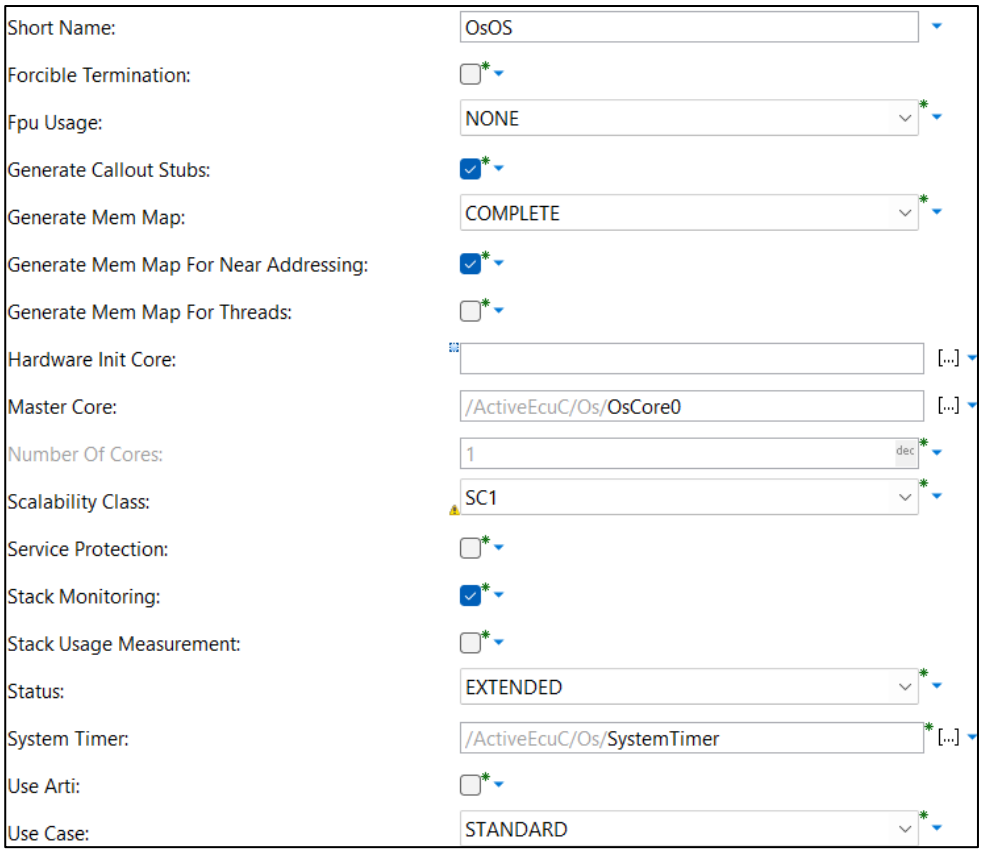


The module initialization settings are one of these Vector-own parameters. Every module configures its mandatory initialization API and the suitable init phase in parallel to the EcuC module also inside the vSetBswInitialization container.
This information is also used by the vBrs EcuM stub, which will generate the initialization sequence according to these settings.

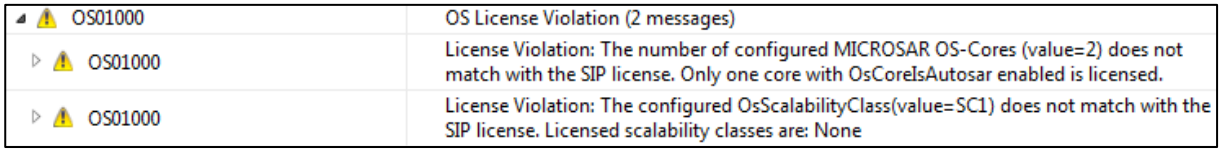
© 2025 Vector Informatik GmbH

3.3 OS

The Autosar Operating System is responsible for the interrupt handling and the timer handling. If the vBaseEnv package of your platform was modelled correctly, nothing should be needed to be modified in here, every value should have a meaningful default value:



If your SIP does not contain an OS license (because not ordered by customer), messages like the following are required and should be visible inside the Validation window:



These messages have no effect on the functionality of the OS, this is just a license limitation (the OS is not allowed for series production if no OS license is ordered).

3.4 vBaseEnv

Within the Base Environment module, the derivative for which the environment should be configured for, must be chosen. The “Tested Derivative” could be different to the “Derivative”, configured in the Cfg5 “Project Settings”, as maybe a derivative with different RAM/ROM sizes than the explicit ordered derivative is used for development.

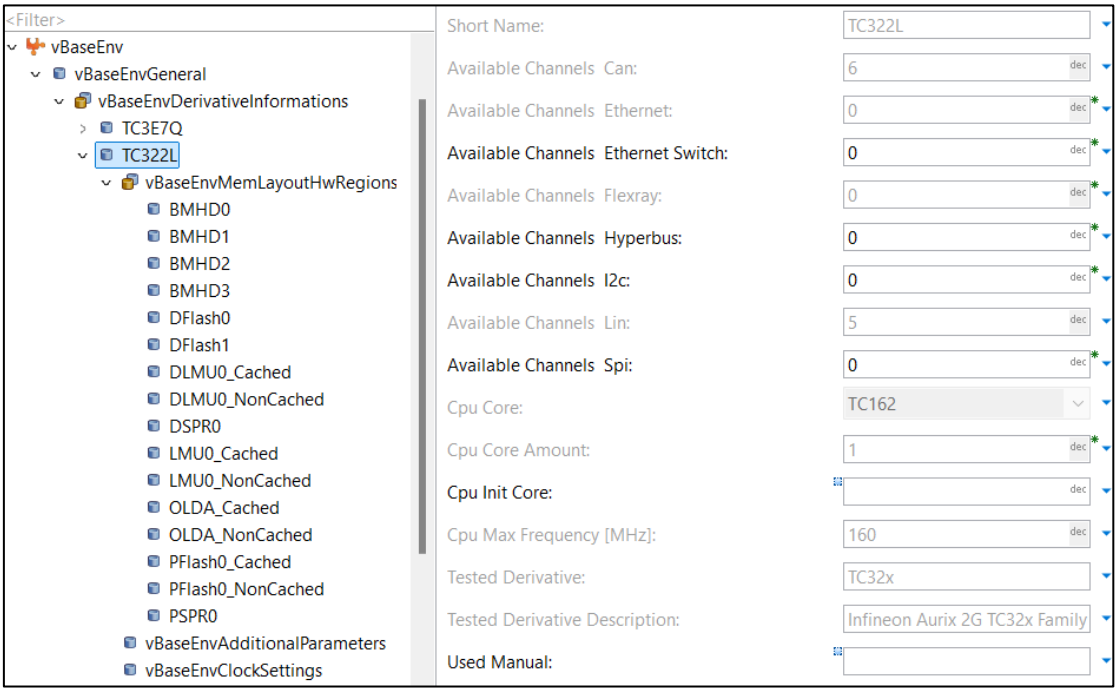


All derivative and platform specific information of the environment is placed in here. The other base modules get this information from here.

3.4.1 Derivative information

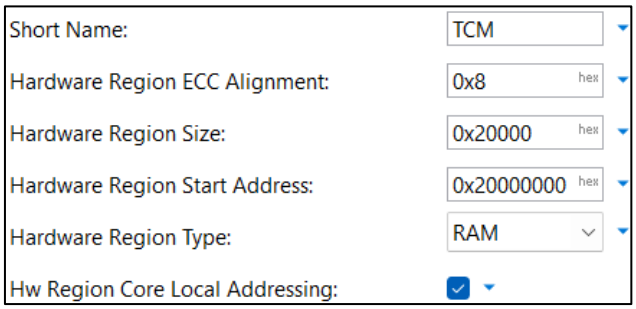
Contains the derivative specific information, which differ between the different derivatives of the controller platform. All supported derivatives of this specific platform are listed in here.

A platform is a group of derivatives, which are compatible in means of the used OS, compiler and startup environment. They are based on the same core architecture but could differ in the number of cores, memory regions and the equipped peripherals. E.g., they could differ in the size and amount of memory regions, the number of cores, the maximum frequencies, and the amount of communication channels. All these differences are configured in here.



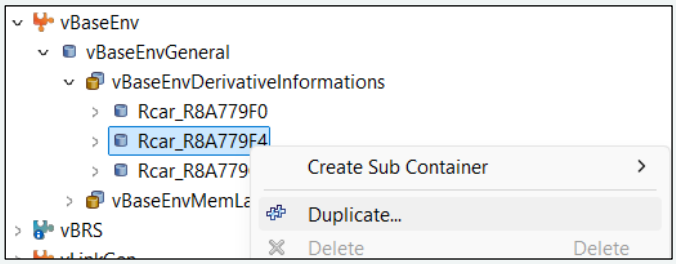
Also, typical frequencies for the oscillator and peripheral clocks of the specific derivative could be configured in the container vBaseEnvDerivativeInformation/ vBaseEnvClockSettings.

The MemLayoutHwRegions contain all memory areas, available on the derivative. Every area is configured with all information, needed by vLinkGen to provide a proper handling:



Parameter	Description
ECC Alignment	Required ECC Alignment of the hardware memory region. It is necessary for zero init functionality, e.g. native 64-bit writes with 8-byte aligned ECC RAM. This parameter is not mandatory.
Size	Size of the hardware memory region.
Start Address	Start address of the hardware memory region.
Type	Type of the hardware memory region. This parameter is necessary for some linkers (e.g. Tasking), to distinguish between RAM/ROM/NVRAM. This parameter is not mandatory.
Core Local Addressing	Configure, if this hardware memory region is accessible within a core local address range only. This parameter is necessary for memory, which is accessed via a core local address range (e.g. every core has core local RAM at the same address). vLinkGen will need this information to generate additional keywords into the linkerfile (e.g. OVERLAY), so the linker allows multiple usage of the same address range. See chapter 8.5 for details. This parameter is not mandatory.

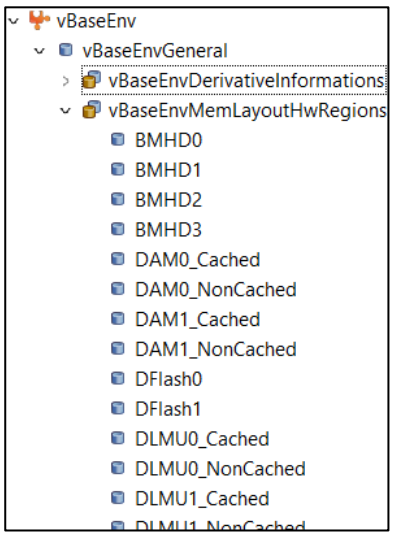
Note
If you need to change some of these mainly preconfigured (and for this reason greyed out) parameters (e.g. you are using a derivative different to the reference derivative of a VectorDE platform), it is possible to “clone” single derivatives. Right-click on the most suitable derivative and select “Duplicate”:



Within the duplicated derivative, all parameters will be editable. Change them according to your needs and change the “Tested Derivative” to this new created one.

3.4.2 Memory Layout Hw Regions

This container shows a list of all memory regions, which are existent with at least one derivative of the platform. This is just a list, to provide the information to vLinkGen. The specific locations and sizes of the regions are derivative specific. The vLinkGen module is checking automatically, which memory regions are existing for the actual selected “Tested Derivative” and selects the derivative specific sizes of them.



3.4.3 Peripheral Clocks

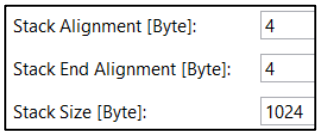
This container could show a list of default peripheral clocks for the supported communication devices (Can, Lin, Flexray, Ethernet). These values should fit to the default PLL initialization routine of the BrsHw package. If a MCAL is used, it is also possible to configure the dependent Mcu clock references in here. These settings are automatically used by the DriverHandling containers of the vBRS module.

© 2025 Vector Informatik GmbH

3.4.4 Startup Stack Settings

This container could show platform specific settings for the Startup Stack, that differ from the default settings. These parameters are used to configure the Startup stacks of all actual configured cores inside vLinkGen, if “Enable core spedific vLinkGen config validation” is enabled in vBRSGeneral. The core specific “LogicalVarGroup” STACK_Cx and the underlying “VarSectionGroup” are created by usage of these values.

If this container and the parameters do not exist for the actual platform, the default values are valid for this platform. The default values for the Startup stack of every core are:

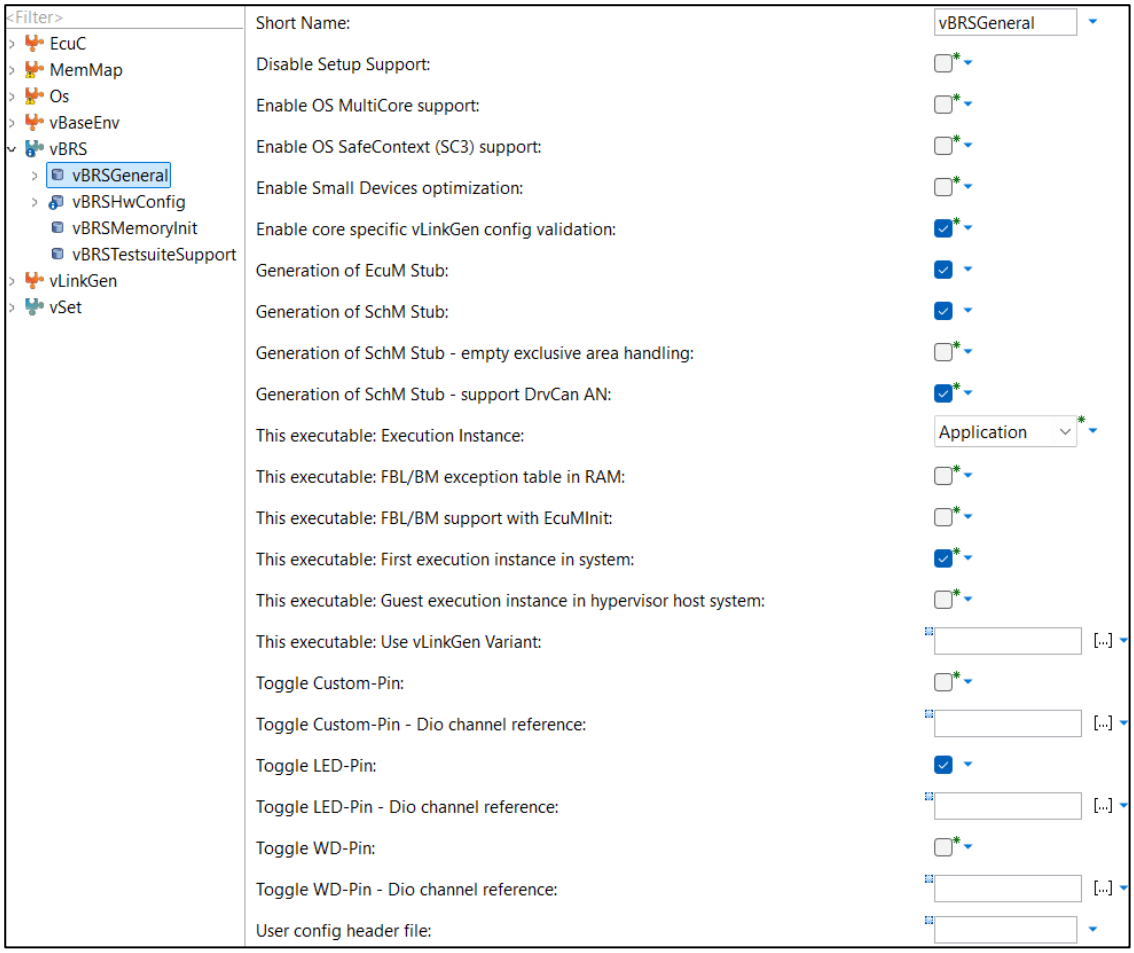


© 2025 Vector Informatik GmbH

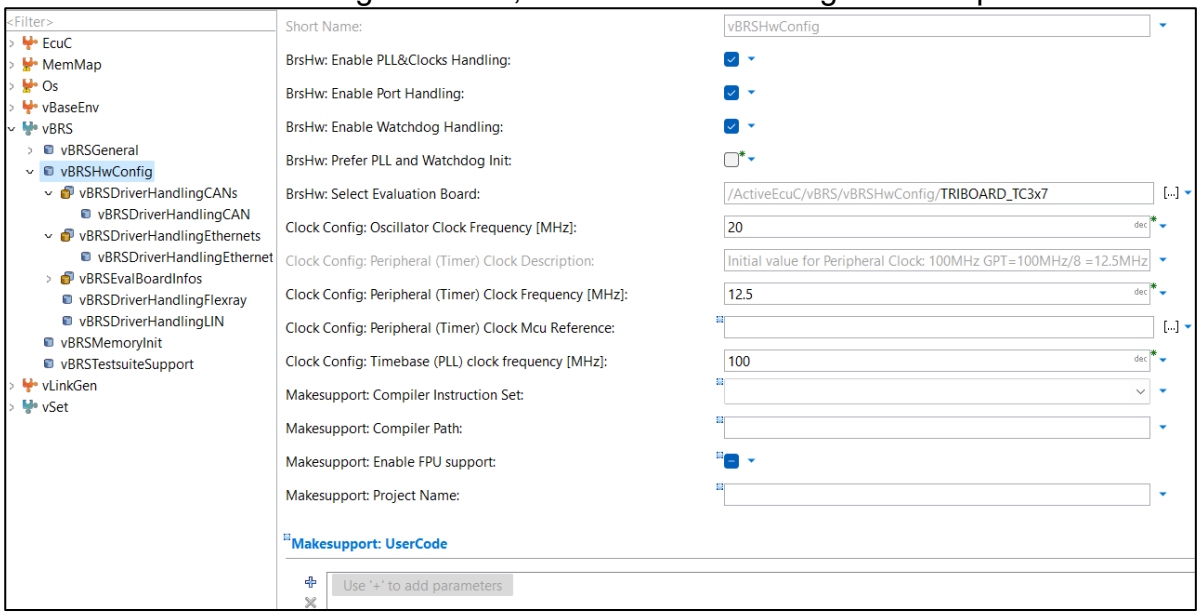
3.5 vBRS

The Vector Basic Runtime System is the main module to configure the base environment.

Within the vBRSGeneral container, some BRS-related additional features can be configured:



Within the vBRSHwConfig container, the HW related configuration is placed:



For more information about the vBRS, refer to chapter 4.

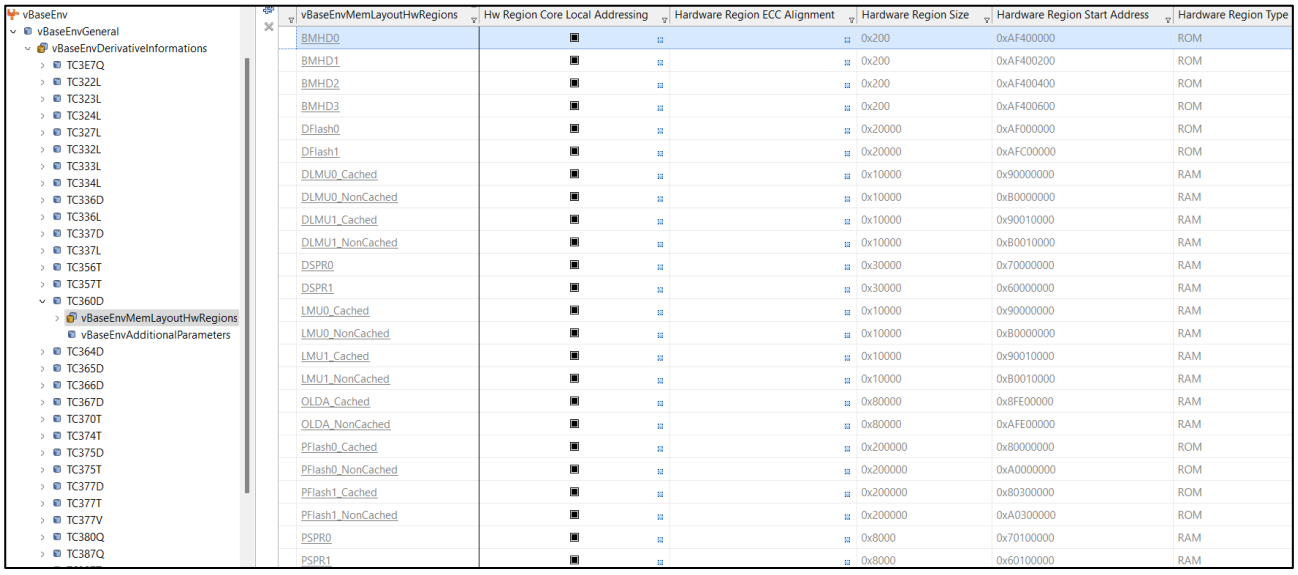
3.6 vLinkGen

The Vector Linker Script Generator is responsible for the generation of the linker script for your chosen “Tested Derivative” and the compiler of your project.

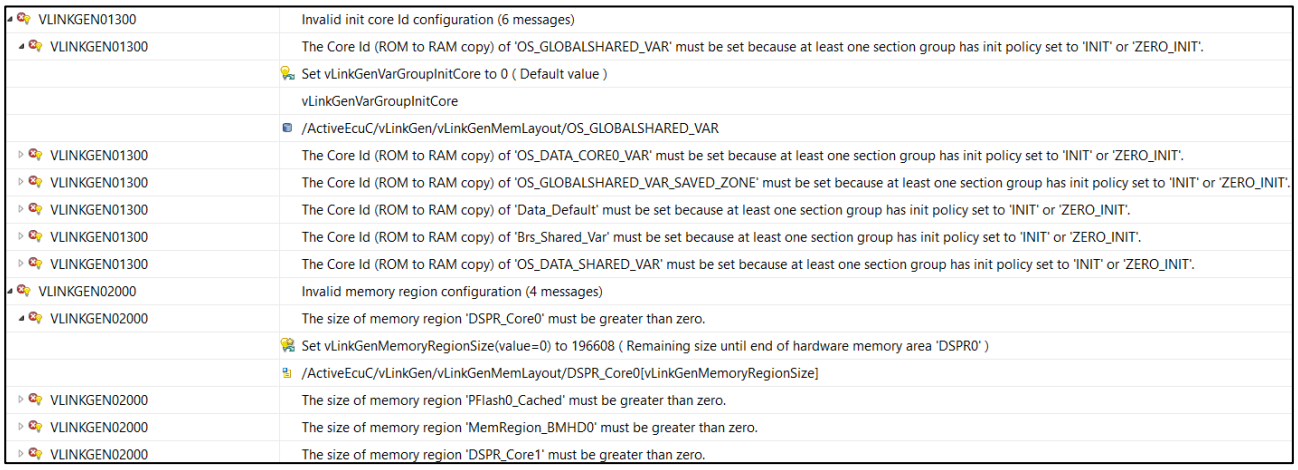
For more information about vLinkGen, refer to the Technical Reference of this module.

3.6.1 Memory Region mapping

Every vBaseEnv platform provides a default memory mapping. As the memory regions are derivative specific, the information of availability and sizes is taken out of the vBaseEnv container, dependent to the selected “Tested Derivative”:

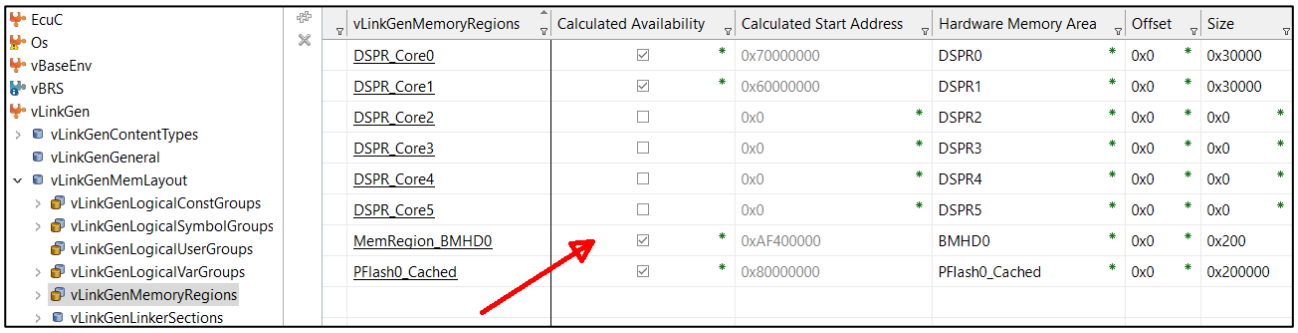


Solving actions transfer the values automatically (they are mainly also handled via “Solve All”):



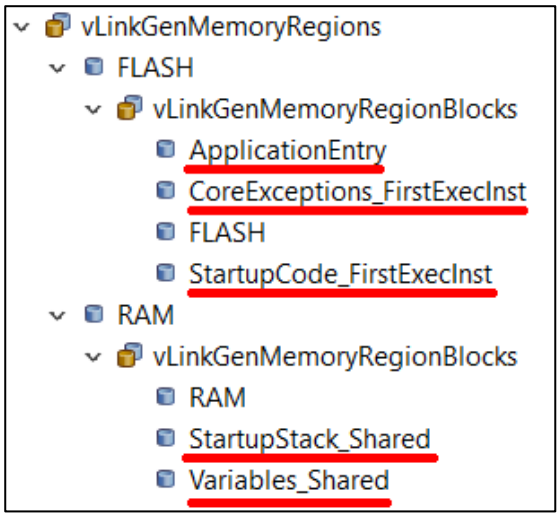
The default memory layout of the vBaseEnv platform family only uses a set of memory regions, which are needed for a basic default setup. This default memory layout is a superset layout, which configures e.g., the highest possible number of cores within the whole platform family (if core specific memory exists). vLinkGen is checking automatically, which memory regions are available for the selected “Tested Derivative” and sets the value

“Calculated Availability” accordingly. This ensures, that LogicalGroups are only mapped automatically into memory regions, which are currently available.

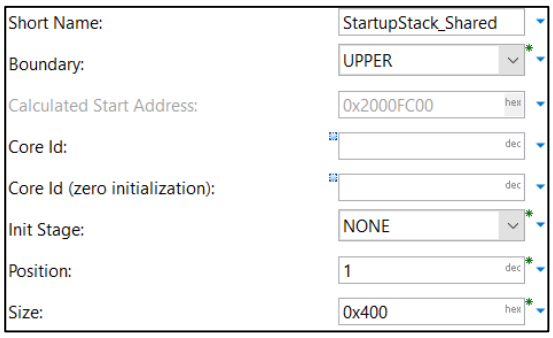


3.6.2 Default MemoryRegionBlocks

To ease the fragmentation of the memory, which is especially needed in projects with several execution instances (mainly Bootloader projects), there exist some default memory region blocks:



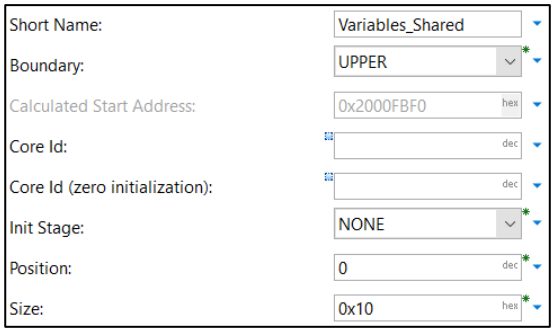
StartupStack_Shared is used for the stack, the stack pointer of the boot core is initialized with during startup (mainly used by the LogicalVarGroup “STACK_C0”). If several execution instances are in the project, this startup stack should be shared between the single execution instances (as the stack pointer is only initialized from the startup code of the 1st execution instance).



By usage of the “Boundary” parameter, it is always placed to the end of the RAM memory region, it is placed to. The default size is platform specific and changeable.

Note
If the used memory is ECC RAM, the “Init Stage” of the memory region block “StartupStack_Shared” needs to be configured as “EARLY”. The used memory is then initialized before the configuration of the stackpointer in startup code.

Variables_Shared is used for global variables, that need to be shared between several execution instances. It is mainly used to store brsMain_ResetReason, which is written in Brs_PreMainStartup() of the 1st execution instance to share the reset reason with the subsequent instances.



By usage of the “Boundary” parameter, it is always placed to the end of the RAM memory region, it is placed to, just before the startup stack. The default size is 16 Byte.

Note
If the used memory is ECC RAM, the “Init Stage” of the memory region block “Variables_Shared” needs to be configured as “EARLY”. The used memory is then initialized before brsMain_ResetReason is written.

ApplicationEntry is used for a fix mapping of the API Brs_ApplicationEntry() of BrsMainStartup.c. The LogicalConstGroup “Brs_ApplicationEntry” is automatically placed into it. If several execution instances are in the project, every execution instance will need its own mapping for this memory region.

The fix mapping of Brs_ApplicationEntry() is essential for projects with several execution instances, to allow the jump from one execution instance to another (for example a Boot Manager 1st execution instance, which needs to jump into an application instance).

CoreExceptions_FirstExecInst is used for the core exception table, which is initialized during startup. The LogicalConstGroup “Brs_ExcVect” is automatically placed into it (for implementation, see BrsHwIntTb.c). If several execution instances are in the project, this exception table is only needed to be mapped into the 1st execution instance. It stays also valid in additional execution instances, until it is overwritten via a call of BrsHw_ExceptionTable_Init() (e.g., to use the exception table of an operating system; the main() inside BrsMain.c is handling this automatically).



Note
An exception may be the FBL Updater, which could be start-able as first execution instance as well.

On platforms, where the location of the core exception table is essential for the correct startup/reset handling (e.g., Arm Cortex-M), the default memory layout grants the correct linkage of this core exception table via this specific memory region block.

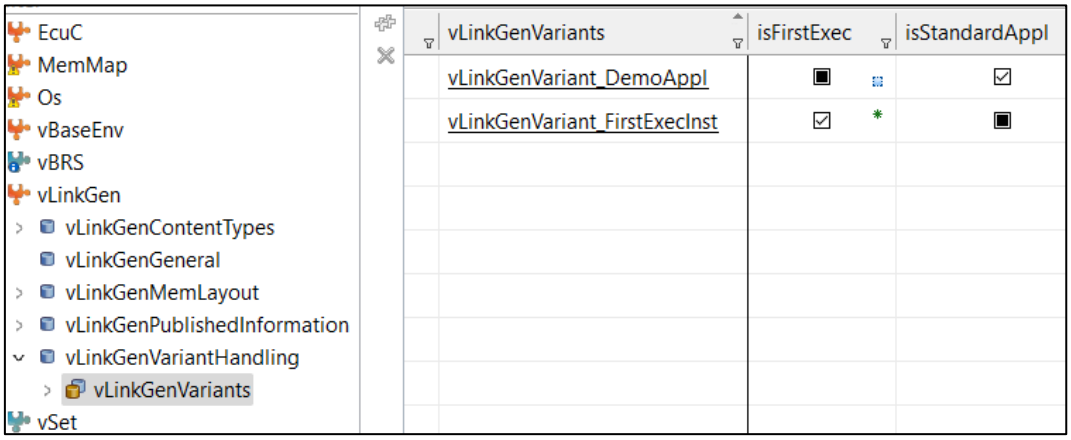
StartupCode_FirstExecInst is used for the startup code. The LogicalConstGroup “Brs_Startup_Code” is automatically placed into it (consists of the implementation from BrsHwStartup.c and Brs_PreMainStartup() of BrsMainStartup.c). If several execution instances are in the project, the startup code is only needed to be mapped into the 1st execution instance.

On platforms, where the location of the startup code is essential for the correct startup/reset handling (e.g., Infineon Aurix), the default memory layout grants the correct linkage of the startup code via this specific memory region block.

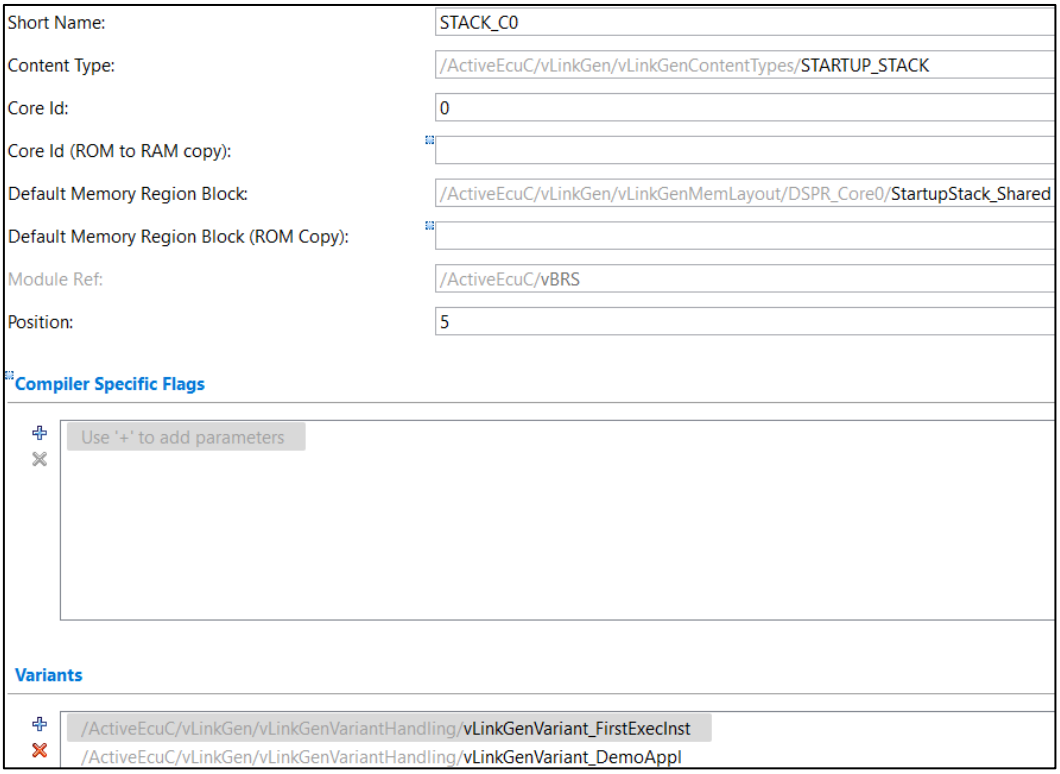
3.6.3 Default Variants

To ease the separation of several execution instances (e.g. a demo application behind a BootManager), the vBaseEnv is already containing of two default vLinkGen variants:

- **vLinkGenVariant_FirstExecInst** – 1st execution instance of the system (like e.g. BootManager)
- **vLinkGenVariant_DemoAppl** – "normal" application, located behind the 1st execution instance (like e.g. MSR4 SIP or FBL)



Via the flags "isFirstExec" and "isStandardAppl", the vBrs generator can be controlled. Variants, where at least one of these flags is activated, are automatically added to the StartupStack LogicalVarGroups (STACK_Cx):



Both default variants are selected for the StartupStacks, as it is shared between the several execution instances.

The suitable variants are already configured within default LogicalVarGroups and LogicalConstGroups of the vBaseEnv vLinkGen preconfiguration. As an example, the LogicalConstGroup “Brs_Startup_Code” has the variant “vLinkGenVariant_FirstExecInst” selected, because the StartupCode is only relevant for the 1st execution instance of the project. The LogicalVarGroup “Brs_Shared_Var” is used by several execution instances, that’s why both default variants are selected for it. Also, for HW-dependent sections like option bytes, the suitable variant is selected within the platform vLinkGen preconfiguration.

All this allows a sample configuration with two execution instances out-of-the-box. The mechanism can be updated or changed project specific. Also, the automatically created StartupStacks are controllable by setting of the depending on variant flags.



Note
The variants only take effect if the vLinkGen file generation mode (vLinkGenGeneral/vLinkGenFileGeneration) is set to “STANDARD_WITH_VARIANTS” or “ONE_FILE_PER_VARIANT”.

3.7 MemMap

The Memory Mapping (MemMap) module is the MICROSAR solution of the AUTOSAR memory mapping module. It is used to generate dedicated memory mapping header files for BSW modules as well as for software component types. These memory mapping header files contain all memory allocation keywords (derived from the MEMORY-SECTIONS) of a BSW module or software component type.

Memory allocation keywords are defined as preprocessor macros and always occur in pairs to open (START) and close (STOP) a memory section. Within these defines, compiler specific pragma statements are generated to map code, constants or variables into dedicated linker sections.

This module is only part of a vBaseEnv environment, as it could become mandatory by new versions of the OS. Nothing needs to be configured in here for the vBaseEnv mechanisms and the vBaseEnv modules are not using it actively. It will add several LogicalGroups into the vLinkGen configuration (like MSR_VAR, MSR_CODE), but also these are not used by default.

For more information on this module, please consult TechnicalReference_MemMap.pdf.

4 vBRS

The configuration of the Vector Basic Runtime System is separated into four parts:

General, HwConfig, MemoryInit and TestsuiteSupport

The General, HwConfig and MemoryInit container are mandatory and always present, TestsuiteSupport is optional.

The generator is generating several files:

- Makefile.config.generated (into the “Appl” folder), for all parameters, necessary for the BuildEnvironment (including additional files to build, via APP_SOURCE_LST)
- vBrs_Lcfg.c, vBrs_Lcfg.h and vBrsCfg.h (into the “Appl\GenData” folder), for all parameters, used inside the SourceCode implementation
- If the dependent stubs are activated, files of the EcuM and SchM stubs are generated into Appl\Include and Appl\Source (e.g., SchM_Can.h, EcuM.c)

The Vector Basic Runtime System also consists of several static files, which are platform and program independent:

- BrsMain.c/.h - Main files of the Vector Basic Runtime System
- BrsMainStartup.c - Startup routines, mainly for memory initialization
- BrsMain_Callout_Stubs.c - Templates for hooks out of other BrsMain files
- BrsMain_Appl.c/.h - Additional, application-specific, implementation
- BrsMain_Types.h - Declaration of BrsMain specific types
- Brs_MemMap.h - BRS specific header for memory mapping mechanism
- BrsMain_Cfg.h – BrsMain specific user configuration header.

See chapter 6 for details.

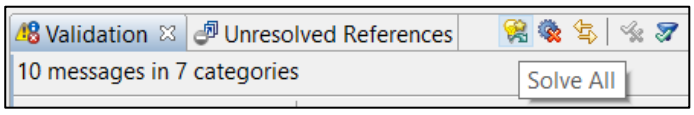
4.1 vBRS Setup Support

To help the user with the initial setup of a project, a lot of so called “solving actions” are implemented within the vBRS generator in Cfg5. This generator is analyzing your actual configuration and is checking the actual setup of several modules accordingly.

See this sample for the initial validation messages of a default configuration:

vBRS01041	Tasks for vBRSEnableSupportLEDs / vBRSEnableSupportToggleCustomPin / vBRSEnableSupportToggleWdPin / vBRSEnableTestsuiteSupport missing (1 message)
vBRS01041	BrsMainTask and further necessary tasks for MsrTestsuite are missing.
	Create Task 'BrsMainTask'
	Create Task 'BrsMainBackgroundTask'
	Create all missing Tasks: 'BrsMainTask', 'BrsMainBackgroundTask'
	/ActiveEcuC/Os
vBRS01042	Default init tasks are missing (1 message)
vBRS01042	Default_Init_Task and further necessary tasks for MsrTestsuite are missing.
	Create Task 'Default_Init_Task'
	Create Task 'Default_Init_Task_Trusted'
	Create all missing Tasks: 'Default_Init_Task', 'Default_Init_Task_Trusted'
	/ActiveEcuC/Os
vBRS01080	Os counter and/or clock configuration settings are inconsistent (1 message)
vBRS01080	Os counter available - configuration inconsistent (required relation: OsCounterTicksPerBase = BRS Peripheral Clock * OsSecondsPerTick).
	Set OsCounterTicksPerBase(value=1) to 12500
	/ActiveEcuC/Os/SystemTimer[OsCounterTicksPerBase]
vBRS01200	The configuration of the booleans vBRSHwEnable{PllClocks,Port,Watchdog}Handling and the usage of modules Mcu, Port and Wdg is not consistent (1 message)
vBRS01200	Execute the following changes to adapt the vBRS hardware driver configuration to the modules contained in the current project.
	Set vBRSEnablePLLClocksHandling(value=false) to true
	Set vBRSEnablePortHandling(value=false) to true
	Set vBRSEnableWatchdogHandling(value=false) to true
	Adjust all vBRSHwEnable{PllClocks,Port,Watchdog}Handling boolean values.
	/ActiveEcuC/vBRS/vBRSHwConfig[vBRSEnableWatchdogHandling]
	/ActiveEcuC/vBRS/vBRSHwConfig[vBRSEnablePLLClocksHandling]
	/ActiveEcuC/vBRS/vBRSHwConfig[vBRSEnablePortHandling]
vBRS01700	Configuration of EcuM/SchM stub generation and usage of Microsar EcuM/SchM modules is not consistent (2 messages)
vBRS01700	Enable the generation of EcuM stub by vBRS because no Microsar EcuM is part of the configuration.
	Set vBRSEnableEcuMStubGeneration(value=false) to true
	/ActiveEcuC/vBRS/vBRSGeneral[vBRSEnableEcuMStubGeneration]
vBRS01700	Enable the generation of SchM stubs by vBRS because no Microsar Rte is part of the configuration.
	Set vBRSEnableSchmStubGeneration(value=false) to true
	/ActiveEcuC/vBRS/vBRSGeneral[vBRSEnableSchmStubGeneration]
vBRS02400	Some core specific vLinkGen Logical Var Groups are missing. (1 message)
vBRS02400	Multiple vLinkGen Logical Var Groups are missing.
	Create all missing vLinkGen Logical Var Groups: 'STACK_C0'
	Create vLinkGen Logical Var Group 'STACK_C0'
	/ActiveEcuC/vLinkGen/vLinkGenMemLayout
	/ActiveEcuC/vLinkGen

After the TestedDerivative in the vBaseEnv module was selected, all these messages should be clear and valid. Via the “Solve All” button, all clear solutions are selected automatically:



Only parameters, where several solutions are provided, must be handled manually.

4.1.1 OS tasks creation

vBRS is creating all tasks, needed for an initial setup. This contains the Default_Init_Task and Default_Init_Task_Trusted for every Autosar core, that is configured in the OS (Os/OsCore). The static implementation of all tasks can be found in BrsMain.c. The init tasks of the master core (Os/OsOS/OsMasterCore) will not include a core postfix. All other init tasks will include a core postfix, depending to the physical core Id of the specific OsCore (Os/OsPublishedInformation/OsDerivativeInformation/OsPhysicalCore/OsPhysicalCoreId).

The init tasks are mapped automatically to the suitable default OsApplication of the core. Be aware, that this mapping needs to be checked if OS mechanisms are used, which need a special OsApplication configuration (like memory protection).

If one of the toggle pin functionalities and/or the testsuite support are enabled, also the tasks BrsMainTask and BrsMainBackgroundTask, as well as the belonging OS alarm will be created. They are needed for the BrsMain millisecond handler functionality.

4.1.2 OS SystemTimer calculation

Depending on the value of the peripheral timer clock frequency setting (vBRS/vBRSHwConfig/vBRSPeriphClock), the counter ticks per base value of the OS (Os/OsCounter/OsCounterTicksPerBase) will be calculated and configured.

4.1.3 Core specific vLinkGen settings

For every core, that is configured in the OS (Os/OsCore), a logical var group and the underlying var section group for the startup stack will be created in vLinkGen. They will include a core postfix, depending to the physical core Id of the specific OsCore (Os/OsPublishedInformation/OsDerivativeInformation/OsPhysicalCore/OsPhysicalCoreId). This is done for Autosar and non-Autosar cores (Os/OsCore/OsCoreIsAutosar).

These STACK_Cx LogicalVarGroups are mapped automatically into a suitable MemoryRegionBlock, via the ContentType “STARTUP_STACK”.

See chapter 3.4.4 for details of the settings.

4.1.4 Additional consistency checks

Several additional checks show warnings and provide a solving action to optimize the configuration.

The availability of MCAL drivers in the configuration (DrvMcu, Port, DrvWd) is checked and the belonging parameters in vBRS/vBRSHwConfig for the same functionality are optimized.

The availability of the Autosar modules EcuM and SchM, as well as the belonging parameters in vBRS/vBRSGeneral to enable stubs for these modules, are checked.

Several frequency parameters in vBRS/vBRSHwConfig are checked with the tested derivative and evaluation board specific values.

4.2 vBRSGeneral

Within this section, additional features of the environment can be configured.

Parameter	Description
Disable Setup Support	If you don’t want the vBRS generator to create necessary OS tasks automatically (via solving actions, e.g., BrsMainTask, Default_Init_Task), you can disable it here.
Enable OS MultiCore support	If you want the environment, to support more than 1 Core, this switch should be enabled. This will lead to generation of “BRS_ENABLE_OS_MULTICORESUPPORT”.
Enable OS SafeContext (SC3) support	This will lead to generation of “BRS_ENABLE_SAFECTXSUPPORT”. Via this switch, several parts to better support Memory Protection UseCases are enabled (e.g., call of BrsHw_EnableHwAccess()).
Enable Small Devices optimization	This functionality will help to setup the environment for controllers with less available memory. The OS default stack sizes of some basic setup parameters will be decreased, to reduce the RAM consumption to a minimum. Additionally, some hooks will be checked to be disabled. See chapter 4.2.1 for details.
Enable core specific vLinkGen config validation	This will lead to the creation of core specific LogicalVarGroups in the vLinkGen configuration. Solving actions will occur, which create startup stack groups and sections for every configured core (for the boot core and every additional configured OS core). This mechanism is still under development.
Generation of EcuM Stub	This enables the generation of the EcuM Stub. See chapter 4.6 for details.
Generation of SchM Stub	This enables the generation of the SchM Stub. See chapter 4.7 for details.
Generation of SchM Stub – empty exclusive area handling	For normal, all module specific exclusive are calls are mapped to SuspendAllInterrupts()/ResumeAllInterrupts(). If this feature is activated, the SchM stub will generate them as empty calls instead. E.g. in SchM_Mcu.h: # define SchM_Enter_Mcu_MCU_EXCLUSIVE_AREA_00() # define SchM_Exit_Mcu_MCU_EXCLUSIVE_AREA_00() instead of # define SchM_Enter_Mcu_MCU_EXCLUSIVE_AREA_00() SuspendAllInterrupts() # define SchM_Exit_Mcu_MCU_EXCLUSIVE_AREA_00() ResumeAllInterrupts() This is mainly for polling environments, where no OS is used and therefore no specific disabling of interrupts is necessary.

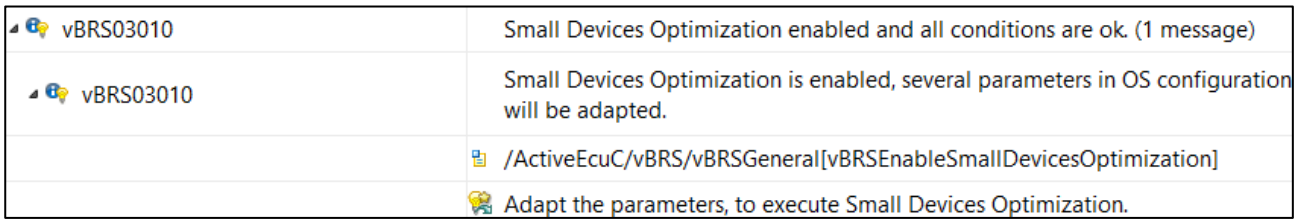
Parameter	Description
Generation of SchM Stub – support DrvCan AN	Prevents the SchM Stub from generation of the following exclusive areas: Classic DrvCan: CAN Driver (CAN_EXCLUSIVE_AREA_6) CAN Interface (CANIF_EXCLUSIVE_AREA_0) CANSM (CANSM_ EXCLUSIVE_AREA_1, CANSM_ EXCLUSIVE_AREA_4) COMM (COMM_ EXCLUSIVE_AREA_1) Can_30_Core/vCan drivers: Can_30_Core Driver (CAN_30_CORE_EXCLUSIVE_AREA_6) vCAN Driver (VCAN_30_x_EXCLUSIVE_AREA_0, VCAN_30_x_EXCLUSIVE_AREA_1) See chapter 4.7 for details.
Execution Instance	Configure the execution instance of the application, handled by this configuration. "Application" stands for a "normal" application executable (e.g., MSR or CANbedded stack), which could be standalone or called out of an FBL or BM. "FBL", "FBL_Updater", "BM" and "BM_FBL" stands for a FlashBootLoader specific executable. See chapter 4.3 for details. "HSM" stands for a Hardware Security Module executable. See chapter 4.4 for details. "HSM_Updater" stands for a Hardware Security Module Updater executable. See chapter 4.4.1 for details. "FlashDriver" stands for a standalone binary, containing a flasher driver instance only. There is no runtime part included. See chapter 4.3.3 for details. "Hypervisor" stands for a virtual machine host executable, which enables hardware virtualization. See chapter 4.5 for details.
FBL/BM exception table in RAM	Configure, if an additional ExceptionTable in RAM should be initialized. See chapter 4.3.1 for details.
FBL/BM support with EcuMInit	Configure, if the FBL stack should be started inside main() via a call of EcuM_Init() or a call of FblMain(). See chapter 4.3.1 for details.
First execution instance in system	Configure, if this executable is the first instance of the system. Set this parameter to “enabled” if this instance should contain the brsStartupEntry and initialize the core. Only one single executable of a system with several executables should be the first instance. All other executables are starting with Brs_ApplicationEntry(). See chapter 4.3.1 for details.

Parameter	Description
Guest execution instance in hypervisor host system	Configure, if this executable is a guest execution instance for use in a hardware virtualization hypervisor host system. This will allow the 1st execution instance to keep the virtualization functionality enabled during boot (e.g. Cortex-R52). Application instances will handle the SW-Reset different. See chapter 4.5.1 for details.
Use vLinkGen Variant	vLinkGenVariant reference for current executable. The chosen variant will be generated into vBrsCfg.h and then be used in vLinkGen_Lcfg.c. Furthermore, it will also be generated as BRS_LINKER_COMMAND_FILE_POSTFIX into Makefile.config.generated. This will override the LINKER_COMMAND_FILE parameter in the Makefile. LINKER_COMMAND_FILE is used for the linkage target of the MakeSupport. Note: /MICROSAR/vLinkGen/vLinkGenGeneral/vLinkGenFileGeneration shall be set to ONE_FILE_PER_VARIANT, to generate the corresponding link file for each variant. This mechanism is mainly used by the FBL demo applications.
Toggle Custom-Pin	This will lead to generation of “BRS_ENABLE_SUPPORT_TOGGLE_CUSTOM_PIN”. It will cause the environment to cyclic toggle the custom pin, configured inside BrsHw_Ports.h.
Toggle Custom-Pin – Dio channel reference	Dio channel reference setting for toggling the Custom port. Only relevant if non-stub Dio is part of the configuration.
Toggle LED-Pin	This will lead to generation of “BRS_ENABLE_SUPPORT_LEDS”. It will cause the environment to cyclic toggle the LED pin, configured inside BrsHw_Ports.h.
Toggle LED-Pin – Dio channel reference	Dio channel reference setting for toggling the LED port. Only relevant if non-stub Dio is part of the configuration.
Toggle WD-Pin	This will lead to generation of “BRS_ENABLE_SUPPORT_TOGGLE_WD_PIN”. It will cause the environment to cyclic toggle the WD pin, configured inside BrsHw_Ports.h.
Toggle WD-Pin – Dio channel reference	Dio channel reference setting for toggling the WD port. Only relevant if non-stub Dio is part of the configuration.
User config header file	If a file name is entered here, it will be included within vBrsCfg.h

4.2.1 Small Devices optimization

To allow the initial usage of vBaseEnv also on devices with small memory sizes (especially RAM, for example Cortex-M based controllers with 8kB of RAM), so called "Small Devices", this feature will decrease several OS stack sizes. The OS is using a default value for stack sizes (1024 Byte), which supports a wide range of controllers and grants the user to not hit stack overflows at start of the development immediately. For dedicated environments, we evaluated minimum sizes, which will be suitable for an initial setup.

The feature can be activated by enabling the configuration parameter vBRS/vBRSGeneral/vBRSEnableSmallDevicesOptimization and performing the depending solving action:



Note
The feature is only available for singleCore and SC1 configurations.

The following OS core stack sizes (Os/OsCore0/OsCoreStacks) will be adapted by this feature:

- Error Stack Size = 128
- Init Stack Size = 128 (Iar compiler: 192)
- Kernel Stack Size = 128

The usage of the other core stacks (Protection, Shutdown, Startup) is completely disabled by unchecking the depending on hook usage (/Os/OsOS/OsHooks).

Also, the stack sizes of the following tasks will be adapted:

- BrsMainBackgroundTask: stack size = 256 (Iar compiler: 328)
- BrsMainTask: stack size = 256 (Iar compiler: 320) (Testsuite Support enabled: 512)
- Default_Init_Task: stack size = 256
- Default_Init_Task_Trusted: stack size = 256
- IdleTask_OsCore0: stack size = 4

Additionally, the stack size of the OS timer ISR "CounterIsr_SystemTimer" is set to 256.

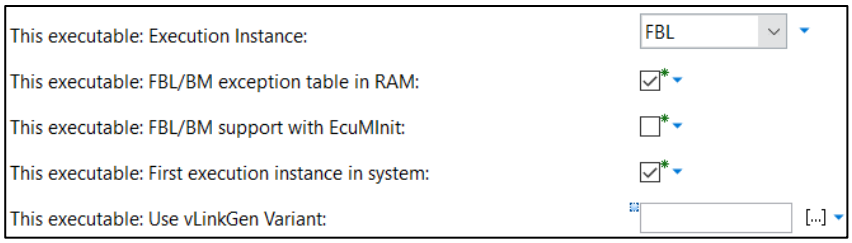
All these actions will decrease the initial RAM usage significantly, so the initial usage of vBaseEnv will also be possible on such “Small Devices”.



Caution
Be aware, that this will increase the risk of runtime errors, in consequence of stack overflows. But the OS stack monitoring feature (Os/OsOS/OsStackMonitoring) should be able to detect them.

4.3 FBL support

If your project is a FlashBootLoader project, you can configure your UseCase with the parameters “Execution Instance”, “FBL/BM Exception Table in RAM”, “FBL/BM Support with EcuMInit” and “First execution instance in system”.



The BrsMain and BrsHw parts will be configured accordingly. The following values for “Execution Instance” will lead to generation of “BRS_ENABLE_FBL”:

"**FBL**", "**FBL_Updater**", "**BM**" and "**BM_FBL**"

Note

For the UseCase FBL, the OS is not part of the environment.

As FBL is always running in polling mode and provides its own timer implementation and scheduling, the OS is not needed at all.

Core Exceptions are handled inside a separate core exception table in the file BrsHwIntTb.c.

Also, a 2nd core exception table is provided in BrsHwIntTb.c, to be copied in RAM before flash erase/re-programming.

A stub for Os.h is provided, to allow the usage of the Autosar SchM.

4.3.1 FBL UseCases

The following FBL UseCases exist:

- "**FBL**" stands for a FlashBootLoader executable, which could be standalone or called out of a BM. For standalone, also configure “First execution instance in system”. For specific implementation parts, the define “BRS_FBL_INSTANCE_FBL” is generated.
- "**FBL_Updater**" stands for a FlashBootLoader Updater executable, which could be standalone or called out of a BM. For standalone, also configure “First execution instance in system”. For specific implementation parts, the define “BRS_FBL_INSTANCE_UPDATER” is generated.
- "**BM**" stands for a BootManager executable, which is the startup entry and decides to call a FBL, Updater or Application. “First execution instance in system” should always be enabled for this executable. For specific implementation parts, the define “BRS_FBL_INSTANCE_BM” is generated.
- "**BM_FBL**" stands for a BootManager and FlashBootLoader executable, which are compiled together. The BootManager part is the startup entry and decides to call the FBL part, Updater or Application. “First execution instance in system” should always be enabled for this executable. For specific implementation parts, the define “BRS_FBL_INSTANCE_BM_FBL” is generated.

For the UseCase “FBL”, the file BrsMain_Appl.c contains a 4-Byte PresencePattern, which will be mapped into the specific section “.FblPresencePatternSection”. This presence pattern is only required when downloading the FBL with debugger. When using an updater (or an EOL programmer), this presence pattern is not required as part of the .hex file and should be removed.

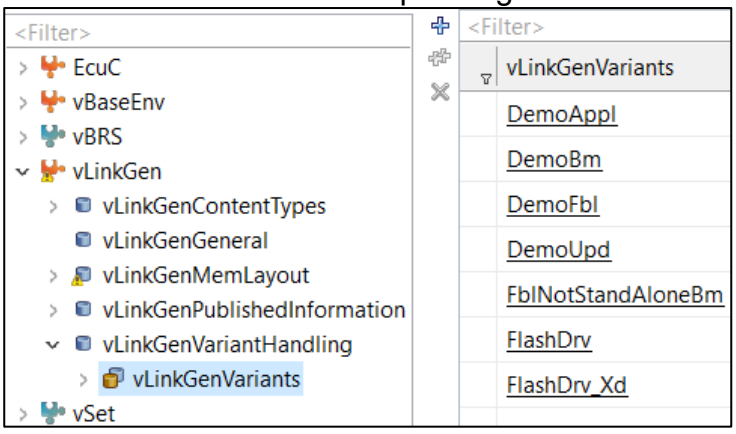
If the actual FBL execution instance runtime part is going to run completely out of RAM (e.g., if it is mentioned to erase/re-program the flash), it could be necessary to initialize a core exception table, which will also be in RAM. For this, there exists a 2nd core exception table in BrsHwIntTb.c (section .brsExcVectRam). Via the parameter “FBL/BM Exception Table in RAM”, this additional core exception table will be initialized in main() of this execution instance. It will replace the default core exception table (usually located in flash), which was initialized during startup. The depending on SectionGroup “Brs_ExcVectRam” needs to be configured with Init Policy “COPY_TO_RAM” in vLinkGen as well.

Depending to the SLP and FBL Release, the project is based on, there exist several variants for any FBL stack. Via the parameter “FBL/BM Support with EcuMInit”, you need to configure, if the FBL stack should be started inside main() via a call of EcuM_Init() or a call of FblMain().

- FBL without EcuM_Init()
- Neither the OS is going to be started, nor the EcuM stub is going to be used. BrsMainInit() skips the HW initialization and main() calls FblMain() directly. The SchM stub is activated per default.
- This is handled in the implementation by the generated define “BRS_FBL_NO_ECUMINIT”.
- FBL with EcuM_Init()
- The OS is not started. BrsMainInit() skips the HW initialization and main() calls EcuM_Init(). The Autosar modules initialization is handled by the EcuM stub. Also, the generated SchM stub is used (if needed). FblMain() is called from EcuM stub, after Init-zero and Init-one phase. The SchM stub is activated per default.
- This is handled in the implementation by the generated define “BRS_FBL_WITH_ECUMINIT”.

4.3.2 vLinkGen Variants

If you are using vLinkGen variants, for example to distinguish the memory layout of multiple execution instances, you can choose the variant of this configuration via the parameter vBRS/vBRSGeneral/vBRSVariantUsageOfvLinkGen. Choose the valid variant, which is derived from the depending on list in vLinkGen:



This will lead the vBrs generator to generate the suitable defines into vBrsCfg.h and Makefile.config.generated. If vLinkGen/vLinkGenGeneral/vLinkGenFileGeneration is set to ONE_FILE_PER_VARIANT, a suitable LinkerFile and suitable code for memory initialization will be generated by vLinkGen and the MakeSupport will use them automatically.

4.3.3 FlashDriver

For some projects, it is needed to create an own binary with the flash driver implementation.
For this UseCase, the execution instance “FlashDriver” should be used. The vBRS generator will check to disable the EcuM Stub and the SchM Stub, as well as “First execution instance in system” and “exception table in RAM”. No FBL specific defines will be generated into vBrsCfg.h and vBrs_Lcfg.c will not be added to APP_SOURCE_LST in Makefile.config.generated.
In such instances, no files from BrsMain and BrsHw should be part of the build.

4.4 HSM support

If your project is a Hardware Security Module project, you can enable the support with the value “HSM” of the parameter “Execution Instance”.



The BrsMain and BrsHw parts are configured accordingly. This will lead to the generation of “BRS_ENABLE_HSM_SUPPORT” in vBrsCfg.h and in Makefile.config.generated.

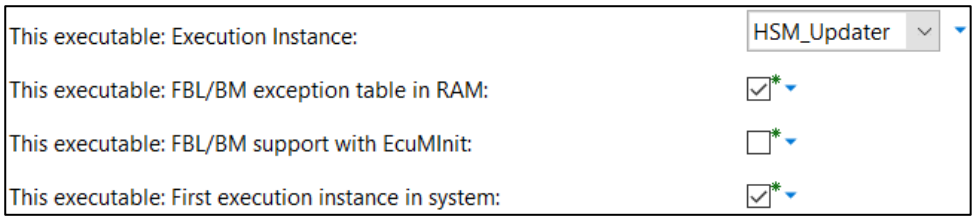
As with HSM support, the EcuM is not used at all, the initialization changes as follows:

- In Default_Init_Task, vHsm_StartupTwo() is called instead of EcuM_StartupTwo().
- In main(), vHsm_Init() is called instead of EcuM_Init().

Port handling is not needed with UseCase HSM, so the Port handling in vBRSHwConfig must always be disabled.

4.4.1 HSM Updater

"HSM_Updater" stands for a Hardware Security Module Updater project, which is the startup entry and decides to call the HSM or update the HSM.

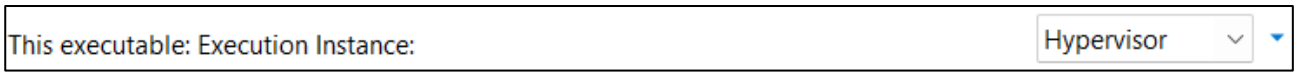


This will lead to the generation of “BRS_ENABLE_HSM_UPDATER_SUPPORT” in vBrsCfg.h and “BRS_ENABLE_HSM_SUPPORT” in Makefile.config.generated.

Unlike the HSM instance, the HSM Updater instance is always without an OS and is always the first execution instance. Like at the FBL instance, also the HSM Updater instance will provide a 2nd core exception table, to be copied into RAM.

4.5 Hypervisor support

If your project contains a Hypervisor virtual machine host instance, you can enable the support for the host execution instance with the value “Hypervisor” of the parameter “Execution Instance”.



The BrsMain and BrsHw parts are configured accordingly. This will lead to the generation of “BRS_ENABLE_HYPERVISOR_SUPPORT” in vBrsCfg.h and Makefile.config.generated.

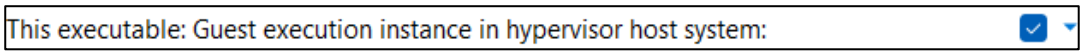
As within a Hypervisor instance, the OS and the EcuM are not used at all, the initialization changes as follows:

- In main(), vHyp_InitMemory() and vHyp_Init() are called instead of Os_InitMemory() and Os_Init().
- Previously, BrsMain_Appl_HypervisorStartCoreApi() is called. Within MultiCore hypervisor configurations, the additional cores are started by the init core, via calls of BrsHw_StartCore().
- Afterwards, BrsMain_Appl_HypervisorApi() is called. Within MultiCore hypervisor configurations, the additional cores are registered by the init core, via calls of vHyp_RegisterCore().
- At the end, vHyp_Start() is called instead of EcuM_Init(). The hypervisor is now taking over and will start the scheduling of the VMs.

EcuM Stub and SchM Stub are not needed with UseCase Hypervisor, so these parameters must always be disabled.

4.5.1 Hypervisor Guest

If your project contains a Hypervisor virtual machine host instance, all other execution instances need to enable the parameter vBRSHypervisorGuest



This will lead to the generation of “BRS_HYPERVISOR_GUEST_EXECUTION_INSTANCE” in vBrsCfg.h and Makefile.config.generated.

If the Hypervisor virtual machine host instance is not the 1st execution instance, this will allow the 1st execution instance to keep the virtualization functionality enabled during boot (e.g. Cortex-R52). Application instances which run within a Hypervisor virtual machine will handle the SW-Reset different.

4.6 EcuM Stub

If no real ECU State Manager is part of your SIP, a solving action will advise you, to enable the generation of the EcuM Stub. This will lead the vBRS to generate the files EcuM.c, EcuM.h, EcuM_Cbk.h, EcuM_Error.h and EcuM_Generated_Types.h. The file EcuM.c will be added into APP_SOURCE_LST within Makefile.config.generated automatically.

This Stub will use the listed parameters within /MICROSAR/vSet/vSetBswInitialization/vSetInitFunction or, if this is not available in your environment, /MICROSAR/EcuC/EcucGeneral/BswInitialization, to call the module initialization functions accordingly. E.g., 3rd-party MCAL modules register their initialization functions there.

The following APIs will be provided:

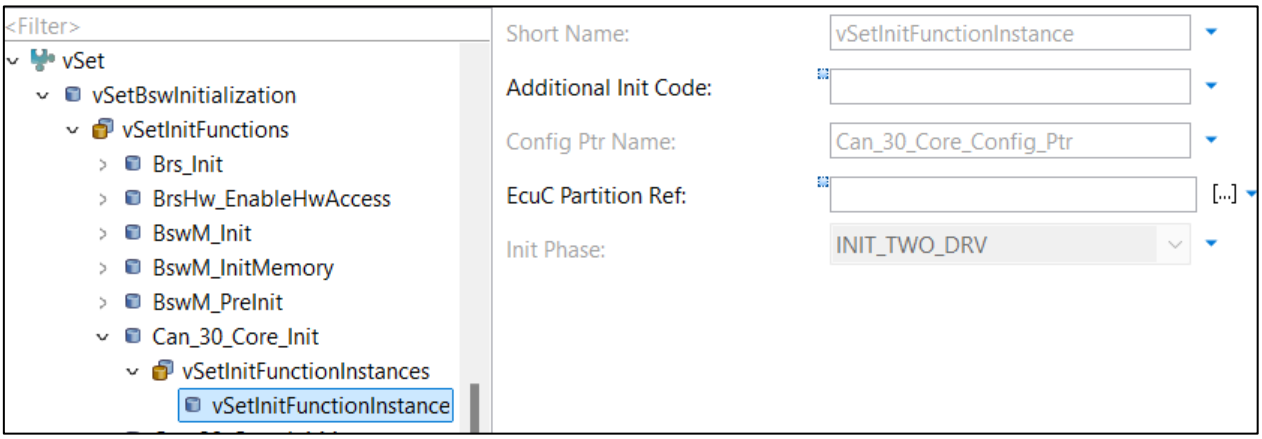
API	Description
EcuM_Init()	This is the default entry point of any Autosar stack during startup. This API will always be called by the main() function inside BrsMain.c. Within this function, all module initialization functions of “Init Phase” Zero and One will be called. At the end, either EcuM_StartOS() (FBL UseCase = None) or FblMain() (FBL UseCase = FBL_with_EcuMInit) is called.
EcuM_StartupTwo()	This is the default entry point of any Autosar stack, after the OS is running. This API will always be called by the Default_Init_Task() inside BrsMain.c. Within this function, all module initialization functions of “Init Phase” Two and Three will be called. At the end, Rte_Start() is called.
EcuM_Shutdown()	This callout will always be empty by default.
EcuM_StartOS()	This callout will always call StartOS(). This API is not available for FBL UseCase = FBL_with_EcuMInit.
EcuM_BswErrorHook()	This callout will always be empty by default.
EcuM_CheckWakeup()	This callout will always be empty by default.
EcuM_GeneratorCompatibilityError()	This callout will always be empty by default.
EcuM_SetWakeupEvent()	This callout will always be empty by default.
EcuM_ValidationWakeupEvent()	This callout will always be empty by default.

The EcuM Stub is also supporting Postbuild selectable. As soon as several variants are configured in Cfg5, new functionality is generated into EcuM.c:

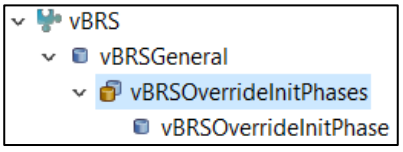
API	Description
EcuM_DeterminePbConfiguration()	Called at the end of EcuM_Init(), to determine the actual variant to be initialized. Per default, the variant EcuM_FirstVariant is selected. This can be changed by any kind of implementation. See EcuM_GlobalConfigRootType for all available variants.

4.6.1 EcuM Stub override feature

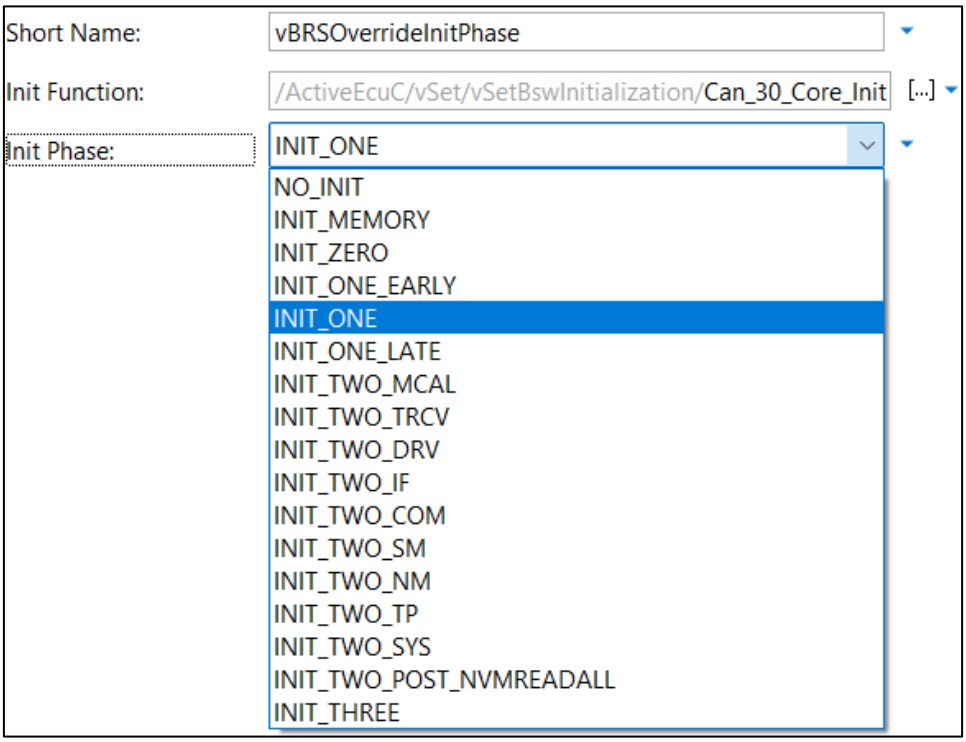
Sometimes, it is necessary to change the initialization sequence of the modules, which automatically configured their init functions within vSet/vSetBswInitialization by a mostly suitable default value. As these values are mostly preconfigured, it is not always possible to change them inside the vSetBswInitialization container (greyed out).



The override is possible via the vBRSOverrideInitPhases container:



It is possible to choose single init list entries from vSet/vSetBswInitialization and configure an alternative init phase for them:



Instead of the vSetBswInitialization/vSetInitFunction configured init phase, the overridden ones will be used for vBRS generation of the EcuM Stub.

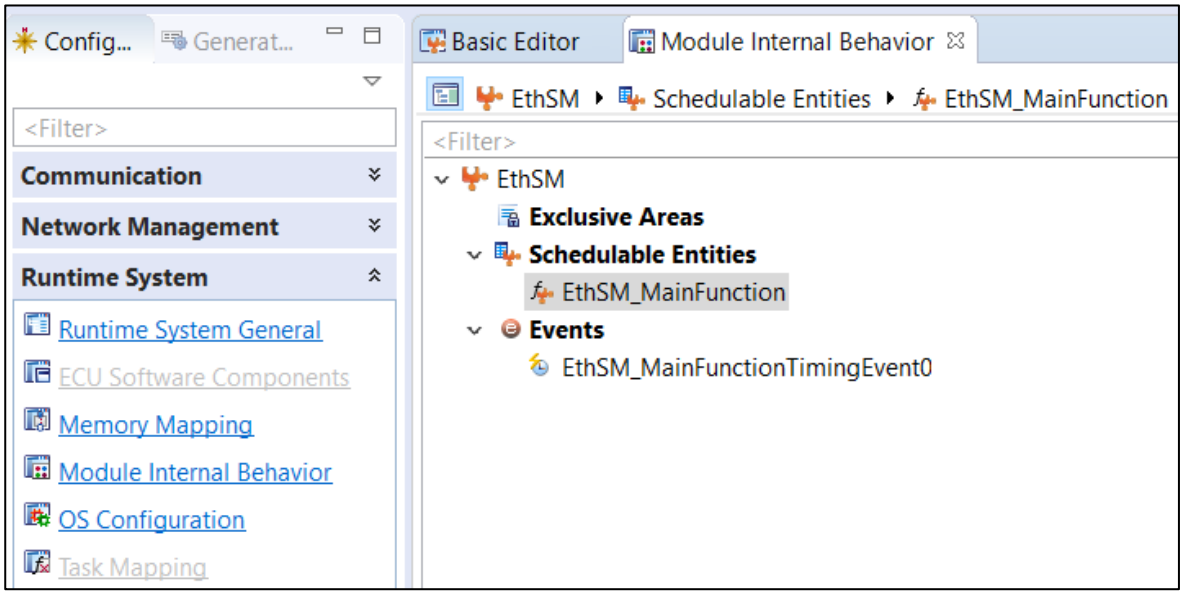
© 2025 Vector Informatik GmbH

4.7 SchM Stub

If no real Runtime Environment (RTE) is part of your SIP, a solving action will advise you, to enable the generation of the SchM Stub. This will lead the vBRS to generate several modules specific SchM_<MSN> header files with several definitions for enter- and exit exclusive areas. They all will be mapped to the default enable and disable interrupt functions of the OS: SuspendAllInterrupts(), ResumeAllInterrupts(). If the functionality “Generation of SchM Stub – empty exclusive area handling” is activated, they are generated as empty calls (main UseCase for polling environments, where interrupts are already disabled).

Declarations for schedulable entities (mainly MainFunctions) are also generated, if available.

The information, which is taken as basis for generation of this SchM stub, is visible in Cfg5 within the “Module Internal Behavior” window. The parameters are configuration specific configured by the dependent modules themselves and therefore they are read-only.



For better support of MICROSAR Classic CAN drivers, it is possible to prevent the SchM stub from generation of the following exclusive areas, via the dependent parameter within vBRSGeneral (“Generation of SchM Stub – support DrvCan AN”). These specific APIs are static implemented inside the file BrsMain_Appl.c, according to AN/TechnicalReference:

Classic DrvCan:
CAN Driver (CAN_EXCLUSIVE_AREA_6)
CAN Interface (CANIF_EXCLUSIVE_AREA_0)
CANSM (CANSM_ EXCLUSIVE_AREA_1, CANSM_ EXCLUSIVE_AREA_4)
COMM (COMM_ EXCLUSIVE_AREA_1)

This is for the behavior, described within CanDriver ApplicationNote AN-ISC-8-1149.

Can_30_Core/vCan drivers:

Can_30_Core Driver (CAN_30_CORE_EXCLUSIVE_AREA_6)
vCAN Driver (VCAN_30_x_EXCLUSIVE_AREA_0, VCAN_30_x_EXCLUSIVE_AREA_1)

This is for the behavior, described within TechnicalReference_Can_30_Core.pdf and TechnicalReference_vCan_30_x.pdf, note within chapter 3.2.

4.8 vBRSHwConfig

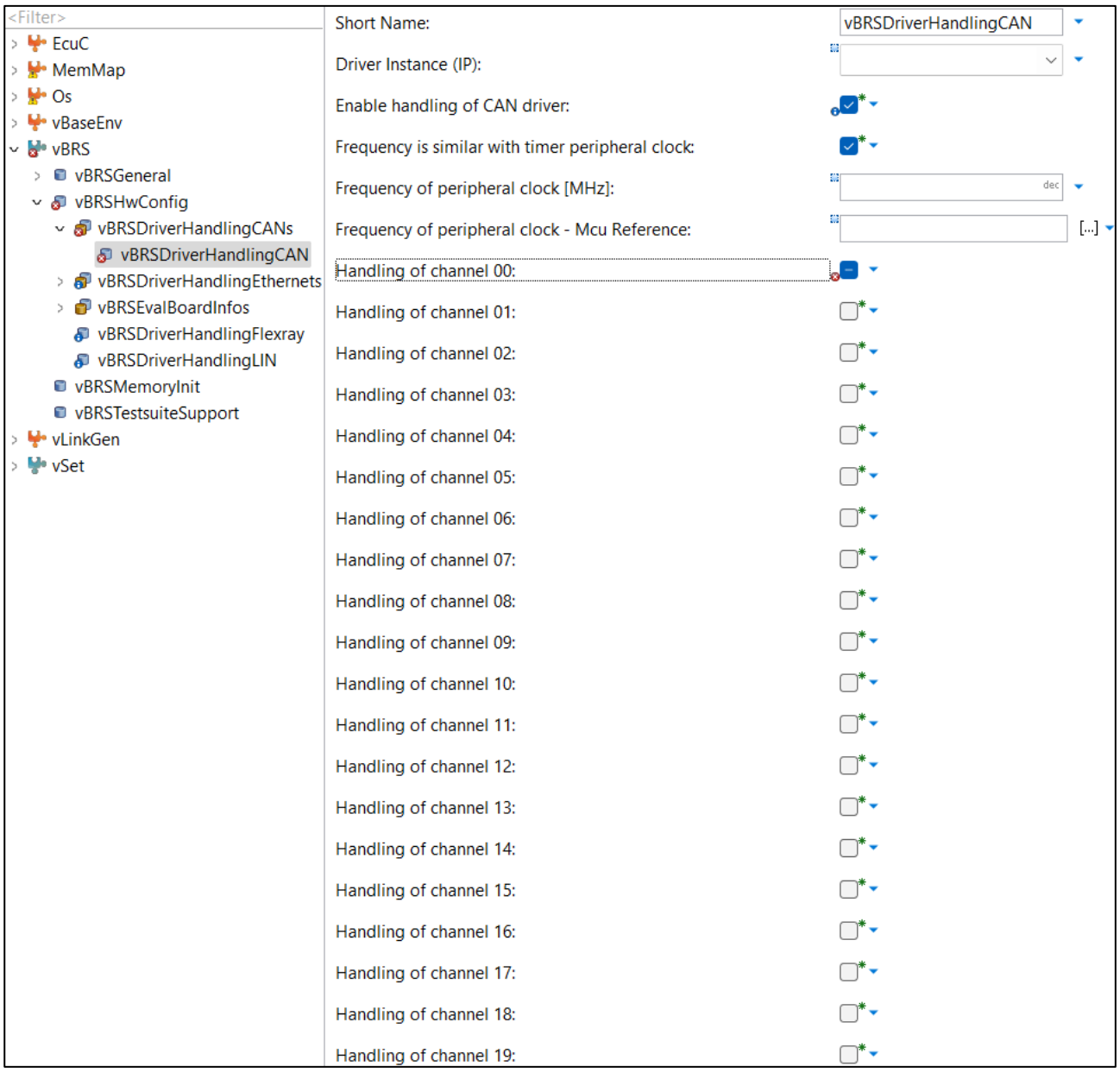
Within this section, HW dependent features of the environment are configured.

Parameter	Description
BrsHw: Enable PLL & Clocks Handling	This will lead to the generation of “BRS_ENABLE_PLLCLOCKS”. It will cause the BrsMain to call BrsHwPllInitPowerOn() within BrsMainInit(). This function will configure the PLL and clocks of the derivative, according to the settings of oscillator and peripheral clocks. If no DrvMcu is part of the SIP, a Solving Action wants to enable this.
BrsHw: Enable Port Handling	This will lead to the generation of “BRS_ENABLE_PORT”. It will cause the BrsMain to call BrsHwPortInitPowerOn() within BrsMainInit(). This function will configure the Port pins of the derivative, according to the configured communication channels and additional features. If no Port driver is part of the SIP, a Solving Action wants to enable this.
BrsHw: Enable Watchdog Handling	This will lead to the generation of “BRS_ENABLE_WATCHDOG”. It will cause the BrsMain to call BrsHwWatchdogInitPowerOn() within BrsMainInit(). This function will configure the Watchdog of the derivative into off mode, if necessary. If no DrvWd is part of the SIP, a Solving Action wants to enable this.
BrsHw: Prefer PLL and Watchdog Init	This will lead to the generation of “BRS_ENABLE_PREFER_PLL_WATCHDOG_INIT”. It will cause the BrsPreMain to call BrsHwPreInitPowerOn(), BrsHwWatchdogInitPowerOn() and BrsHwPllInitPowerOn() within the callout BrsMain_MemoryInit_StageOne_Hook(). This feature is intended to be used, if without disabling of the watchdog and without a proper PLL configuration, the initialization of memory is not possible.
BrsHw: Select Evaluation Board	If evaluation board specific port pin settings are available, these could be selected here.
Clock Config: Oscillator Clock Frequency [MHz]	Configure here the frequency of the external oscillator of your environment.
Clock Config: Peripheral (Timer) Clock Description	This field contains a description for the peripheral timer clock of this microcontroller platform. It gives a hint in how this clock is to be evaluated.
Clock Config: Peripheral (Timer) Clock Frequency [MHz]	Configure here the frequency of the internal clock, related to the timer that is used by the operating system.
Clock Config: Peripheral (Timer) Clock Mcu Reference	If you are using a MCU module to configure the PLL and the clocks, select here the depending on Mcu clock reference point frequency, that is related to the timer that is used by the operating system.

Parameter	Description
Clock Config: Timebase (PLL) Clock Frequency [MHz]	Configure here the output frequency of the PLL, to be configured inside BrsHwPllInitPowerOn() function.
Makesupport: Compiler Instruction Set	If the current platform is providing several Instruction Sets, the one to use is selected via this switch.
Makesupport: Compiler Path	Base path of the embedded compiler, to be used from the MakeSupport. This parameter allows to override the COMPILER_BASE_WIN parameter in the Makefile. Use absolute paths, w/o blanks. See default path in Makefile for an example.
Makesupport: Enable FPU support	If the current platform is providing a Floating-Point Unit, the support can be enabled via this switch.
Makesupport: Project Name	Name of this project, to be used by the MakeSupport. This parameter allows to override the PROJECT_NAME parameter in the Makefile.static. PROJECT_NAME is used as target name by the MakeSupport.
Makesupport: UserCode	The UserCode will be generated into Makefile.config.generated.

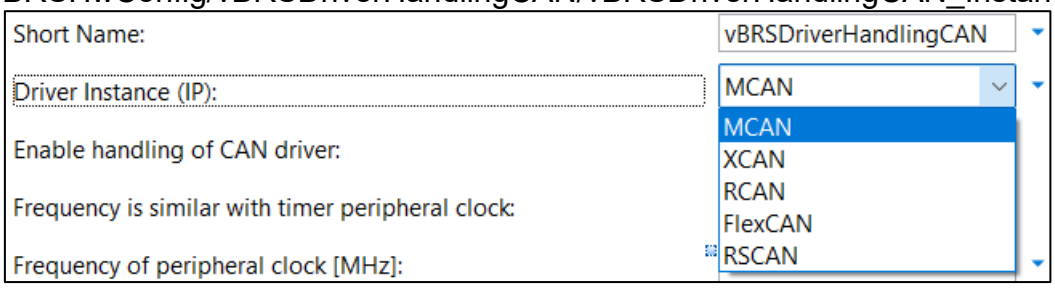
4.8.1 DriverHandlingCAN

This is not a mandatory container, but a solving action suggests creating it, as soon as a DrvCan is available in the configuration.

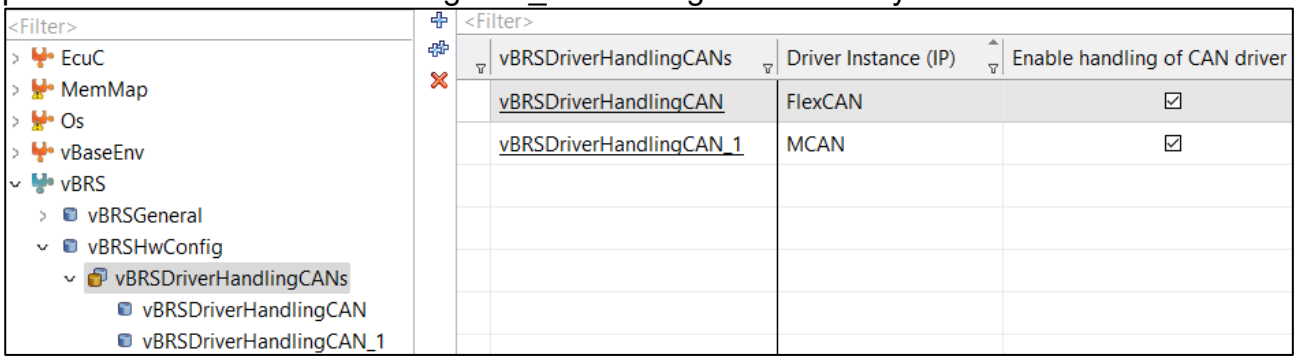


If the handling is enabled, the BrsHw will set the depending peripheral clocks and pin settings during HW initialization.
It is also possible to configure peripheral clocks, if they differ from the peripheral timer clock, configured in vBRSHwConfig/vBRSPeriphClock.

If more than one CAN IP is available on the controller, it could become necessary to select the one, to be used. That's the task of the not mandatory parameter "vBRS/vBRSHwConfig/vBRSDriverHandlingCAN/vBRSDriverHandlingCAN_Instance":



It is also possible to configure 2 DrvCan instances in parallel. In such a configuration, the parameter "vBRSDriverHandlingCAN_Instance" gets mandatory:

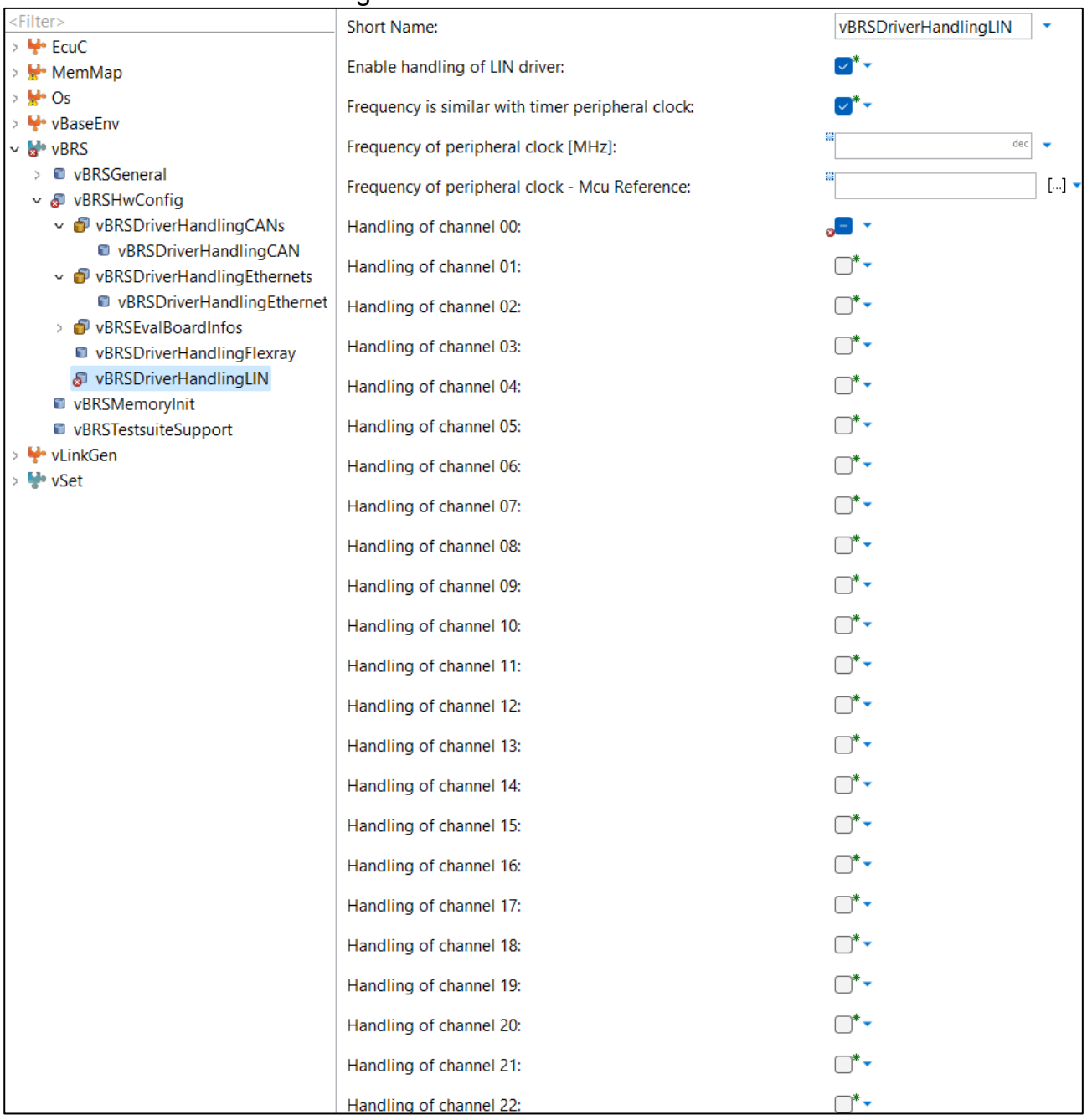


Depending on the selected value of "vBRSDriverHandlingCAN_Instance", the vBRS will generate the DrvCan specific defines in vBrsCfg.h with a corresponding infix:
BRS_CAN_infix_CLOCK
BRS_ENABLE_CAN_infix_CHANNEL_x
This allows the BrsHw implementation to differentiate. Additional to BRS_ENABLE_CAN_SUPPORT, also BRS_ENABLE_CAN_infix_SUPPORT will be generated.

© 2025 Vector Informatik GmbH

4.8.2 DriverHandlingLIN

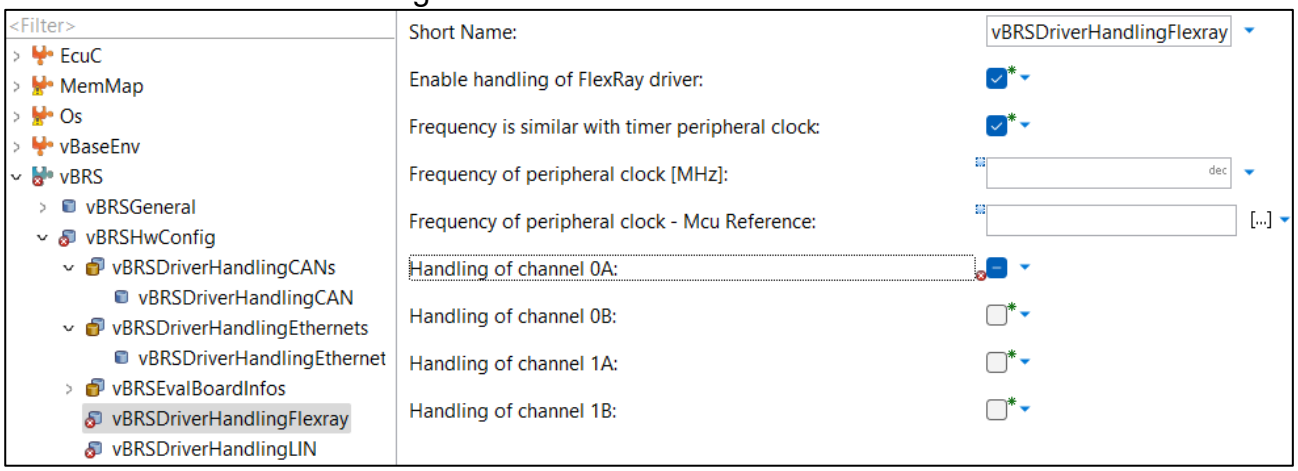
This is not a mandatory container, but a solving action suggests creating it, as soon as a DrvLin is available in the configuration.



If the handling is enabled, the BrsHw will set the depending peripheral clocks and pin settings during HW initialization.
It is also possible to configure peripheral clocks, if they differ from the peripheral timer clock, configured in vBRSHwConfig/vBRSPeriphClock.

4.8.3 DriverHandlingFlexray

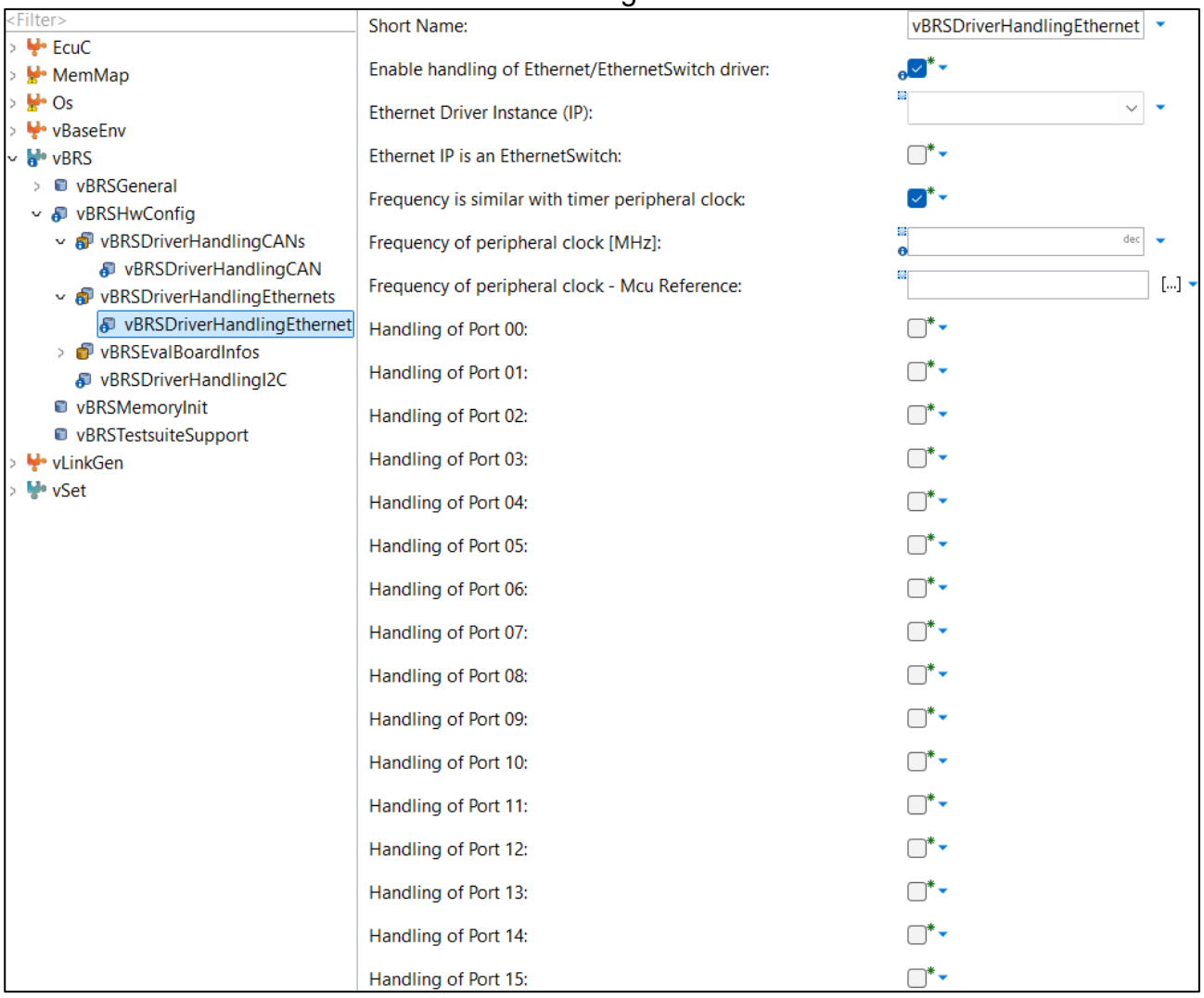
This is not a mandatory container, but a solving action suggests creating it, as soon as a DrvFr is available in the configuration.



If the handling is enabled, the BrsHw will set the depending peripheral clocks and pin settings during HW initialization.
It is also possible to configure peripheral clocks, if they differ from the peripheral timer clock, configured in vBRSHwConfig/vBRSPeriphClock.

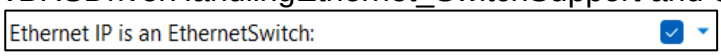
4.8.4 DriverHandlingEthernet

This is not a mandatory container, but a solving action suggests creating it, as soon as a DrvEth or DrvEthSwitch is available in the configuration.



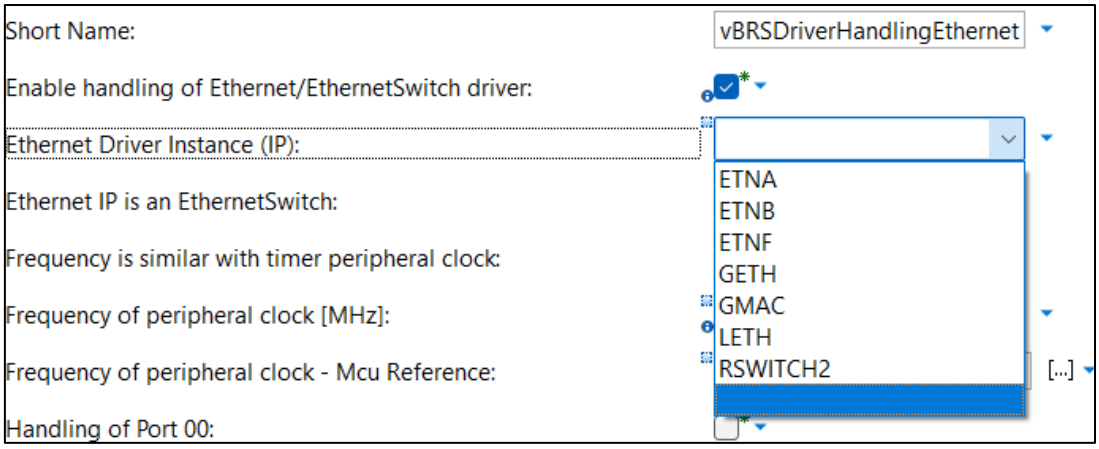
If the handling is enabled, the BrsHw will set the depending peripheral clocks and pin settings during HW initialization.
It is also possible to configure peripheral clocks, if they differ from the peripheral timer clock, configured in vBRSHwConfig/vBRSPeriphClock.

If the configured Ethernet IP is an Ethernet Switch, enable vBRSDriverHandlingEthernet_SwitchSupport and configure the Switch ports to be used.

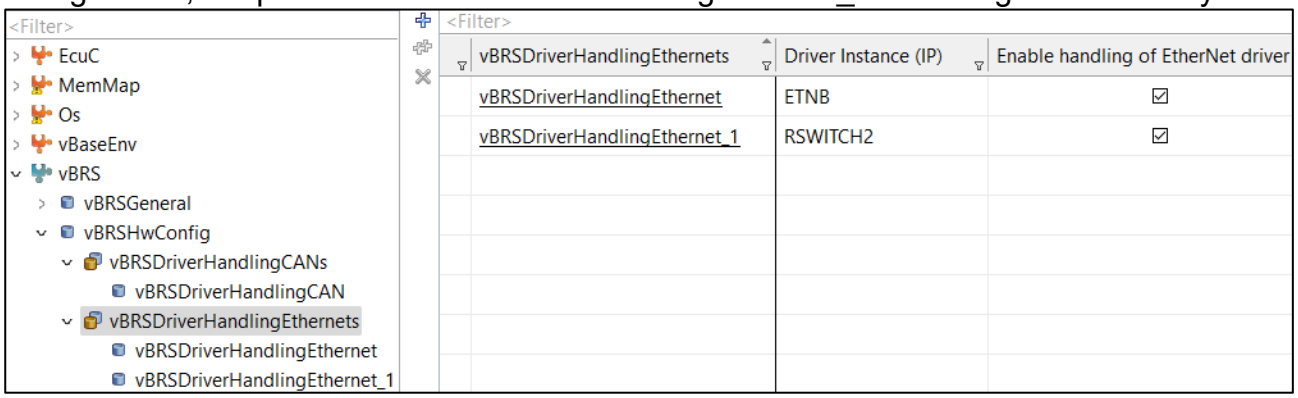


A solving action suggests to enable this parameter, as soon as a DrvEthSwitch is available in the configuration.
The now additional generated BRS_ENABLE_ETHERNET_SWITCH_SUPPORT allows to distinguish such a configuration within the BrsHw implementation.

If more than one Ethernet IP is available on the controller, it could become necessary to select the one, to be used. That's the task of the not mandatory parameter "vBRS/vBRSHwConfig/vBRSDriverHandlingEthernet/vBRSDriverHandlingEthernet_Instance":



It is also possible to configure 2 DrvEth/DrvEthSwitch instances in parallel. In such a configuration, the parameter "vBRSDriverHandlingEthernet_Instance" gets mandatory:

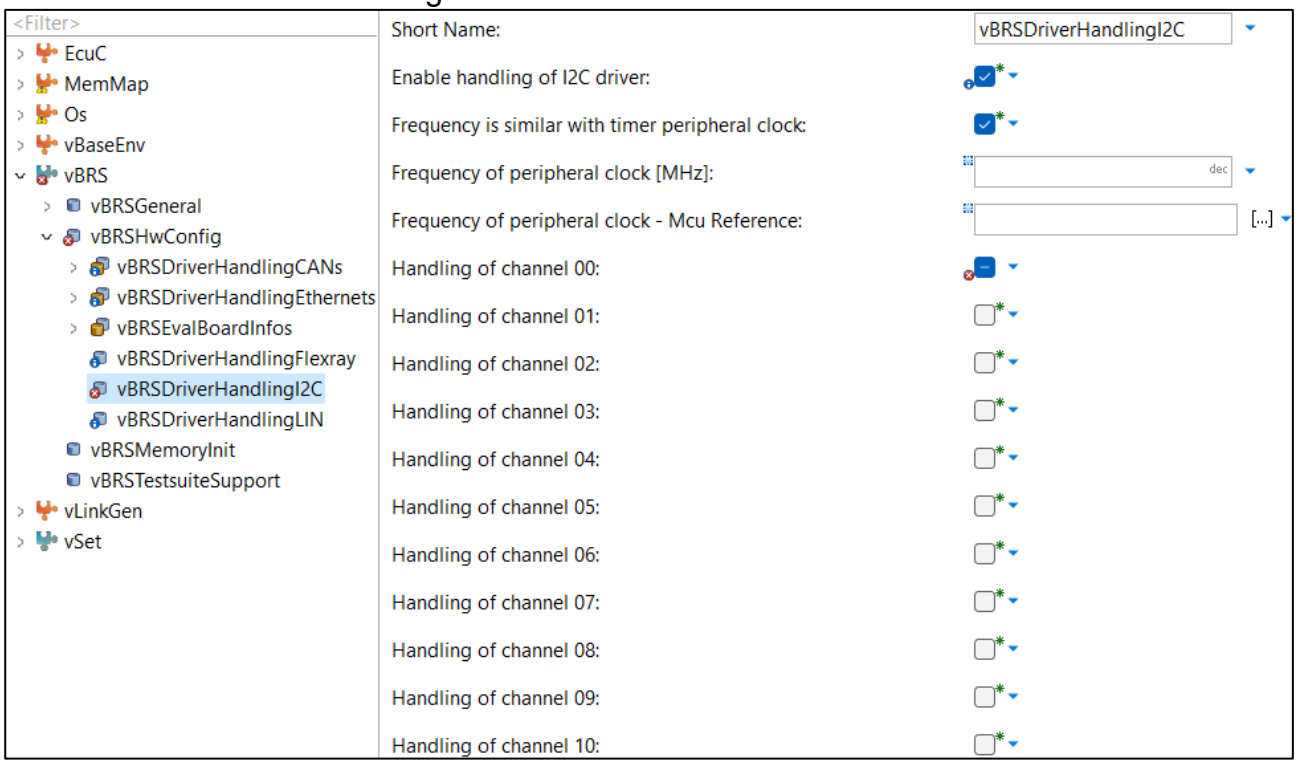


Depending on the selected value of "vBRSDriverHandlingEthernet_Instance", the vBRS will generate the DrvEth/DrvEthSwitch specific defines in vBrsCfg.h with a corresponding infix:
BRS_ETH_infix_CLOCK
BRS_ENABLE_ETHERNET_infix_PORT_x
This allows the BrsHw implementation to differentiate.
Additional to BRS_ENABLE_ETHERNET_SUPPORT, also BRS_ENABLE_ETHERNET_infix_SUPPORT will be generated.
If the configured Ethernet IP is an Ethernet Switch, additional to BRS_ENABLE_ETHERNET_SWITCH_SUPPORT, also BRS_ENABLE_ETHERNET_SWITCH_infix_SUPPORT will be generated.

© 2025 Vector Informatik GmbH

4.8.5 DriverHandlingI2C

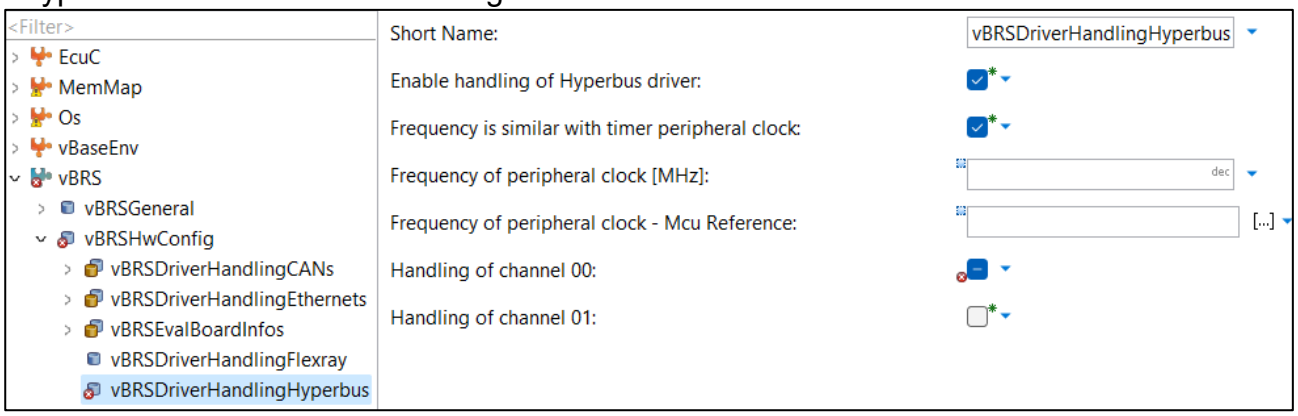
This is not a mandatory container, but a solving action suggests creating it, as soon as a DrvI2c is available in the configuration.



If the handling is enabled, the BrsHw will set the depending peripheral clocks and pin settings during HW initialization.
It is also possible to configure peripheral clocks, if they differ from the peripheral timer clock, configured in vBRSHwConfig/vBRSPeriphClock.

4.8.6 DriverHandlingHyperbus

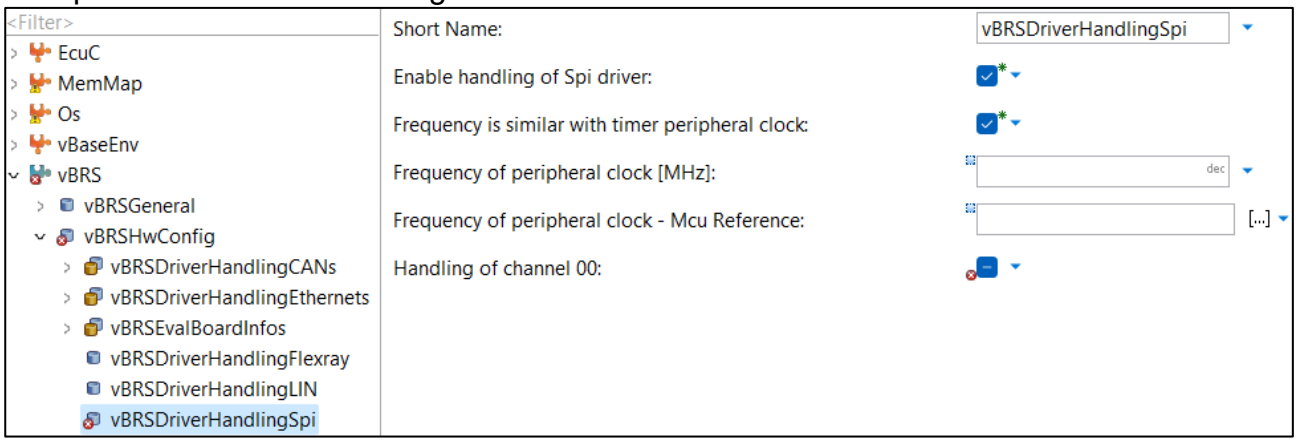
This is not a mandatory container, but a solving action suggests creating it, as soon as a vHyperbus is available in the configuration.



If the handling is enabled, the BrsHw will set the depending peripheral clocks and pin settings during HW initialization.
It is also possible to configure peripheral clocks, if they differ from the peripheral timer clock, configured in vBRSHwConfig/vBRSPeriphClock.

4.8.7 DriverHandlingSpi

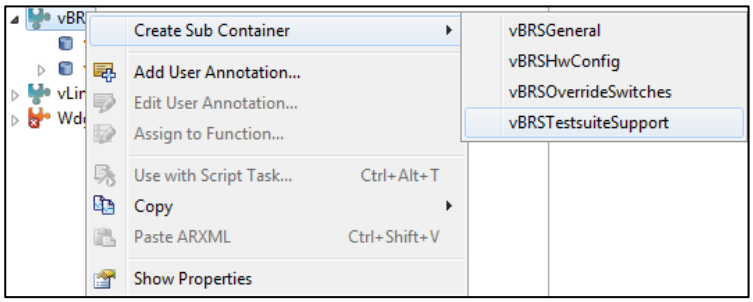
This is not a mandatory container, but a solving action suggests creating it, as soon as a DrvSpi is available in the configuration.



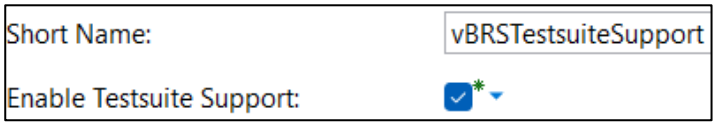
If the handling is enabled, the BrsHw will set the depending peripheral clocks and pin settings during HW initialization.
It is also possible to configure peripheral clocks, if they differ from the peripheral timer clock, configured in vBRSHwConfig/vBRSPeriphClock.

4.9 vBRSTestsuiteSupport

This container is not present per default, it must be created if needed:



In this container, the Testsuite support can be enabled:



If the Testsuite support is enabled, the define BRS_ENABLE_TESTSUITE_SUPPORT is generated into vBrsCfg.h.

4.9.1 Testsuite initialization and cyclic calls

If Testsuite support is enabled, a BrsTestsuite.h must be provided by the testsuite. Inside BrsTestsuite.h, at least the following API must be provided:

API	Description
void BrsTestsuiteInit()	Will be called inside of Default_Init_Task, after the OS was initialized, just before EcuM_StartupTwo() is called.

Inside the Testsuite-local implementation of BrsTestsuiteInit(), it is possible to handle all testsuite specific initialization calls and, via the API BrsMainRegisterCyclic(), it is possible to register cyclic testsuite calls.

Call BrsMainRegisterCyclic() with the following parameters:

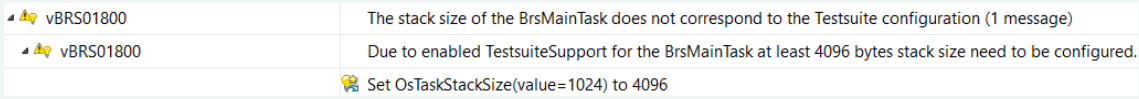
Parameter	Description
FunctionPointer	Must be a pointer to a function of type void function(void)
Cycletime	Describes the cycletime, the callback should be triggered with. Possible values are: BRSMAIN_CYCLETIME_BACKGROUND BRSMAIN_CYCLETIME_1MS BRSMAIN_CYCLETIME_10MS BRSMAIN_CYCLETIME_100MS BRSMAIN_CYCLETIME_250MS BRSMAIN_CYCLETIME_500MS BRSMAIN_CYCLETIME_1000MS

Example:

```
BrsMainRegisterCyclic(BrsMainWrapperTogglePinLED, BRSMAIN_CYCLETIME_500MS);
```

Note

As the cyclic handling is done within the task BrsMainTask, the default task stack size of the OS (1024) will not be sufficient for more than only a few cyclic calls. The vBRS checks the task stack size of BrsMainTask and recommends increasing it, if vBRSTestsuiteSupport is activated. The recommended value 4096 is just a recommendation, based on typically needs of driver development test suites at PES. It is solved automatically via “solve all”.



4.10 vBRS module stubs

In addition to the generated EcuM and SchM stubs (see chapter 4.5 and 4.6), there exist several additional static stub implementations inside the vBRS package.

These module stubs are UseCase specific (e.g., incomplete SIP projects) and not always part of the environment (there exist only rare cases, where these stubs are needed).

Modul Stub	Included files	Description
CanIf	CanIf.c CanIf.h CanIf_Cbk.h CanIf_GenertatedTypes.h	The CAN Interface stub provides callbacks, types and enums, necessary to use a CAN driver w/o CAN stack.
CanSM	CanSM.c CanSM_Cbk.h	The CAN StateManager stub provides callbacks, necessary to use a CanIf w/o complete CAN stack.
Dem	Dem.c Dem.h	The Diagnostic Event Manager stub provides types, EventParameter and APIs, necessary to use MCAL modules w/o diagnostic stack. Keep in mind: the DemConf_DemEventParameter_<EVENT_NAME> parameters in Dem.h are key sensitive!
Det	Det.c Det.h	The Development Error Tracer stub provides types and APIs, necessary to use the module Det checks w/o real Det.
Eth	Eth.c Eth.h Eth_GeneratedTypes.h Eth_Types.h	The Ethernet Driver stub provides types and APIs, necessary to use Ethernet stacks w/o real Ethernet driver.
EthIf	EthIf.c EthIf.h EthIf_Cbk.h	The Ethernet Interface stub provides callbacks, types and enums, necessary to use an Ethernet driver w/o Ethernet stack.
EthTrcv	EthTrcv.h EthTrcv_30_EthCt.c EthTrcv_30_EthCt.h	The Ethernet Transceiver stub provides callbacks and types, necessary to use an Ethernet stack w/o Ethernet transceiver driver.
LinIf	LinIf.c LinIf.h LinIf_Cbk.h LinIf_GeneralTypes.h	The LIN Interface stub provides callbacks, types and enums, necessary to use a LIN driver w/o LIN stack.

Modul Stub	Included files	Description
MemIf	MemIf.h MemIf_Types.h	The Memory Abstraction Interface stub provides types and enums, necessary to use DrvFls and Fee modules w/o memory stack.
NvM	NvM.c NvM.h NvM_Cbk.h	The Non-Volatile Manager stub provides types and APIs, necessary to use diagnostic stacks w/o memory stack.
OS	Os.c Os.h Os_MemMap.h	The Operating System stub provides the mandatory APIs and allows the initialization and cyclic program flow of a vBaseEnv environment. All function macros and the function stubs will provide the functionality of a cyclic 1ms handling inside BrsMainTask. No code adaptions inside the extended task implementation of BrsMainTask are necessary for this. The Alarm BrsEvCyclicAlarm_1ms is used as 1ms flag, which is triggered inside the ISR TimeBaseInterrupt() and read within the endless for loop of BrsMainTask. This mechanism will cause the function BrsMainCyclic1ms() to be called every 1ms, if the ISR TimeBaseInterrupt() is handled every 1ms. The flag reset is done within ClearEvent().
Rte	Rte.c Rte.h Rte_Main.h	The Run-Time Environment stub provides APIs, necessary to use stacks w/o real RTE.
WdgIf	WdgIf_Types.h	The Watchdog Interface stub provides types and enums, necessary to use DrvWd modules w/o watchdog stack.

4.10.1 vBRS module stubs callout concept

To enable the usage of callout calls into your local testsuite implementation, it is possible to activate a BrsTestuite callout inside of every module stub API, provided by vBRS.

To enable the single callout, set the following define inside BrsTestsuite.h:

<Modulname>_STUB_<Functionname>_CALLOUT

Example:

#define ECUM_STUB_INIT_CALLOUT

The testsuite callout itself must be named according the following schema:

BrsTestsuite_<Modulname>_<Functionname>_Callout()

Example:

BrsTestsuite_EcuM_Init_Callout()

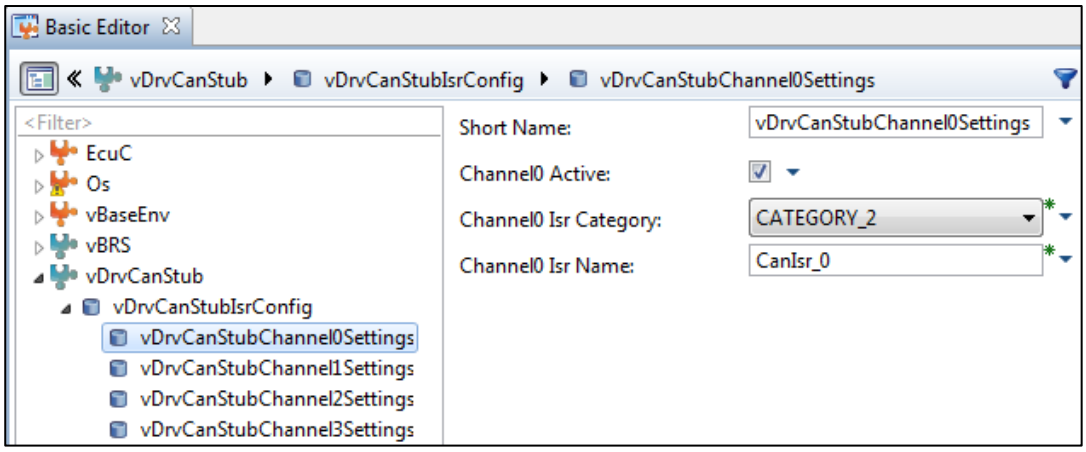
See here the complete example implementation of EcuM_Init(), generated into EcuM.c:

```
void EcuM_Init(void)
{
#if defined (ECUM_STUB_INIT_CALLOUT)
  BrsTestsuite_EcuM_Init_Callout();
#endif
}
```

If the API has parameters and/or return values, also the testsuite callout must provide these.

4.11 vDrvCanStub

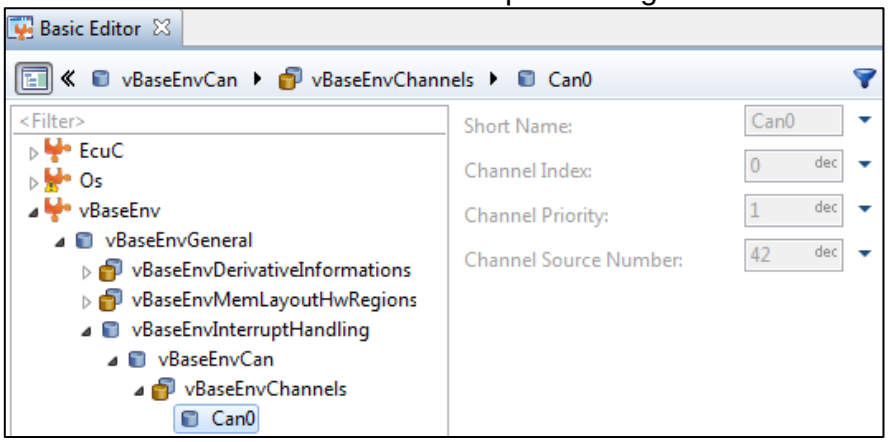
For projects, where the DrvCan configuration is not part of the Configurator5 (e.g., CANbedded, configured in GENy), there exists the additional module vDrvCanStub, which is also part of the vBRS.



This module will help with the configuration of the vBaseEnv modules, which have dependencies to the driver module. If the driver configuration itself is part of the Configurator5 configuration, this is done via solving actions and several checks. The vDrvCanStub will function as wrapper between the different configuration tools.

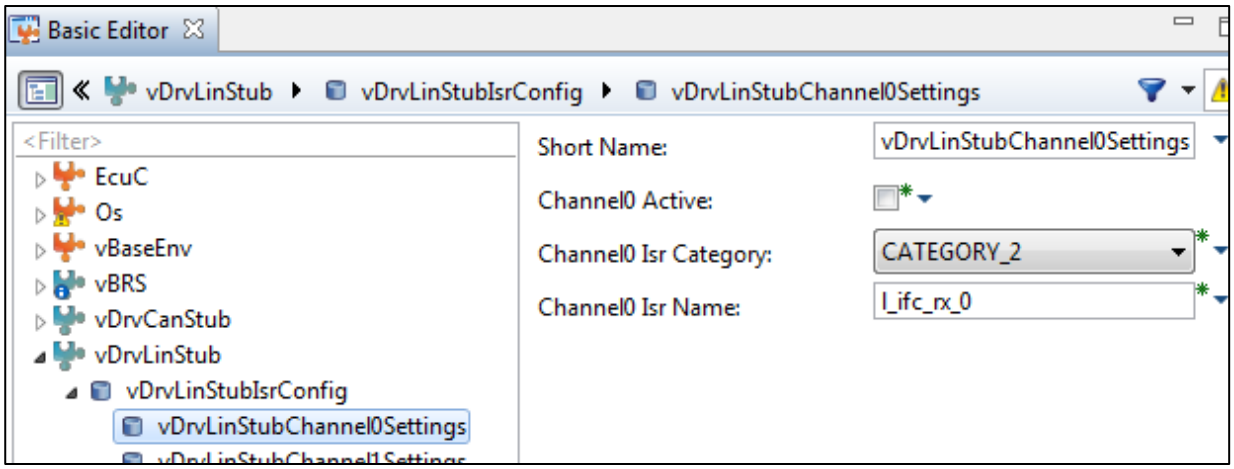
The following support is implemented:

- Adjustment of the channel handling configuration between active channels in vDrvCanStub and vBRSDriverHandlingCAN via solving actions.
- Configuration of the channel depending interrupts into the OS Isr container.
  - Interrupt category and name must be configured in vDrvCanStub. A later adjustment in the OS configuration is not possible later (this is a limitation of the Configurator5 itself).
  - If available for your vBaseEnv platform, also a default value for the interrupt channel source number and channel priority is configured automatically into the OS Isr container. If available, these settings are visible through vBaseEnvGeneral/vBaseEnvInterruptHandling:



4.12 vDrvLinStub

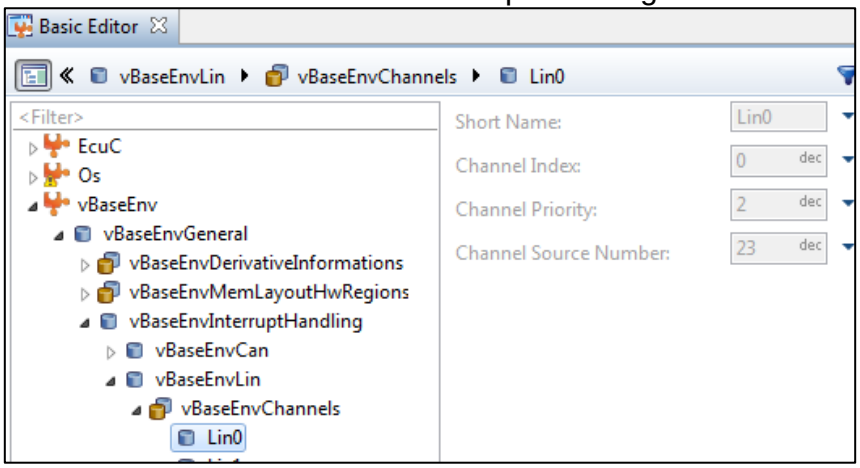
For projects, where the DrvLin configuration is not part of the Configurator5 (e.g., CANbedded, configured in GENy), there exists the additional module vDrvLinStub, which is also part of the vBRS.



This module will help with the configuration of the vBaseEnv modules, which have dependencies to the driver module. If the driver configuration itself is part of the Configurator5 configuration, this is done via solving actions and several checks. The vDrvLinStub will function as wrapper between the different configuration tools.

The following support is implemented:

- Adjustment of the channel handling configuration between active channels in vDrvLinStub and vBRSDriverHandlingLIN via solving actions.
- Configuration of the channel depending interrupts into the OS Isr container.
  - Interrupt category and name must be configured in vDrvLinStub. A later adjustment in the OS configuration is not possible later (this is a limitation of the Configurator5 itself).
  - If available for your vBaseEnv platform, also a default value for the interrupt channel source number and channel priority is configured automatically into the OS Isr container. If available, these settings are visible through vBaseEnvGeneral/vBaseEnvInterruptHandling:



4.13 vBRSMemoryInit

If inside the vLinkGen configuration, MemoryRegionBlocks or VarSectionGroups have configured the Init Policy “ZERO_INIT” and the Init Stage to “ZERO”, “HARD_RESET_ONLY”, “ONE”, “TWO”, “THREE” or “POWER_ON”, these memory areas will be initialized within Brs_ApplicationEntry() of BrsMainStartup.c (see chapter 6.1.1 for details).

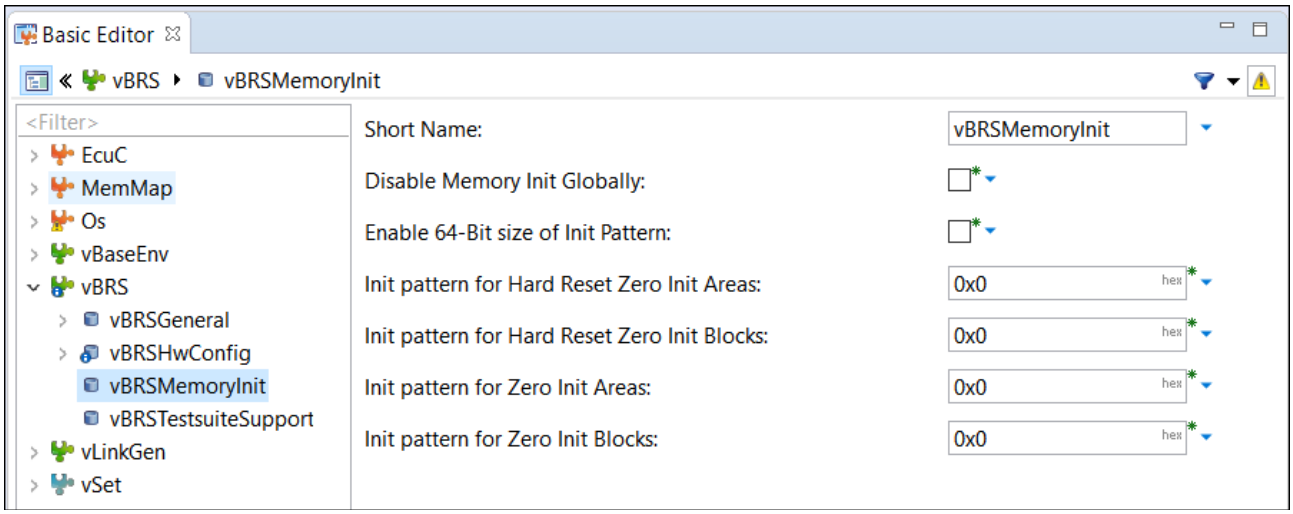
The checkbox "Disable Memory Init Globally" allows a global deactivation of Brs_MemoryInit() calls within Brs_ApplicationEntry(). The Brs_MemoryZeroInit() calls are still available within Brs_ApplicationEntry() and are not excluded.

Caution
This functionality is mainly intended for flashless devices, where the application image is copied from external memory into RAM by a boot loader.

Be careful, as the linker needs to be configured in a way, that the init values are located at the variables addresses directly.

The checkbox “Enable 64-Bit size of Init Pattern” allows the configuration of 64-Bit init patterns for the memory initialization.

Per default, these areas will be initialized with zeroes. Within this container, it is possible to configure different 4-Byte or 8-Byte values for these initialization algorithms.



If the depending “HW Memory Area” is configured to a specific 8-Byte ECC alignment (vBaseEnv/vBaseEnvGeneral/vBaseEnvDerivativeInformation/vBaseEnvMemLayoutHwRegion/vBaseEnvHwRegionEccAlignment) and 64-bit pattern is deactivated, the here configured init pattern will simply be doubled.

5 BrsHw

The module BrsHw contains the platform and compiler specific implementation.

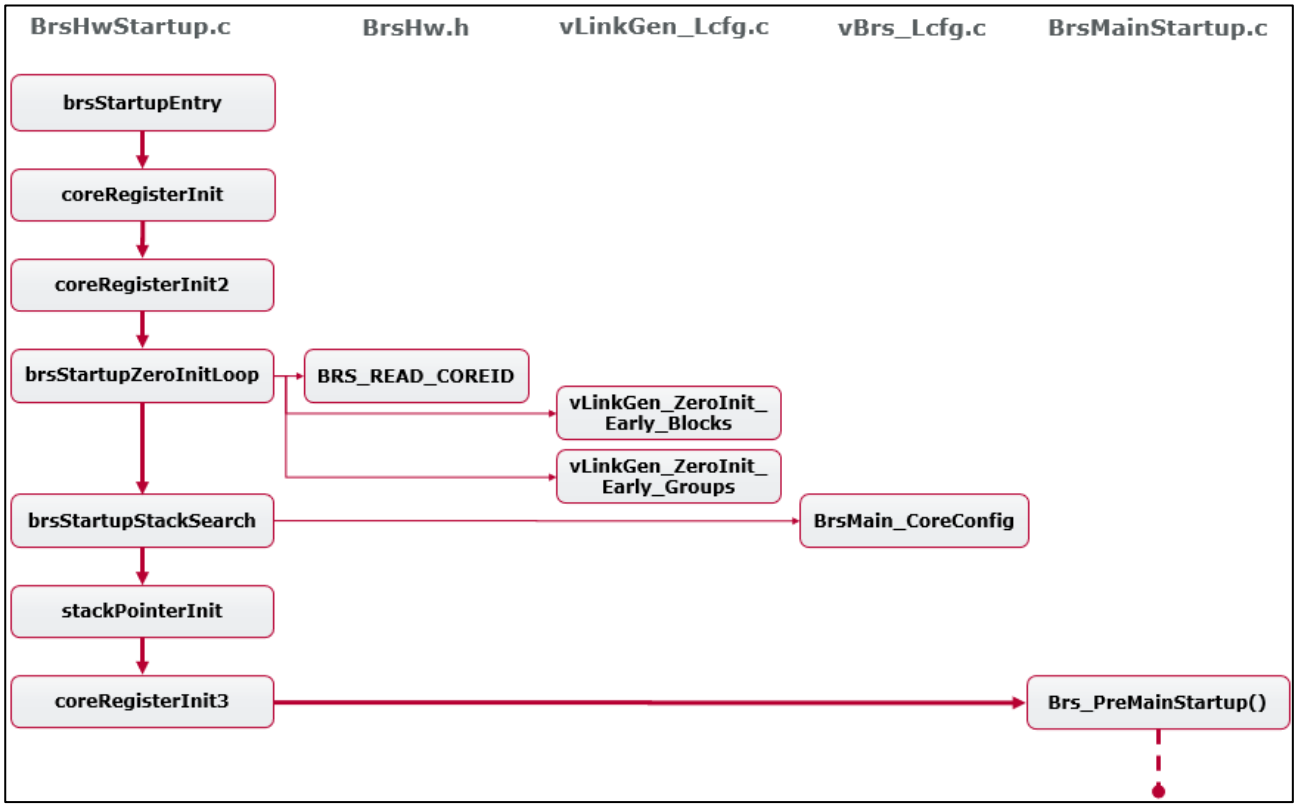
5.1 Startup code

Normally part of the file BrsHwStartup.c, this code provides the fundamental initialization of the core. It is typically written in assembler, starts with the startup entry label brsStartupEntry and ends after the stackpointer initialization with the jump to Brs_PreMainStartup().

The following steps are completed, after the startup code was traversed and the program sequence reaches Brs_PreMainStartup():

- core registers are initialized (if necessary)
- core exception table (brsExcVect) initialized
- the memory blocks and groups, configured for Init stage “EARLY”, init policy “ZERO_INIT” and the actual running core ID are zeroed
- the startup stack end label, suitable for the actual running core, is taken from the generated BrsMain_CoreConfig and initialized accordingly

Typical program flow and dependencies:



5.2 Hardware initialization

The file BrsHw.c contains several standardized APIs, to initialize all necessary parts of the HW, that are needed to use it within a standard embedded project (e.g., MSR4 stack with CAN communication). Some parts of this implementation are also part of a classic Autosar MCAL, therefore it is possible to deactivate them (e.g. PLL initialization, which is part of a DrvMcu).

5.2.1 BrsHwEarlyInitPowerOn

This function can be used to initialize controller specific stuff that should be handled by the first execution instance, even before the memory initialization took place (for example everything that is necessary to access the memory or to handle exceptions). This function is called at the beginning of BrsMainStartup(), immediately after the stackpointer was initialized in StartupCode. This API is not mandatory and enabled via the define BRSHW_EARLYINIT_AVAILABLE in BrsHw.h.

5.2.2 BrsHwApplicationInit

This function can be used to initialize controller specific stuff that should be handled by every execution instance, even before the memory initialization took place (for example everything that needs to be re-programmed to fit with the memory layout of the actual execution instance). This function is called at the beginning of Brs_ApplicationEntry(), immediately after it was called from Brs_PreMainStartup() or another execution instance. This API is not mandatory and enabled via the define BRSHW_APPLICATIONINIT_AVAILABLE in BrsHw.h.

5.2.3 BrsHwPreInitPowerOn

This function must be used to initialize controller specific stuff that is not related to one of the other InitPowerOn functions (e.g., code that is always mandatory, also if MCAL and/or OS is used). This function is called from BrsMain immediately after BrsHwDisableInterruptAtPowerOn() during initialization. This API is not mandatory and enabled via the define BRSHW_PREINIT_AVAILABLE in BrsHw.h.

5.2.4 BrsHwWatchdogInitPowerOn

This function must be used to initialize the Watchdog.

Initializing the watchdog in the driver-development context means disabling the watchdog. The controller internal watchdogs should not affect the normal program flow of the whole environment.

If an Autosar MCAL DrvWd is part of the configuration, this API will not be called.

5.2.5 BrsHwPllInitPowerOn

This function must be used to initialize the PLL. This also means to ensure that all peripherals, necessary to use the communication interfaces and mandatory timers, are supplied with a valid clock input. It is configured via the vBRS generated defines for oscillator clock, timebase clock, peripheral clock, and the appropriate clocks of the communication peripherals.

If an Autosar MCAL DrvMcu is part of the configuration, this API will not be called.

5.2.6 BrsHwPortInitPowerOn

This function must be used to initialize the used port pins. This means to configure the correct pin functions (PinMuxing) for all communication channels to be used (e.g., CAN Tx, CAN Rx, transceiver standby, …). To ease the configuration of single pins, some helper functions are implemented:

BrsHwInitPortInput(), BrsHwInitPortOutput(), BrsHwInitPortInputOutput(), BrsHwPort_SetLevel(), BrsHwPort_GetLevel()

The necessary pins are configured in BrsHw_Ports.h.

If an Autosar MCAL DrvPort is part of the configuration, this API will not be called.

5.2.7 BrsHwDisableInterruptAtPowerOn

Disable the global system interrupt. It's the first function call in main@BrsMain.

5.2.8 BrsHw_ExceptionTable_Init

This function is used, to program the vector table base addresses. It is called from main@BrsMain to switch between the core exception table, configured in startup code, and the exception tables of an operating system or a core exception table in RAM (FBL).

5.2.9 BrsHwSoftwareResetECU/BrsHwGetResetReasonStartup

If the running MICROSAR Classic / CANbedded application gets a request to enter the bootloader, it calls this API and expects a SW reset of at least the MICROSAR Classic / CANbedded cores and the used peripherals.

During startup, via a call of BrsHwGetResetReasonStartup(), the SW reset reason is detected (Brs_PreMainStartup()) and the bootloader will be called, instead of the MICROSAR Classic / CANbedded application.

The reset reason is also used to prevent the memory initialization of memory regions and groups, that are configured for initialization after hard reset only (Brs_ApplicationEntry()).

5.2.10 BrsHwTime100NOP

This API is used for the BRS time measurement support to get a default time value for all measurements with this platform to be able to compare time measurements on different dates based on this time result.

5.2.11 BrsHw_EnableHwAccess

This API is used to enable hardware access in untrusted mode, if necessary for this platform.

5.2.12 BrsHw_GetCore

This API is used to read the core ID of the actual running core.

5.2.13 BrsHw_EnableInterrupt / BrsHw_DisableInterrupt / BrsHw_TriggerSoftwareInterrupt

These APIs are used to enable/disable/trigger a single interrupt source in the core interrupt controller.

5.2.14 BrsHwDisableEccErrorReporting

This API is used to disable the flash ECC error reporting, if this is necessary on the platform to grant access to it.

5.2.15 BrsHw_UnlockCores

This API is used as sync barrier. It is used to unlock cores that are locked in the startup code. The implementation is only needed on platforms, where all cores are started automatically after a power-on reset. All additional cores are looped, until the boot core reaches this function.

5.2.16 BrsHw_StartCore

In the use case of Hypervisor multicore configurations, this API is used to start the additional cores. This implementation is only needed on platforms, where cores (except the boot core) are not started automatically after a power-on reset and no OS is present to start the cores.

5.3 Core exception table

The file BrsHwIntTb.c contains the core exception table for startup (used in startup code) and a 2nd one to be copied into RAM for FBL projects.

Each core exception table is placed inside its own memory section:

- brsExcVect
- brsExcVectRam

5.4 MakeSupport

The Vector PES build environment, MakeSupport, is based on GNU make and consists of several parts.

The platform and compiler independent part, called GlobalMake, is mainly implemented within the file Global.Makefile.target.4.mk. This part is not part of BrsHw, it’s a separate module.

The platform and compiler specific parts are part of BrsHw and are implemented in the following files:

- Makefile - project specific settings like compiler path and compiler options; to be modified project specific
- Makefile.static – platform and compiler specific static settings; normally, no changes are needed here

The linker script file, used for linking of the project, is compiler specific generated out of Cfg5, by the module vLinkGen. The default file is configured in Makefile and is Source/vLinkGen_Template.lsl.

6 BrsMain

The module BrsMain contains implementation for several HW-independent functionalities, mainly configured through the vBRS and vLinkGen modules.

6.1 BrsMainStartup

The file BrsMainStartup.c contains the function Brs_PreMainStartup(), if “First execution instance in system” is enabled in vBRSGeneral. This function is always called from the HW-dependent StartUpCode (BrsHwStartup.c). The functions’ task is to call BrsHwEarlyInitPowerOn() on platforms, where this API does exist, and find out the reset reason (via a call of BrsHwGetResetReasonStartup()). The reset reason is saved within the global variable brsMain_ResetReason. As latest, the application entry of the actual running execution instance is called via the call of Brs_ApplicationEntry().



Note
The global variable brsMain_ResetReason is mapped into the section “.brsSharedVar”. The whole Brs_PreMainStartup() is mapped into the section “.brsMainStartup”.

The function Brs_ApplicationEntry() is the application entry of the actual execution instance. If “First execution instance in system” is enabled in vBRSGeneral, it is called from Brs_PreMainStartup(). Otherwise, this is the direct application entry of non-booting execution instances (e.g., FBL, with a previously running BootManager). The functions’ task is to initialize the memory, according to the vLinkGen configuration. All memory region blocks and section groups, configured for “ZeroInit” stage “Early”, will be handled in the HW-dependent StartUpCode, just before the stackpointer is configured. All other memory region blocks and section groups, together with the ROM-to-RAM copy (Init Policy set to “COPY_TO_RAM), will be handled inside the Brs_ApplicationEntry().



Note
The whole Brs_ApplicationEntry() is mapped into the section “.brsApplicationEntry”. This grants a fix address, via mapping of this section into the memory region block “ApplicationEntry”. A proper jump between several execution instances is possible, as the 1st execution instance can be configured accordingly (for example a Boot Manager instance, which jumps into an application instance).



Note
To prevent inlining of the memory init helper functions in BrsMainStartup.c, the prefix BRSMAIN_MEMINIT_FUNC_PREFIX can be configured in BrsMain_Cfg.h.

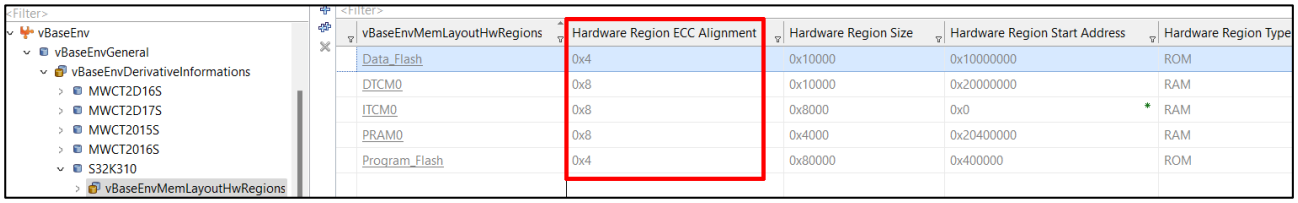
6.1.1 Memory Initialization

Memory initialization is divided into several stages:

- ZERO
- HARD_RESET_ONLY (see chapter 6.1.2)
- ONE
- TWO
- THREE
- POWER_ON

The stages are related to the “Init Stage” setting, configured in the vLinkGen module.

For memory region blocks and section groups, which are configured with Init Policy “ZERO_INIT”, this functionality is used to initialize the memory with a dedicated init value (configured in vBRS/vBRSMemoryInit, see chapter 4.12 for details). It is mainly used for memory with ECC. To support hardware with specific ECC alignments (e.g. mandatory 64-bit access), the configured HW Region ECC Alignment is used here as additional parameter (vBaseEnv/vBaseEnvGeneral/vBaseEnvDerivativeInformation/vBaseEnvMemLayoutHwRegion/vBaseEnvHwRegionEccAlignment).



Brs_ApplicationEntry() is using an ECC alignment of 4-Byte as default and additionally brings support for 8-Byte alignment as well. vLinkGen is generating the depending on information as vLinkGen_MemArea structs into vLinkGen_Lcfg.c.

For memory region blocks and section groups, which are configured with Init Policy “INIT”, this functionality is mainly used for init values of variables. If LogicalConstGroups are configured with Init Policy “COPY_TO_RAM”, the same mechanism is used to copy applications from Flash/ROM to RAM (e.g. FBL applications, before flashing a new application image). vLinkGen is generating the depending on information as vLinkGen_RamMemArea structs into vLinkGen_Lcfg.c.

A designated hook function located inside BrsMain_Callout_Stubs.c is called, prior to each of these init stages (e.g. BrsMain_MemoryInit_StageHardReset_Hook()).

6.1.2 HardReset

This init stage is using the global variable brsMain_ResetReason, which was saved within Brs_PreMainStartup(), to get the reset reason of the actual software start. The dependent memory region blocks and section groups are only initialized, if the reset reason was no software reset.
This functionality is mainly used in FBL Use-Cases, where via execution of a software reset, the application is jumping from application into bootloader. Some memory regions

should not be initialized in this case (e.g., if a RAM region is used to transfer information from application into the bootloader).

6.1.3 Power_On

With the Autosar specification AR21-11, a new requirement was introduced in SWS_MemoryMapping:
“POWER_ON_CLEARED, used for variables that are not explicitly initialized (cleared) during normal start-up. Instead, these are cleared only after either a power on reset of the microcontroller or a power on reset of a battery backup memory itself after battery loss.”
vLinkGen is providing the init stage POWER_ON for exact this requirement. Specific structs are generated for memory init and zero init. But, as there is no general condition available for POWER_ON initialization configured memory, these areas are always initialized in Brs_ApplicationEntry(). A project specific condition could be added manually.

6.2 BrsMain_Callout_Stubs

The file BrsMain_Callout_Stubs.c provides template implementations for all the hook functions, called out of BrsMainStartup.c.
The initial implementations are empty (except BrsMain_MemoryInit_StageHardReset_Hook()) and can be adapted for the individual needs of the projects. BrsMain_MemoryInit_StageHardReset_Hook() does initially contain the implementation for the “Prefer PLL and Watchdog Init” functionality, configurable in vBRSHwConfig.

6.3 BrsMain

The file BrsMain.c contains the function main(), which is always called at the end of Brs_ApplicationEntry(). According to the configured UseCases and features in the vBRS module, further initialization happens here. At the end, the designated stack entry of the actual configured UseCase is called (for example EcuM_Init()).

6.3.1 BrsMainInit

The function BrsMainInit() is called from main(), on the init core. This function handles basic initialization of the features, configured inside the vBRS module. E.g., the HW-dependent initialization for Watchdog, PLL and PortPins are called from here.

6.3.2 BrsMainTogglePin

The function BrsMainTogglePin() provides the functionality, to toggle single pins. Up to three different pins can be configured inside the vBRS module. The BrsMain millisecond handler provides the cyclic call of BrsMainTogglePin() to enable an easy alive visualization, by toggling e.g., a LED pin every 500ms. Triggering external watchdogs by a dedicated pin can also be done.

6.3.3 Default Tasks

The file BrsMain.c contains the implementation of several tasks, created by the vBRS module, if the dependent features are enabled and an OS is part of the configuration.

- Default_Init_Task – Always created as “autostart” task. One separate task per core. Sets the alarm for the BrsMainTask.
- Default_Init_Task_Trusted - Always created as “autostart” task. One separate task per core.
- BrsMainTask – Main task for the BrsMain millisecond handler, used for cyclic pin toggling and testsuite support.
- BrsMainBackgroundTask – Mainly used for testsuite support.

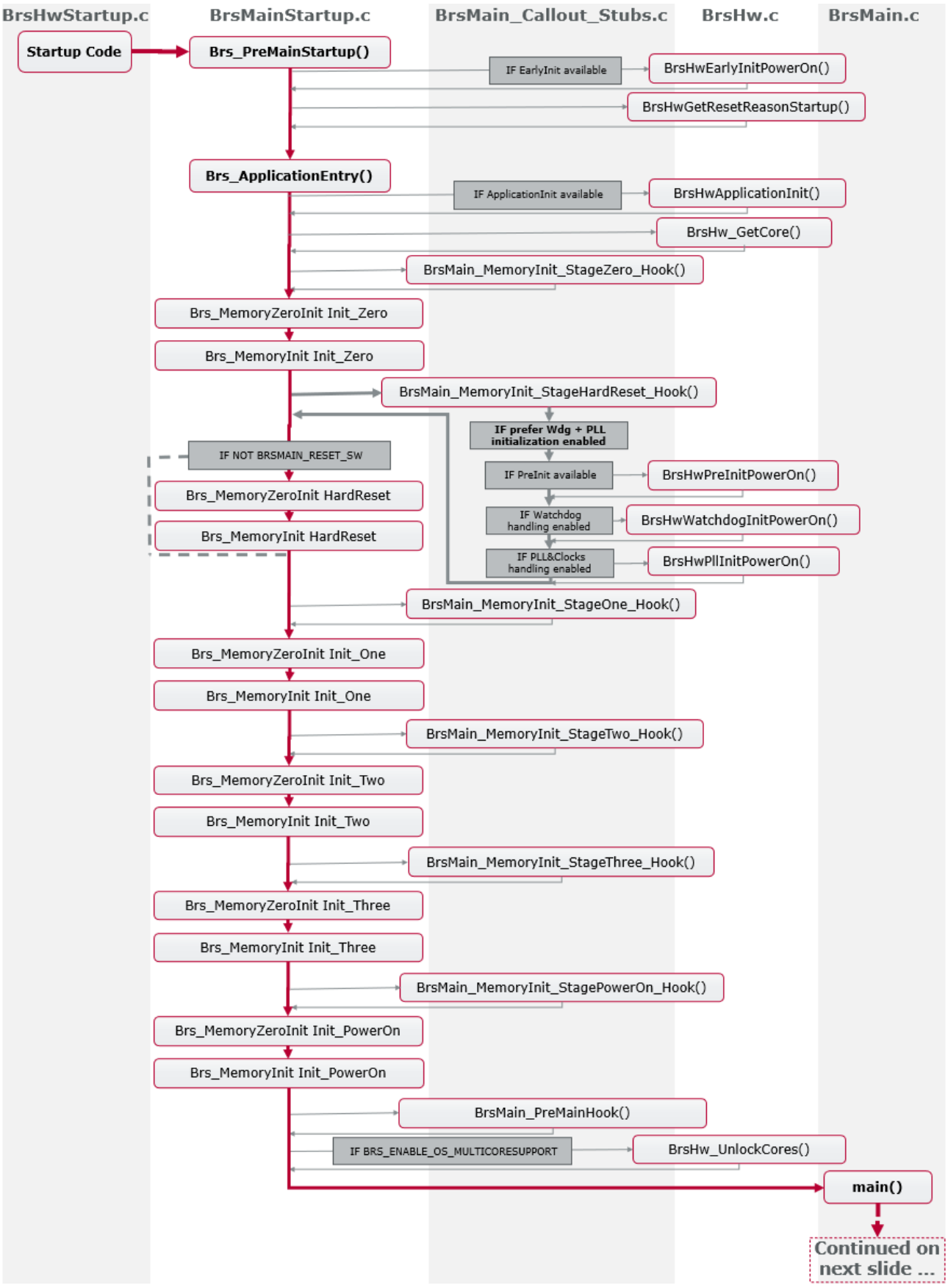
6.3.4 BrsMainExceptionHandler

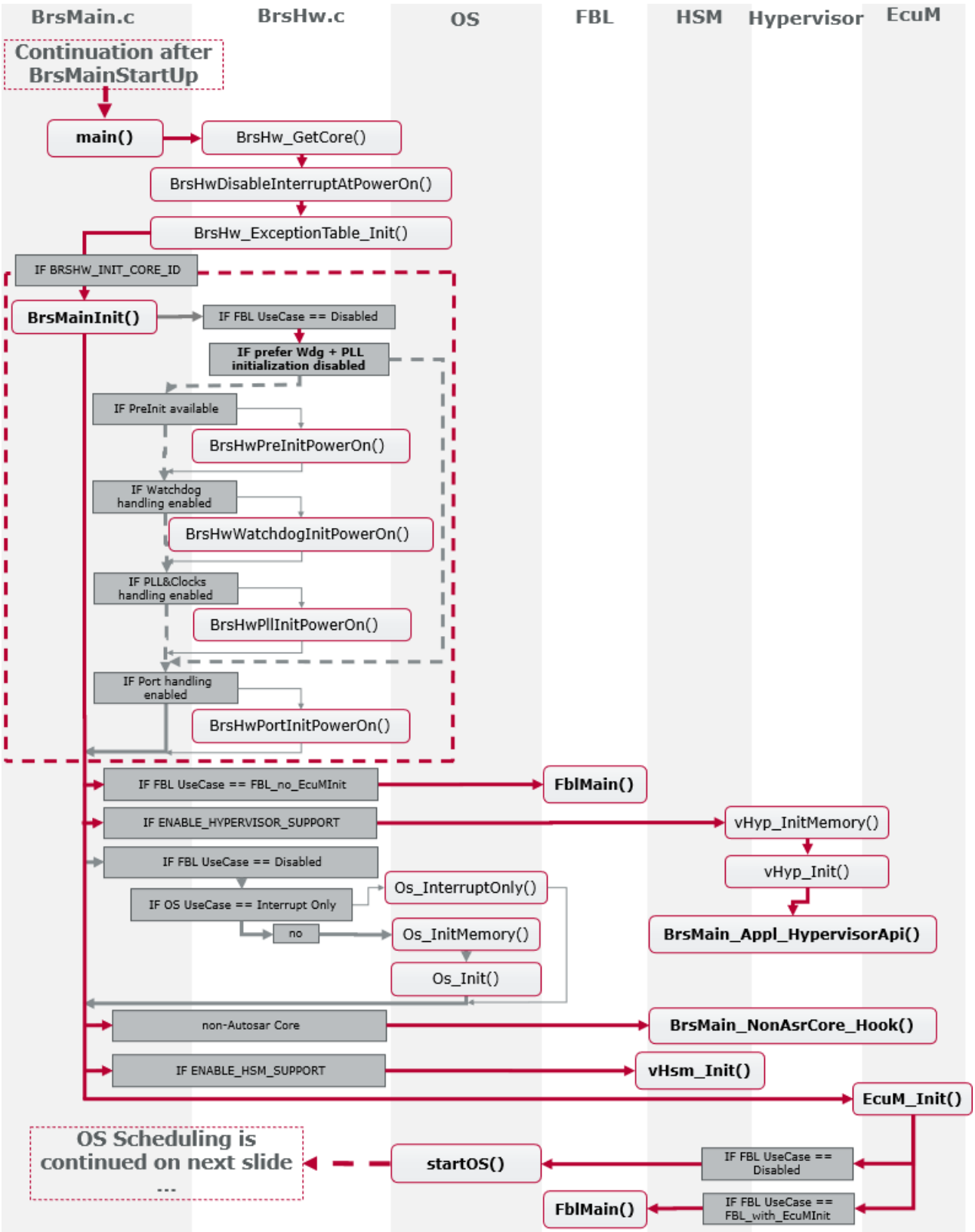
The function BrsMainExceptionHandler() provides easy functionality for a BRS internal error handling. Dedicated ErrorCodes and ModuleCodes are used for this.

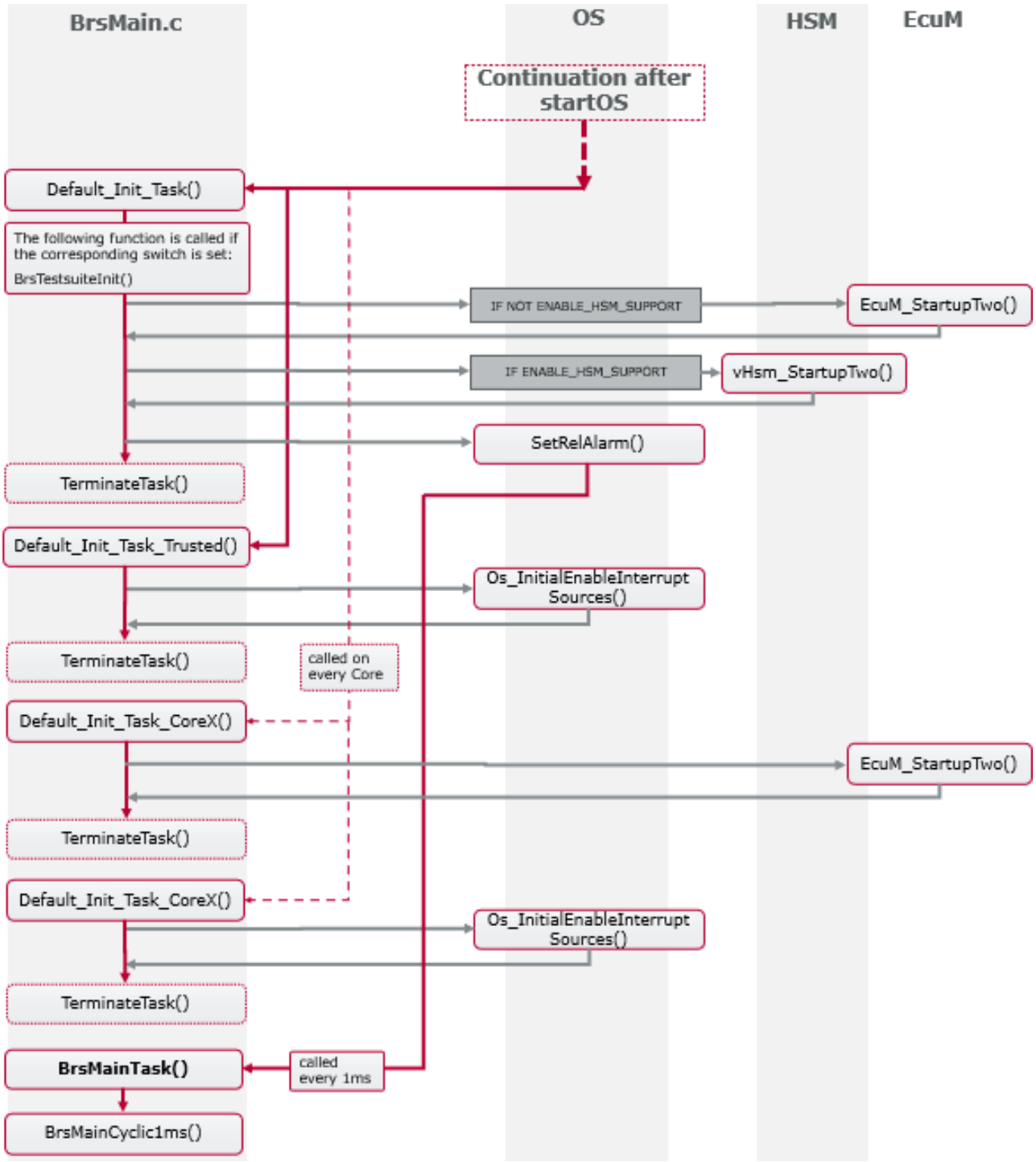
7 Program Flow

Following diagrams visualize the program flow. The affected files/modules are on the top. To increase the visibility, the program flow is split into several diagrams.

© 2025 Vector Informatik GmbH

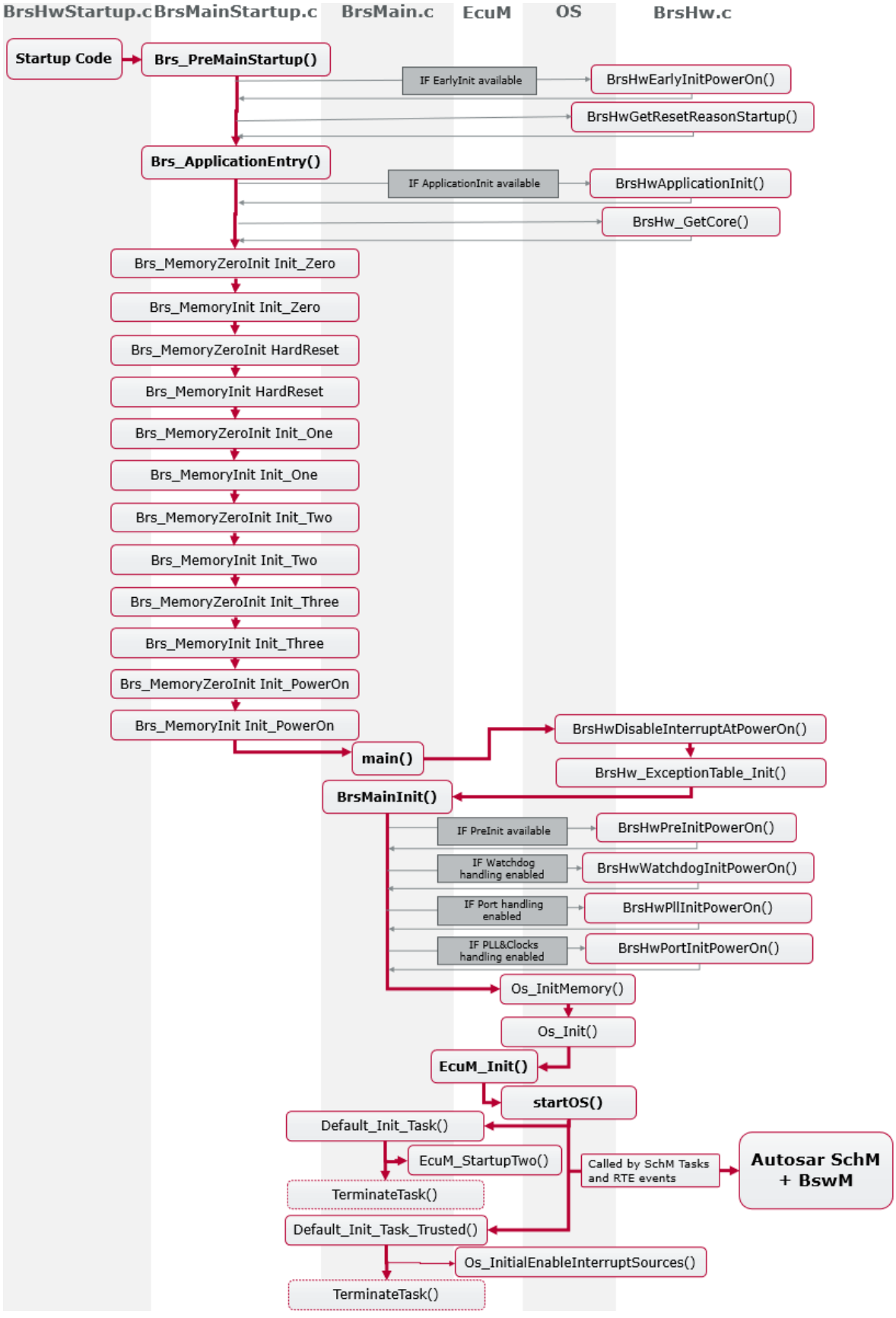

BrsHwStartup.c
BrsMainStartup.c
BrsMain_Callout_Stubs.c
BrsHw.c
BrsMain.c
Startup Code
Brs_PreMainStartup()
IF EarlyInit available
BrsHwEarlyInitPowerOn()
BrsHwGetResetReasonStartup()
Brs_ApplicationEntry()
IF ApplicationInit available
BrsHwApplicationInit()
BrsHw_GetCore()
BrsMain_MemoryInit_StageZero_Hook()
Brs_MemoryZeroInit Init_Zero
Brs_MemoryInit Init_Zero
BrsMain_MemoryInit_StageHardReset_Hook()
IF prefer Wdg + PLL initialization enabled
IF NOT BRSMAIN_RESET_SW
IF PreInit available
BrsHwPreInitPowerOn()
Brs_MemoryZeroInit HardReset
IF Watchdog handling enabled
BrsHwWatchdogInitPowerOn()
Brs_MemoryInit HardReset
IF PLL&Clocks handling enabled
BrsHwPllInitPowerOn()
BrsMain_MemoryInit_StageOne_Hook()
Brs_MemoryZeroInit Init_One
Brs_MemoryInit Init_One
BrsMain_MemoryInit_StageTwo_Hook()
Brs_MemoryZeroInit Init_Two
Brs_MemoryInit Init_Two
BrsMain_MemoryInit_StageThree_Hook()
Brs_MemoryZeroInit Init_Three
Brs_MemoryInit Init_Three
BrsMain_MemoryInit_StagePowerOn_Hook()
Brs_MemoryZeroInit Init_PowerOn
Brs_MemoryInit Init_PowerOn
BrsMain_PreMainHook()
IF BRS_ENABLE_OS_MULTICORESUPPORT
BrsHw_UnlockCores()
main()
Continued on next slide ...


BrsMain.c
BrsHw.c
OS
FBL
HSM
Hypervisor
EcuM
Continuation after BrsMainStartUp
main()
BrsHw_GetCore()
BrsHwDisableInterruptAtPowerOn()
BrsHw_ExceptionTable_Init()
IF BRSHW_INIT_CORE_ID
BrsMainInit()
IF FBL UseCase == Disabled
IF prefer Wdg + PLL initialization disabled
IF PreInit available
BrsHwPreInitPowerOn()
IF Watchdog handling enabled
BrsHwWatchdogInitPowerOn()
IF PLL&Clocks handling enabled
BrsHwPllInitPowerOn()
IF Port handling enabled
BrsHwPortInitPowerOn()
IF FBL UseCase == FBL_no_EcuMInit
FblMain()
IF ENABLE_HYPERVISOR_SUPPORT
vHyp_InitMemory()
vHyp_Init()
BrsMain_Appl_HypervisorApi()
IF FBL UseCase == Disabled
IF OS UseCase == Interrupt Only
Os_InterruptOnly()
no
Os_InitMemory()
Os_Init()
non-Autosar Core
BrsMain_NonAsrCore_Hook()
IF ENABLE_HSM_SUPPORT
vHsm_Init()
EcuM_Init()
OS Scheduling is continued on next slide ...
startOS()
IF FBL UseCase == Disabled
FblMain()
IF FBL UseCase == FBL_with_EcuMInit


BrsMain.c
OS
HSM
EcuM
Continuation after startOS
Default_Init_Task()
The following function is called if the corresponding switch is set:
BrsTestsuiteInit()
IF NOT ENABLE_HSM_SUPPORT
EcuM_StartupTwo()
IF ENABLE_HSM_SUPPORT
vHsm_StartupTwo()
SetRelAlarm()
TerminateTask()
Default_Init_Task_Trusted()
Os_InitialEnableInterrupt Sources()
TerminateTask()
called on every Core
Default_Init_Task_CoreX()
EcuM_StartupTwo()
TerminateTask()
Default_Init_Task_CoreX()
Os_InitialEnableInterrupt Sources()
TerminateTask()
BrsMainTask()
called every 1ms
BrsMainCyclic1ms()

7.1 Sample for MSR4

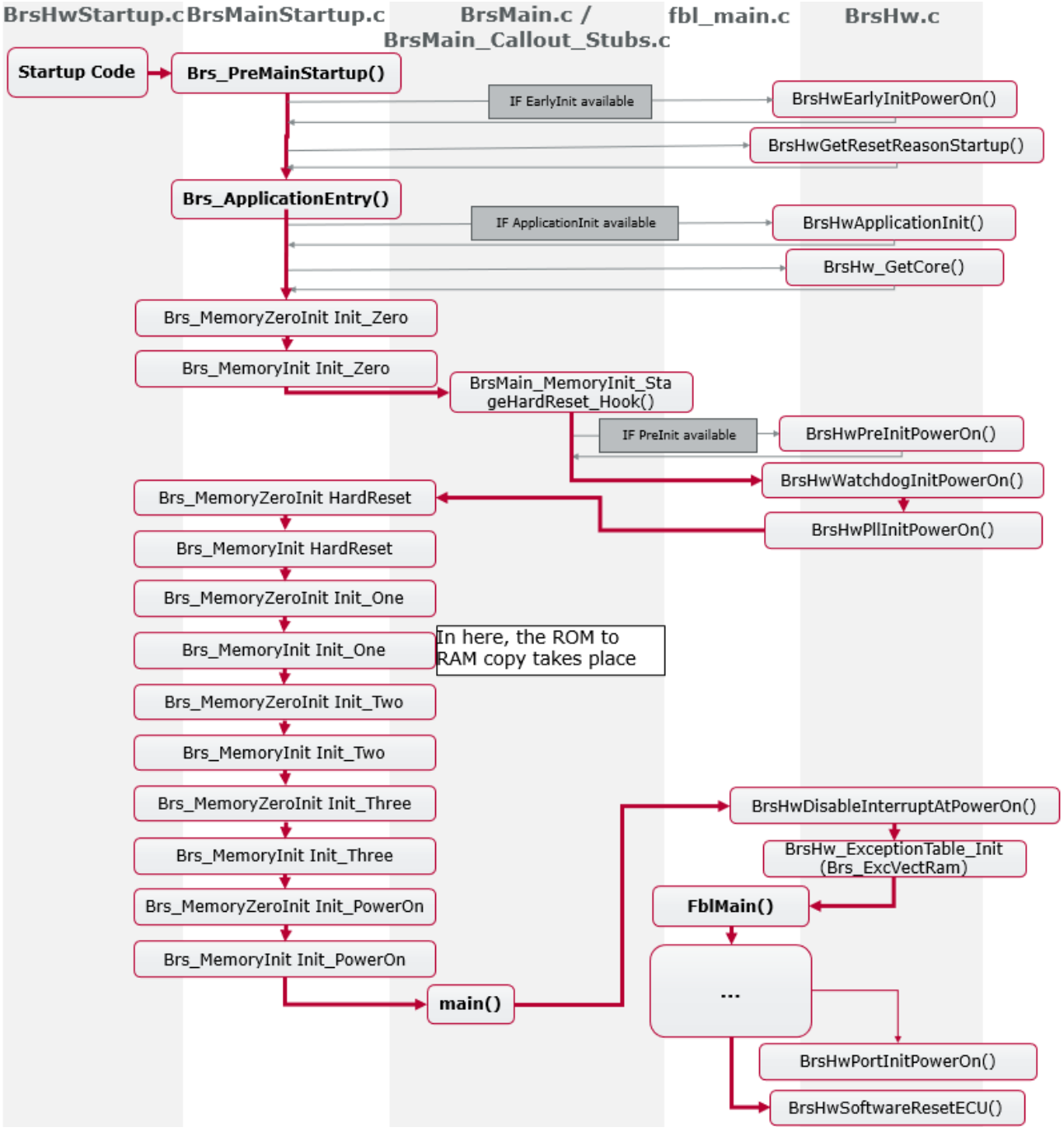
This shows a sample for a default program flow of a MICROSAR Classic project.



7.2 Sample for FBL

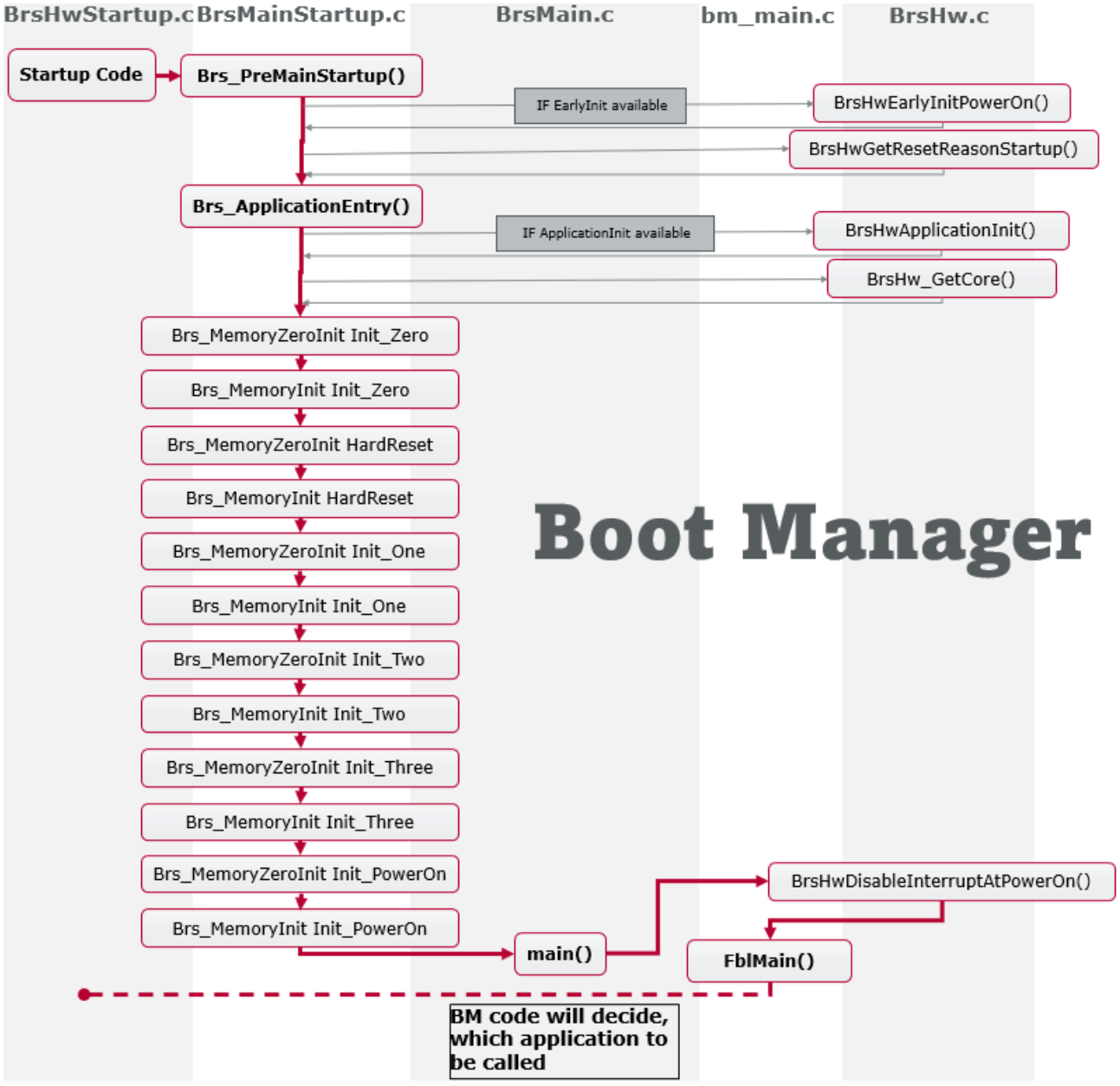
7.2.1 FBL without BM

This shows a sample for a default program flow of a classic Flash Bootloader project, where no Boot Manager component is available.



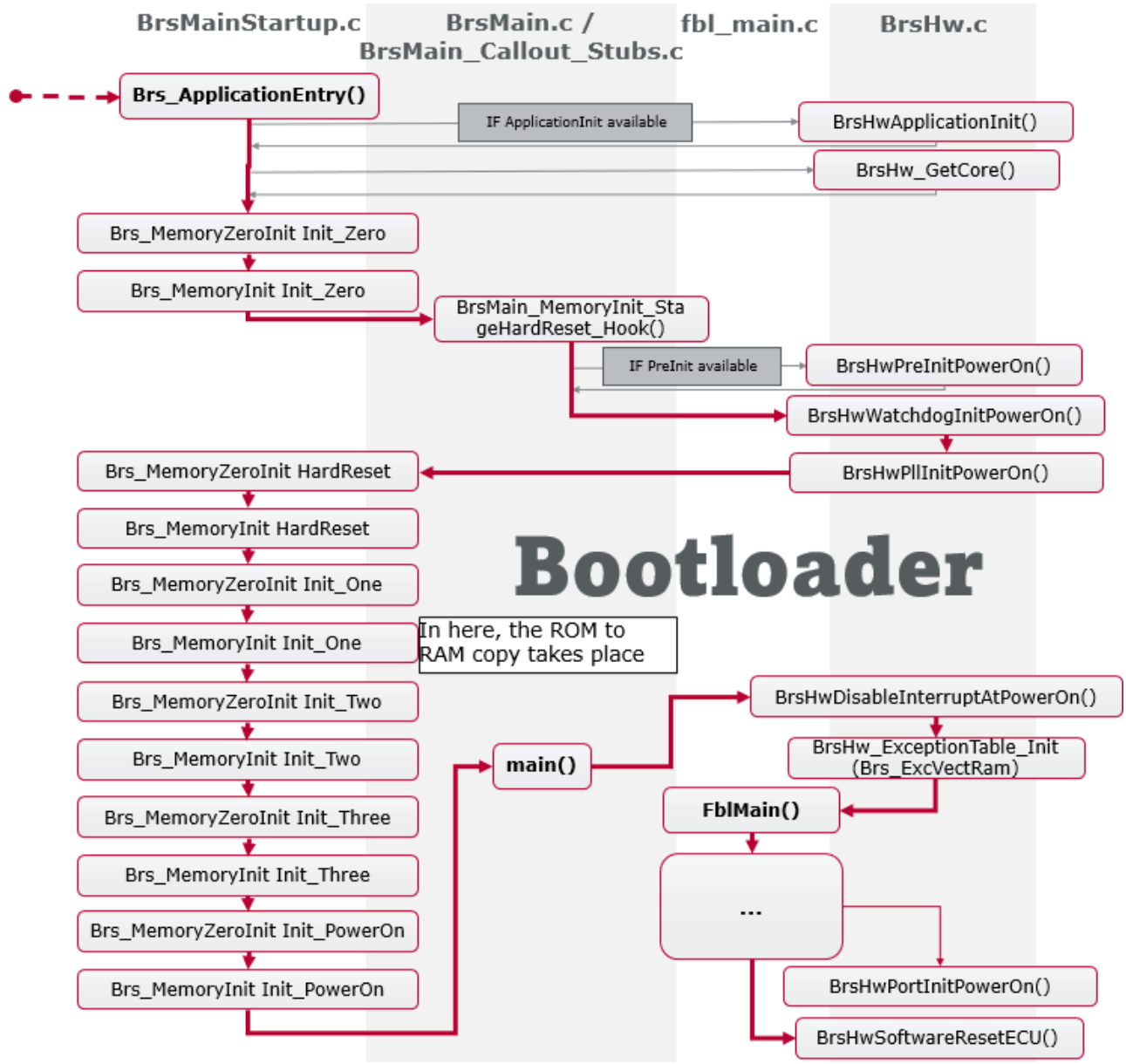
7.2.2 FBL with BM

This shows a sample for a default program flow of a Boot Manager FBL project.

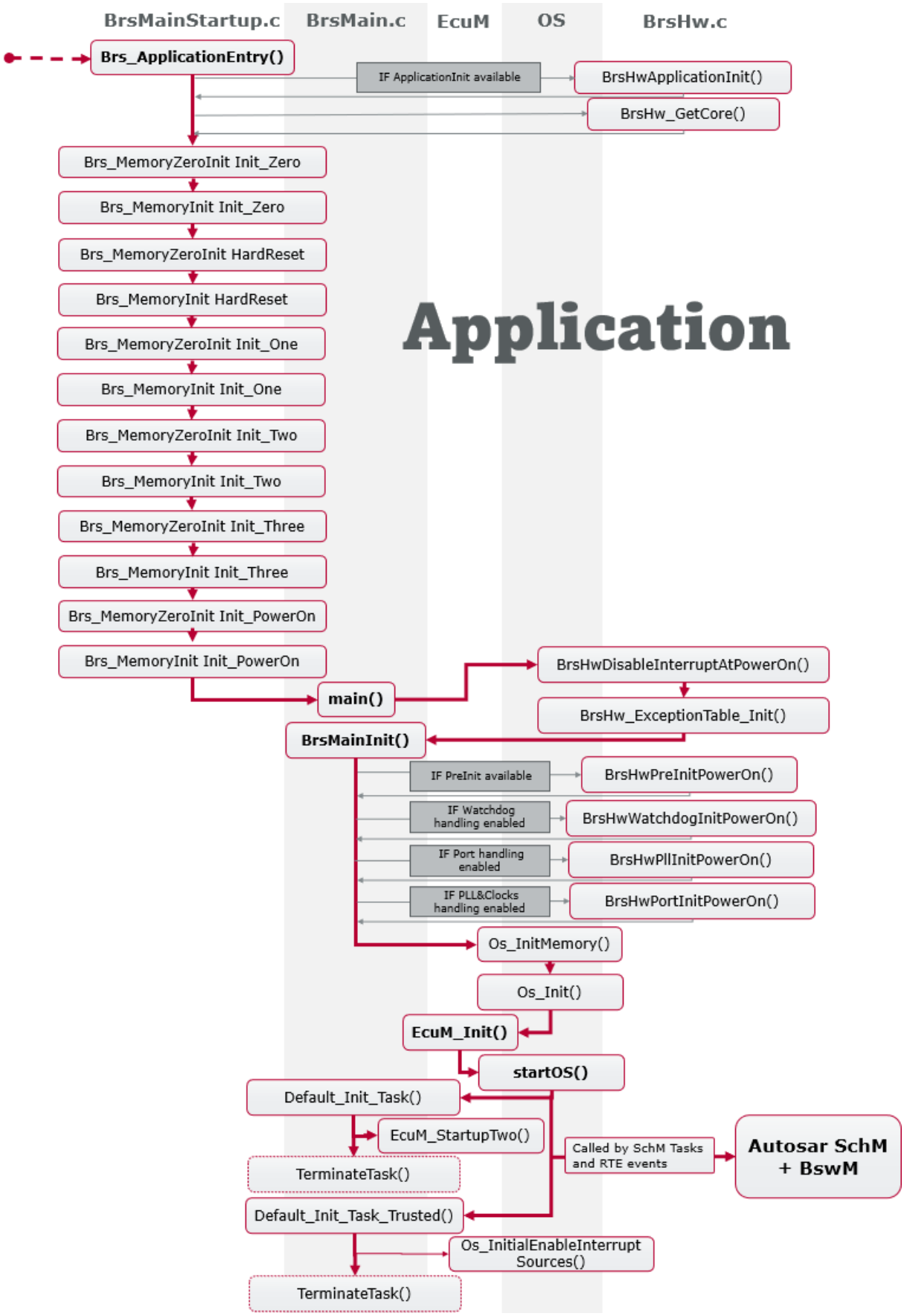


The BM code holds the ApplicationEntry adresses of several execution instances and decides which one will be called this time.

A Flash Bootloader could be called.

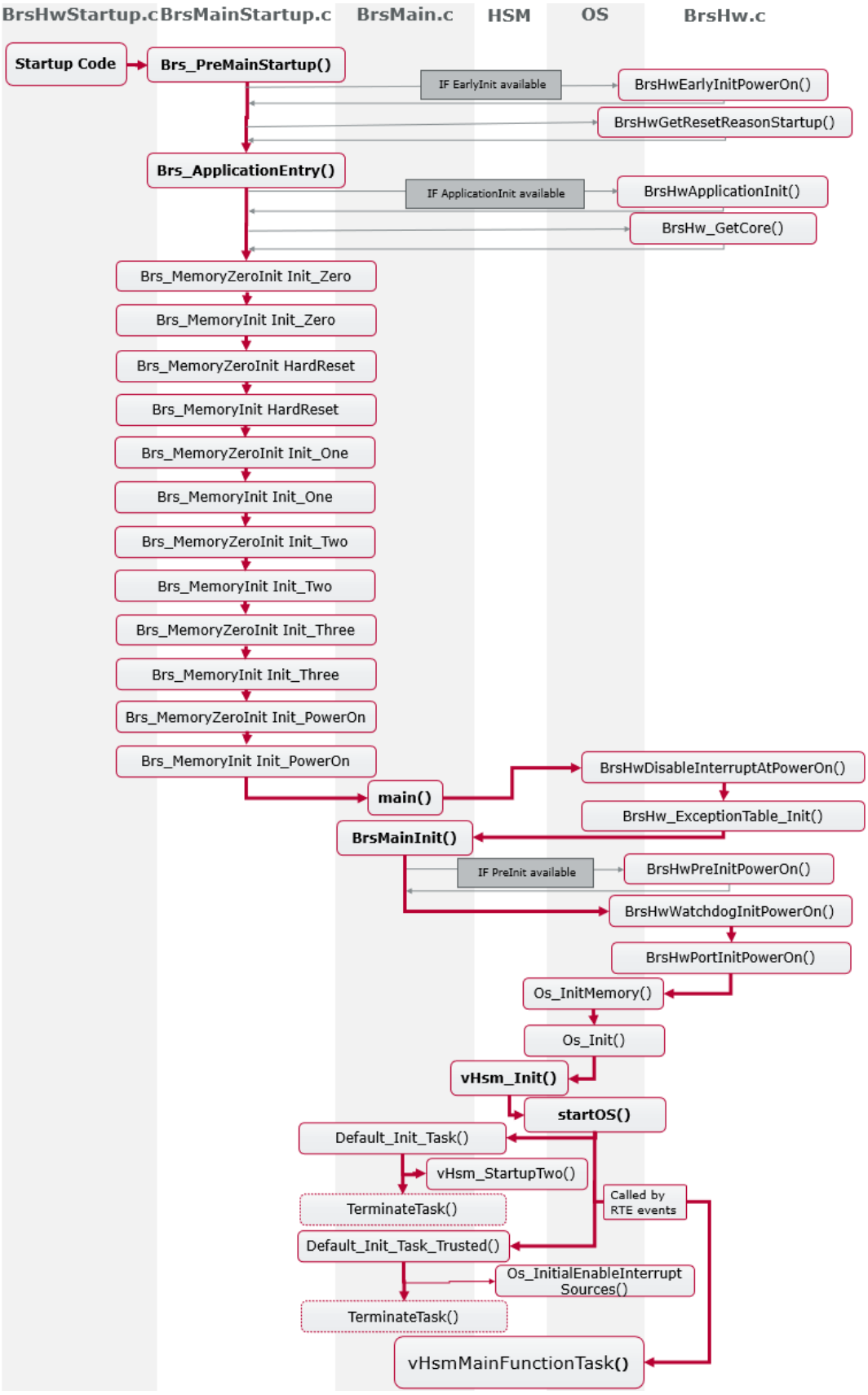


Also a MICROSAR Classic application could be called.



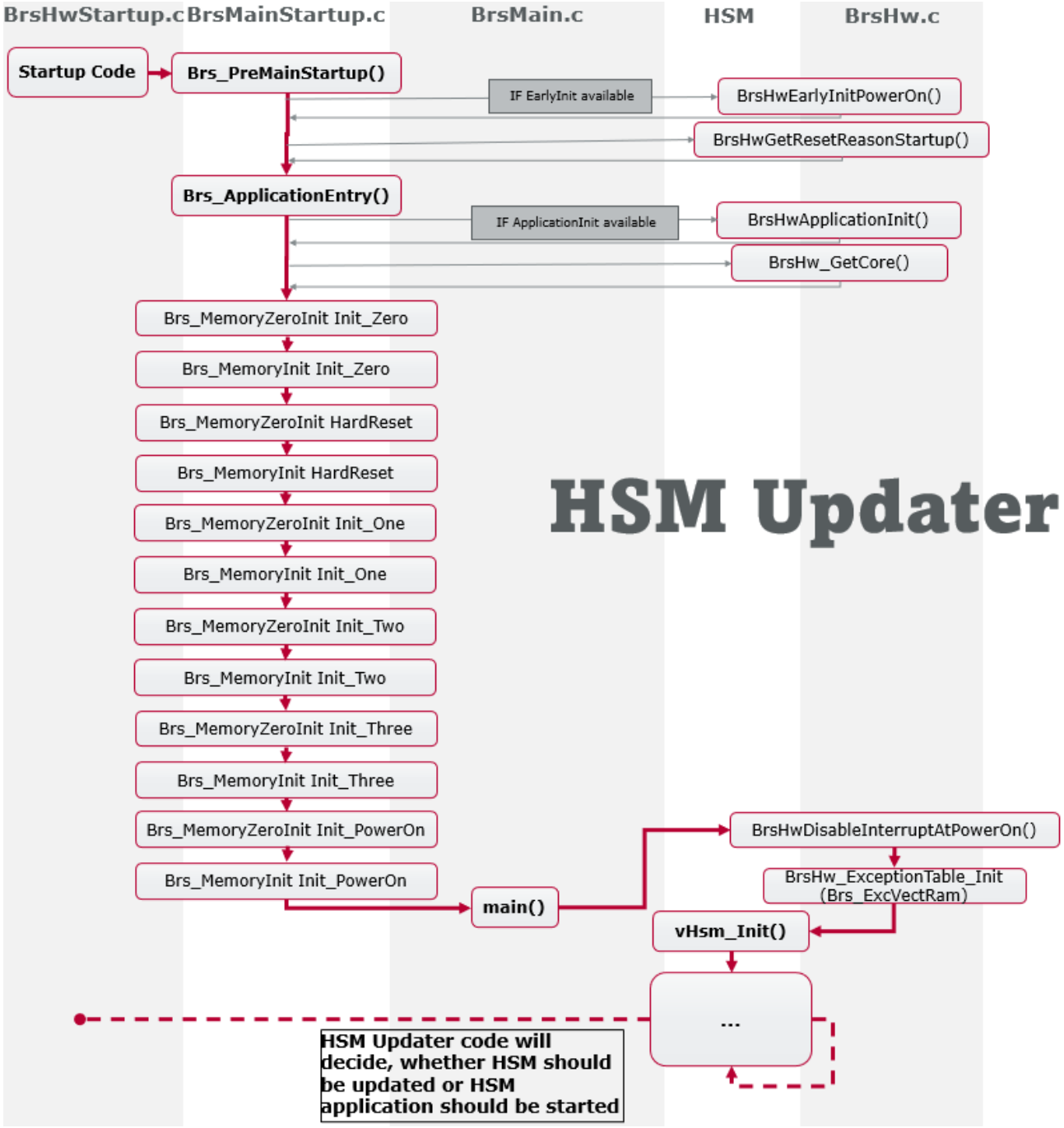
7.3 Sample for HSM

This shows a sample for a default program flow of a Hardware Security Module project.



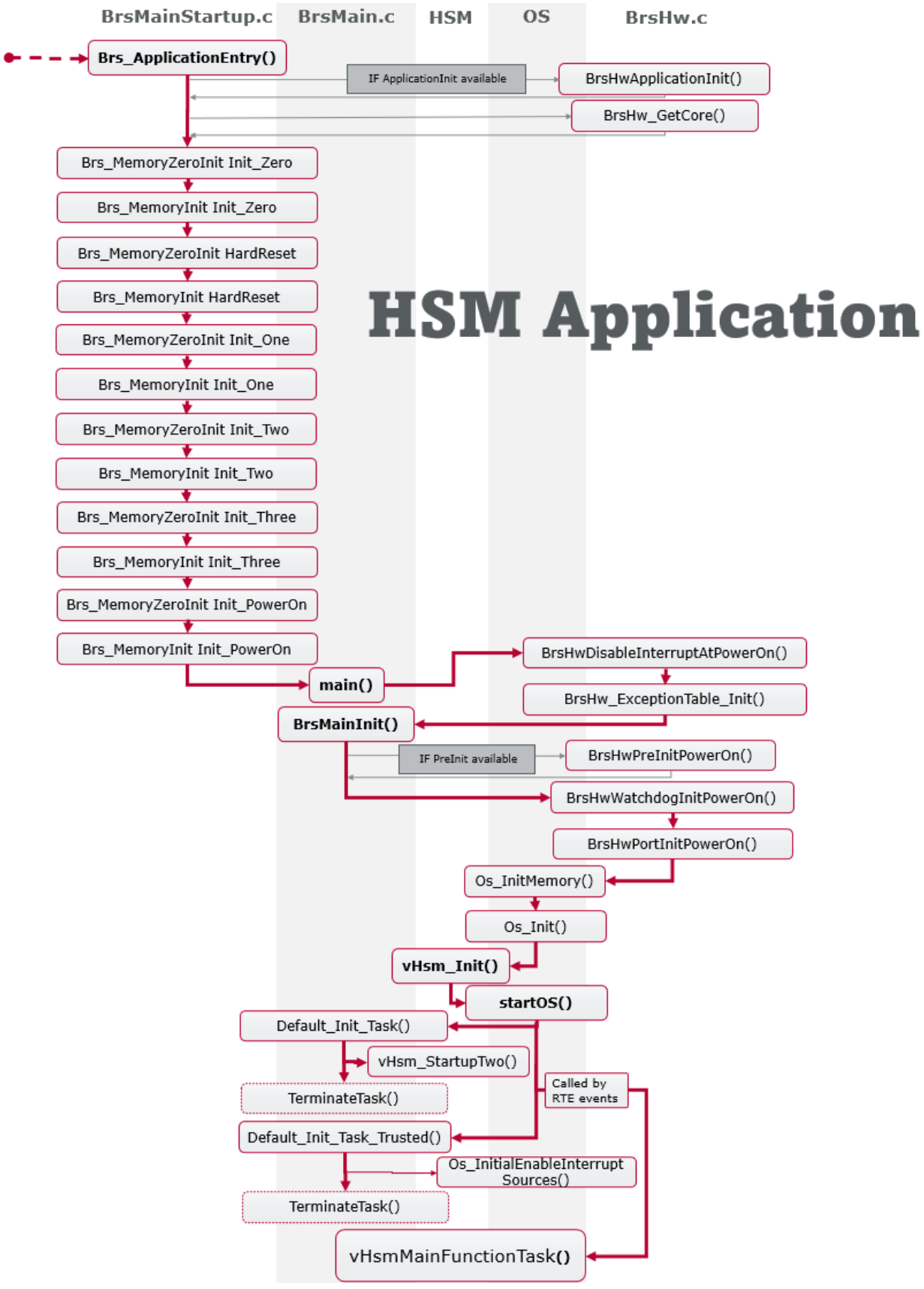
7.3.1 HSM Updater

This shows a sample for a default program flow of a Hardware Security Module project, including an Updater.



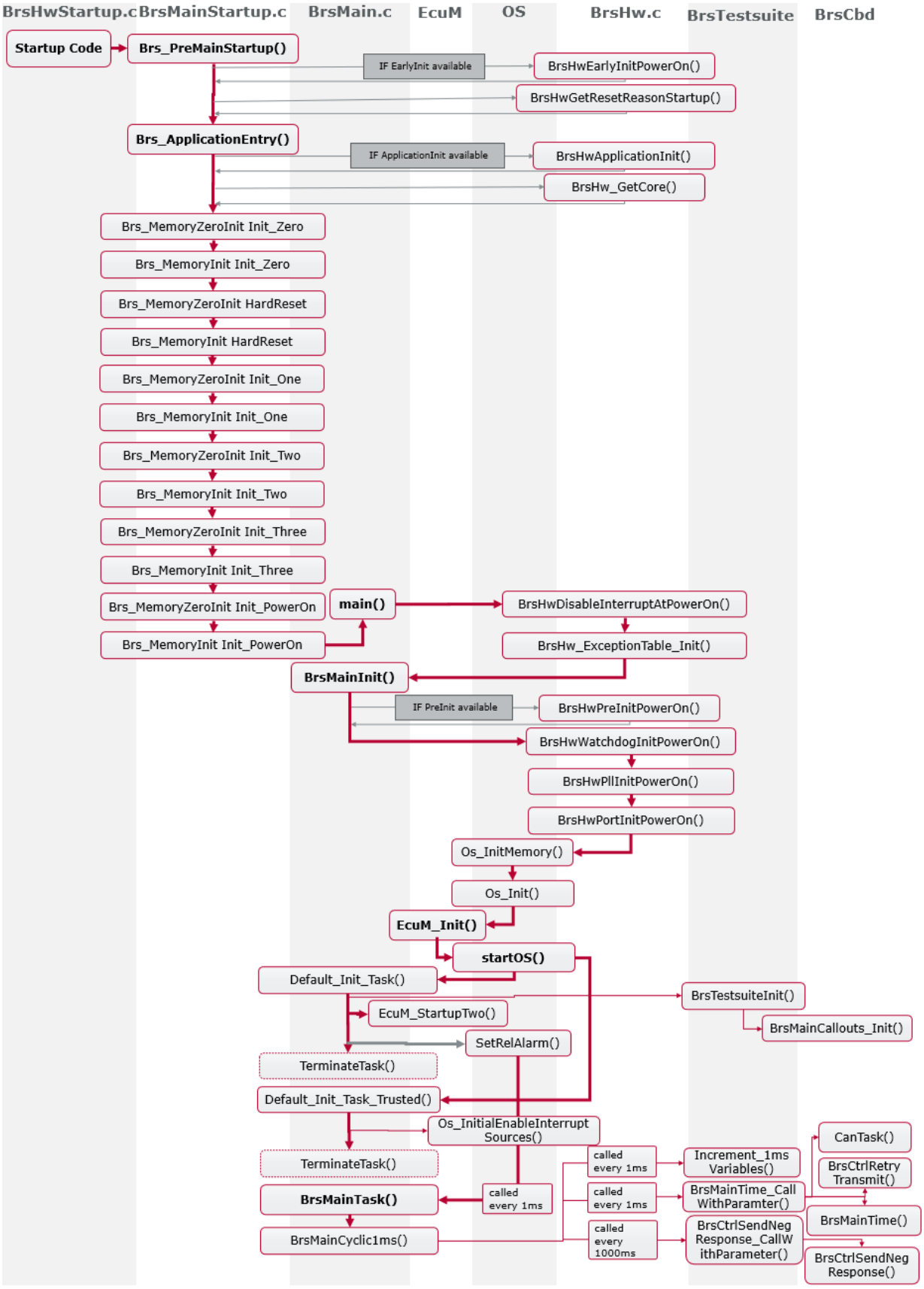
The HSM Updater code holds the ApplicationEntry adress of the HSM application instance.

The HSM application itself is started after the HSM Updater decided that no update is necessary or after the update was finished.



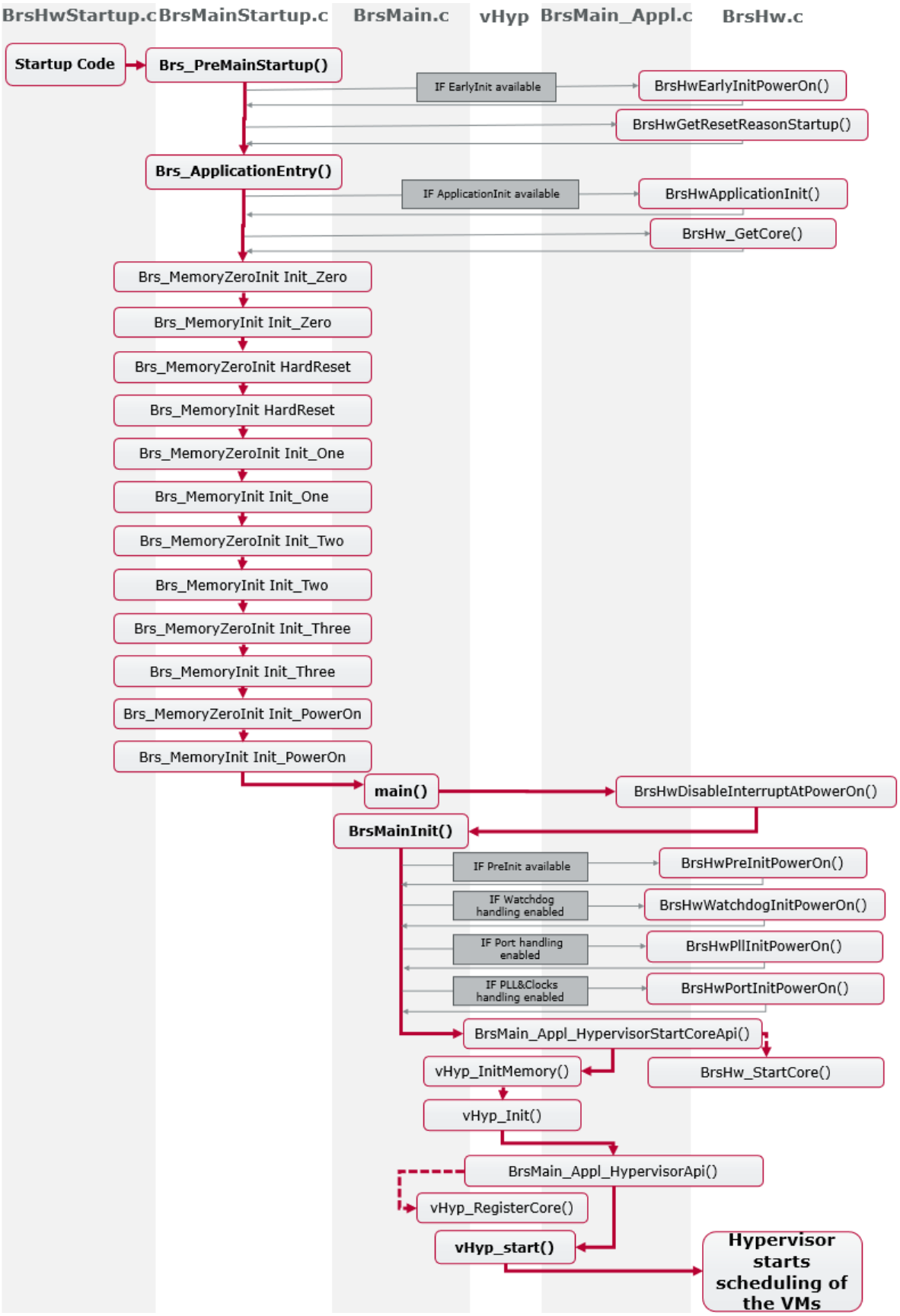
7.4 Sample for CANbedded

This shows a sample for a default program flow of a CANbedded project.



7.5 Sample for Hypervisor host instance

This shows a sample for a default program flow of a Hypervisor host instance project.



8 vLinkGen

MICROSAR vLinkGen is configured with Vectors DaVinci Configurator.

The descriptions of all configuration attributes are described with tool tips within the configuration tool. They can easily be looked up during vLinkGen component configuration.

The following chapters gives some instructions how to configure the typical use cases in vLinkGen.

8.1 Linking of Linker Sections

The descriptions in this chapter assume that a C symbol (no matter if function, constant or variable) is assigned to a specific section in the source code via the AUTOSAR MemMap mechanism or manually (for example via `#pragmas`). The instructions refer only to the vLinkGen and the linking of sections in the memory layout.

To simplify the following descriptions, all specified BSWMD paths are based on the following root: `/MICROSAR/vLinkGen/vLinkGenMemLayout/`

8.1.1 Code or Constant Section

To place a constant section (with code or constant data) at a certain position in the memory layout, it must be configured in the Linker Sections (`vLinkGenLinkerSections`) as follows:

1. Create a new `vLinkGenLinkerCodeSection` or `vLinkGenLinkerConstSection` container.
2. Set the **Name** (`vLinkGenLinker*SectionName`) which will be the name of the section in the generated linker script. The **Short-Name** should correspond to the Name parameter since it is also generated for some compilers.
3. Optionally, set an **Alignment** (`vLinkGenLinker*SectionAlignment`) for the section. This will align the start address of the section accordingly. Note that this is not supported by all compilers.
4. Optionally, mark the section for **Fast Addressing** (`vLinkGenLinker*SectionFast`).
5. Optionally, specify **Filenames** (`vLinkGenLinker*SectionFiles`) for the section. This could be the name of one or more files or archives (libraries) - with the files they contain. This instructs the linker to link only the section contents from these files. Please refer to the parameter description in the configuration tool for details. A valid syntax would be for example: “`filea.o;lib1.a(file*.o,file_b.o);file_?.o`”.
6. Optionally, set some **Compiler Specific Flags** (`vLinkGenLinker*SectionFlag`) for the section. If there are any available. These are compiler specific, and their meaning must be taken from the corresponding manual.

7. After the section is configured, it must be assigned to a CONST Section Group (`vLinkGenLogicalConstGroup/vLinkGenConstSectionGroup`). This group determines where the section ends up in the memory layout.
8. To do this, the Section must be assigned to the **Sections** (`vLinkGenConstSectionGroupRef`) of a group.
9. The actual memory area in which the group is then placed is determined by the **Memory Region Block** (`vLinkGenConstGroupRegion`).
10. If this section is to be loaded into RAM at ECU startup, the Section Group must be configured accordingly. Details can be found in chapter 0 of the vLinkGen Technical Reference Manual.
11. After generating, the section will appear in the corresponding Section Group in the linker script.

8.1.2 Data or Variable Section

To place a section with variable data (to be initialized or not) at a certain position in the memory layout, it must be configured in the Linker Sections (`vLinkGenLinkerSections`) as follows:

1. Create a new `vLinkGenLinkerVarSection` container.
2. Set the **Name** (`vLinkGenLinkerVarSectionName`) which will be the name of the section in the generated linker script. The **Short-Name** should correspond to the Name parameter since it is also generated for some compilers.
3. Optionally, set an **Alignment** (`vLinkGenLinkerVarSectionAlignment`) for the section. This will align the start address of the section accordingly. Note that this is not supported by all compilers.
4. Optionally, mark the section for **Fast Addressing** (`vLinkGenLinkerVarSectionFast`).
5. Optionally, specify **Filenames** (`vLinkGenLinker*SectionFiles`) for the section. This could be the name of one or more files or archives (libraries) - with the files they contain. This instructs the linker to link only the section contents from these files. Please refer to the parameter description in the configuration tool for details. A valid syntax would be for example:
   “`filea.o;lib1.a(file*.o,file_b.o);file_?.o`”.
6. Optionally, set some **Compiler Specific Flags** (`vLinkGenLinkerVarSectionFlag`) for the section. If there are any available. These are compiler specific, and their meaning must be taken from the corresponding manual.
7. After the section is configured, it must be assigned to a VAR Section Group (`vLinkGenLogicalVarGroup/vLinkGenVarSectionGroup`). This group determines where the section ends up in the memory layout.
8. To do this, the Section must be assigned to the **Sections** (`vLinkGenVarSectionGroupRef`) of a group.

9. The actual memory area in which the group is then placed is determined by the **Memory Region Block** (`vLinkGenVarGroupRegion`).
10. If this section is to be initialized with predefined values or zero at ECU startup, the Section Group must be configured accordingly. Details can be found in chapter 8.2.2 of the vLinkGen Technical Reference Manual.
11. After generating, the section will appear in the corresponding Section Group in the linker script.

8.1.3 Sections with Special Values

Normal sections are generated by the vLinkGen in the way the compiler expects (e.g. `*(*.<Section>)` for GCC). Sometimes it is necessary to bypass this syntax and generate a section the way it is configured. For this vLinkGen provides the Special Sections (`vLinkGenLinkerSpecSection`). Please refer to chapter 3.5 “Special Section Generation” of the vLinkGen Technical Reference to see how they are generated for each compiler.



Note
Since the value is generated as it is configured, it must contain a valid syntax for the compiler used.

To generate such a Spec Section, it must be configured as follows:

1. Create a new `vLinkGenLinkerSections/vLinkGenLinkerSpecSection` container.
2. Set the **Value** (`vLinkGenLinkerSpecSectionValue`) to the expected value to be generated. Consider the compiler specific syntax.
3. Optionally, set an **Alignment** (`vLinkGenLinkerSpecSectionAlignment`) for the section. This will align the start address of the section accordingly. Note that this is not supported by all compilers.
4. Optionally, set some **Compiler Specific Flags** (`vLinkGenLinkerSpecSectionFlag`) for the section. If there are any available. These are compiler specific and their meaning must be taken from the corresponding manual.
5. Unlike the other section types, Spec Sections can be assigned to CONST or VAR Section Groups. The assignment is done in the same way via the corresponding **Sections** (`vLinkGen*SectionGroupRef`) parameter of a group.
6. After generating, the section will appear in the corresponding Section Group in the linker script.

8.2 Memory initialization Initialization of ECC Protected Memory

Mainly needed for ECC protected memory, vLinkGen, together with BrsMain, provides the possibility to initialize memory. The memory region blocks and/or section groups, to be initialized, are configured in vLinkGen and BrsMain will perform the initialization accordingly. The init stage “Early” is handled in Startup code (mainly for initialization of startup stack) and the other init stages are handled in BrsMainStartup.

Init Stage	Description
`EARLY`	Initialize always directly after reset. Shall only be used if really necessary. Since the PLL might not be initialized at that time, the initialization of large areas may take a long time.
`ZERO`	Always initialized first during the default vBRS startup routine.
`HARD_RESET_ONLY`	Same initialization stage as ONE. However, initialization is only done if a hardware reset occurred.
`ONE`	Always initialized second during the default vBRS startup routine.By default, this value should be used if an initialization is required.
`TWO, THREE`	Optional init stages, which are initialized accordingly after the `HARD_RESET_ONLY` stage. Shall only be used if really necessary. One use case might be to reset the Wdg between the initialization of two stages.
`POWER_ON`	Initialized to zero or NULL-pointer only on Cold PowerOn reset or brownout reset. No initialization on a regular warm reset (e.g. software reset, watchdog reset, external reset).

8.2.1 Initialization of complete memory region blocks

As described in chapter 2.1.4.1 “ECC Memory Initialization” of the vLinkGen Technical Reference Manual it is possible to initialize ECC protected memory areas. This can be configured for individual Memory Region Blocks (`vLinkGenMemoryRegion/vLinkGenMemoryRegionBlock`) as follows:

1. Set the **Init Stage** (`vLinkGenMemoryRegionBlockInitStage`) to a value other than `NONE`.
2. Set the **Init Core ID** (`vLinkGenMemoryRegionBlockInitCore`) to the core which shall perform the initialization. The value must be the logical ID of the core.
3. vLinkGen then generates an entry for this block in the corresponding initialization table. For details about the generated initialization tables, please refer to chapter 2.1.4 “Initialization Tables” in the vLinkGen Technical Reference Manual.

8.2.2 Initialization of VAR Section Group

As described in chapter 2.1.4.2 “Variable Initialization” in the vLinkGen Technical Reference Manual, it is possible to initialize variables on ECU startup with either predefined values or zeros. To initialize a VAR Section Group (`vLinkGenLogicalVarGroup/vLinkGenVarSectionGroup`) it must be configured as follows:

1. Set the **Init Policy** (`vLinkGenVarSectionGroupInit`) to a value other than `NONE`.

INIT	Initialization values for variables are flashed to the ROM and then copied to RAM at ECU startup.
ZERO_INIT	Variables in RAM are written with zeros at ECU startup.
NONE	No initialization shall be performed for the Section Group.

2. Set the **Init Stage** (`vLinkGenVarSectionGroupInitStage`) to a value other than `NONE`. The different values determine the time of initialization during startup.
3. Switch to the parent Logical Group.
4. If Init Policy is set to `INIT`, the **Memory Region Block – ROM Copy** (`vLinkGenVarGroupRomRegion`) must be set. This is used to specify where the initialization values shall be in ROM.
5. Set the **Init Core ID** (`vLinkGenVarGroupInitCore`) to the core which shall perform the initialization. The value must be the logical ID of the core.
6. The vLinkGen then generates entries in the corresponding initialization table. Refer to chapter 2.1.4 “Initialization Tables” in the vLinkGen Technical Reference Manual for details.



Note
The settings of the **Memory Region Block – ROM Copy** and the **Init Core ID** apply to all Section Groups within a Logical Group. If Section Groups need to be initialized by different cores or their initialization values shall be located somewhere else in ROM, the groups need to be moved to different Logical Groups.

8.2.3 Copy Code to RAM

As described in chapter 2.1.4.3 “RAM Code Execution” in the vLinkGen Technical Reference Manual, it is possible to copy code or constants from a ROM memory area to RAM. To copy a CONST Section Group (`vLinkGenLogicalConstGroup/vLinkGenConstSectionGroup`) from ROM to RAM it must be configured as follows:

1. Set the **Init Policy** (`vLinkGenConstSectionGroupInit`) to `COPY_TO_RAM`.
2. Set the **Init Stage** (`vLinkGenConstSectionGroupInitStage`) to a value other than `NONE`.
3. Switch to the parent Logical Group.
4. Set the **Memory Region Block - RAM Copy** (`vLinkGenConstGroupRamRegion`) to specify where the copy shall be in RAM.
5. Set the **Init Core ID** (`vLinkGenConstGroupInitCore`) to the core which shall perform the initialization. The value must be the logical ID of the core.

6. The vLinkGen then generates entries in the corresponding initialization table. Refer to chapter 2.1.4 “Initialization Tables” in the vLinkGen Technical Reference Manual for details.



Note
The settings of the **Memory Region Block – RAM Copy** and the **Init Core ID** apply to all Section Groups within a Logical Group. If Section Groups need to be copy to different RAM locations, the groups need to be moved to different Logical Groups.

8.3 User Defined Linker Symbols

The vLinkGen also provides the possibility to configure global Linker Symbols compiler independently. That means it is possible to assign values to own symbols, and vLinkGen will generate these assignments in the appropriate syntax into the linker script. These symbols are configured within Logical Symbol Groups (`vLinkGenLogicalSymbolGroup`) as follows:

1. Create a new `vLinkGenLogicalSymbolGroup` container.
2. Set the **Position** (`vLinkGenSymbolGroupPosition`) of the group to specify their location within the linker script. Note that some compilers expect global symbols to be defined at a certain position in the linker script. In this case the position is simply ignored.
3. Optionally, it is possible to assign the group to one or more **Variants** (`vLinkGenSymbolGroupVariant`). More about Variants in chapter 2.1.2 “Variant Handling” in the vLinkGen Technical Reference Manual.
4. Create one or more `vLinkGenLinkerSymbol` containers within the Logical Group.
5. These symbols get a **Name** (`vLinkGenLinkerSymbolName`) and a **Value** (`vLinkGenLinkerSymbolValue`). With some compilers the values can also contain complex expressions.
6. After generating, the symbols appear in the corresponding syntax at the defined position in the linker script.



Caution
Some compilers do not support the definition of global Linker Symbols at all or only to a limited extent. This should be considered when using Linker Symbols. However, vLinkGen will issue warnings if a compiler does not support them or only to a limited extent.

8.4 Variant Handling

As described in chapter 2.1.2 “Variant Handling” in the vLinkGen Technical Reference Manual, vLinkGen provides the possibility to specify different variants of a linker script in one configuration. These variants can be added to the vLinkGen configuration as follows:

1. Add the optional `vLinkGenVariantHandling` container to the configuration.
2. The different variants can then be defined with the `vLinkGenVariant` sub-containers. Where each container reflects one variant. The name of the container is also the name of the variant.
3. The created variants can now be referenced by Logical Groups (`vLinkGenMemLayout/vLinkGenLogical*Group`) using the **Variants** (`vLinkGen*GroupVariant`) reference. This parameter allows the group to be assigned to one or more variants.
4. Finally, the **File Generation** shall be set accordingly. Please refer to chapter 2.1.1 “File Generation” in the vLinkGen Technical Reference Manual for details. Some file generation settings (like STANDARD) are ignoring the Variants.



Expert Knowledge
Only one variant is needed, but still a build process without the Logical Groups of this variant. Then a second variant can be created without being referenced by a Logical Group. Which then must only be defined in the corresponding build process.

8.4.1 Variant Specific Region Block Mappings

As described in chapter 2.1.2.1 “Variant Specific Mappings” in the vLinkGen Technical Reference Manual, vLinkGen provides the possibility to link certain CONST or VAR Logical Groups (respectively all their contained Section Groups) to different Hardware Memory Areas in different variants. These variant specific mappings can be added to the vLinkGen configuration as follows:

1. Add `vLinkGenVariantHandling` to the configuration as described in chapter 8.4. Create variants and reference them in Logical Groups (`vLinkGenLogical*Group`) as needed.
2. Navigate to the Logical Group (CONST or VAR) which Section Groups shall be linked to different Hardware Memory Areas and add the optional **Variant Handling** (`vLinkGen*GroupVariantHandling`) container.
3. Create one or more **Variant Specifics** (`vLinkGen*GroupVariantSpecifics`) container. One for each variant specific mapping. Where one Variant Specifics container can be used to change the mapping for multiple variants.
4. Optionally, set the **Overwriting Memory Region Block** (`vLinkGen*GroupSpecificRegion`) reference to specify where the Logical Group shall be linked in the specified variants. This value overwrites the **Default Memory Region Block** (`vLinkGenConstGroupRegion`) from the parent Logical Group.

5. Optionally, set the **Overwriting Memory Region Block – ROM/RAM Copy** (`vLinkGen*GroupSpecific*Region`) reference to specify where the Logical Group's copy data shall be linked to in the specified variants. This value overwrites the **Default Memory Region Block – ROM/RAM Copy** (`vLinkGen*Group*Region`) from the parent Logical Group.
6. Select the **Variants** (`vLinkGen*GroupSpecificVariant`) to which this specific mapping is to apply. Multiple selection is allowed.
7. Finally, the **File Generation** shall be set accordingly. Please refer to chapter 2.1.1 "File Generation" in the vLinkGen Technical Reference Manual for details.



Note
The Overwriting Memory Region Block references described above are both optional. This means that for a variant, for example, only the ROM/RAM copy can be linked elsewhere.



Caution
Specified **Variants** (`vLinkGen*GroupSpecificVariant`) must also be added to the parent Logical Group Variants. Normally this is ensured by the vLinkGen Generator.

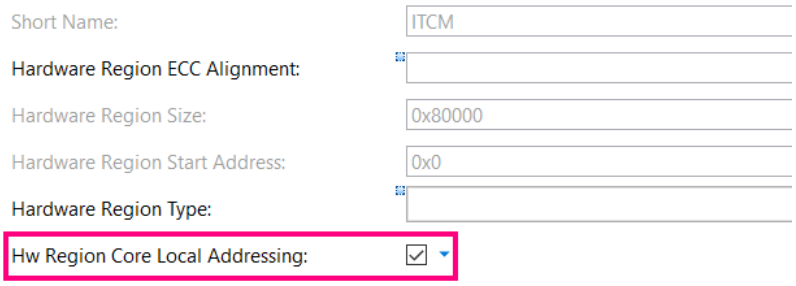
8.5 Using Hardware Regions with Core Local Addressing

If the used hardware provides memory ranges, that are accessed via core local address ranges (e.g. every core has core local RAM at the same address), the optional Boolean

`vBaseEnv/vBaseEnvGeneral/vBaseEnvDerivativeInformation/vBaseEnvMemLayoutHwRegion/vBaseEnvHwRegionCoreLocalAddressing`

must be created and enabled for the corresponding hardware region. The generated linker template file will then comprise the needed keywords to allow this overlapping functionality.

All `vBaseEnvMemLayoutHwRegion` containers with not existing or disabled `vBaseEnvHwRegionCoreLocalAddressing` parameter must not contain fully or partly overlapping memory ranges, regarding all other `vBaseEnvMemLayoutHwRegion` containers.



Caution

The following compiler/linker do not support such a feature and vLinkGen will show a warning message, if the parameter is used in a configuration with such a configured compiler:

- TexasInstruments
- Llvm TexasInstruments
- GccNxp
- IAR

The linker file is still able to be generated, but linker errors will most likely prevent you from linking it.

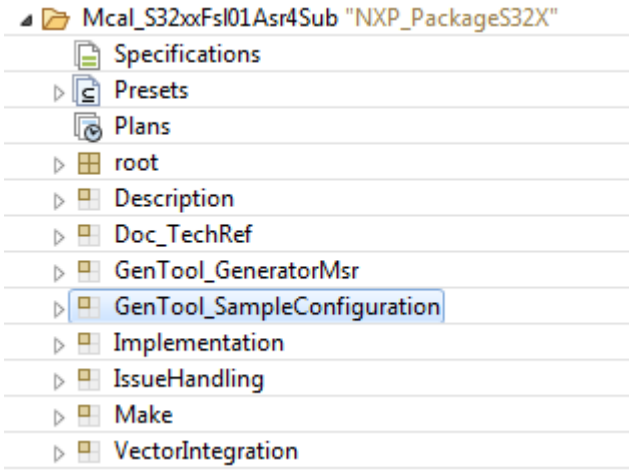
9 MCAL sample configuration

If there is a MCAL included in the vBaseEnv platform package, then there is automatically a sample configuration provided. Please note that this sample configuration is just tested with the specified derivative, compiler, AUTOSAR version and MCAL version. You might have to adapt it to fit your use case.

The naming of these sample configurations follows the following scheme:

```
Platform_Derivative_AutosarVersion_MCALVersion.ecuc.arxml
S32xx_V344_AR43_V044.ecuc.arxml
```

The sample configuration is stored in the MIP (MCAL integration package) in the subpackage “GenTool_SampleConfiguration”.



Inside the delivery structure, the file is located at internal\SampleMCALConfigs and can be imported to your configuration tool setup.

To import the sample configuration into your Cfg5 project please follow these steps:

1. Go to menu File → Import
2. Click at the plus button and select the sample configuration from your hard disk.
3. Click next and select the modules you want to import. For a basic run just select the Wdg, Port and MCU. If you are going to use the LED toggle mechanism, also select DIO.
4. Click next and the modules should be imported into your configuration.

